



数字逻辑与处理器基础

Fundamentals of Digital Logic and Processor



第七讲 流水线技术

第1部分 基本概念、数据通路、冒险

李学清

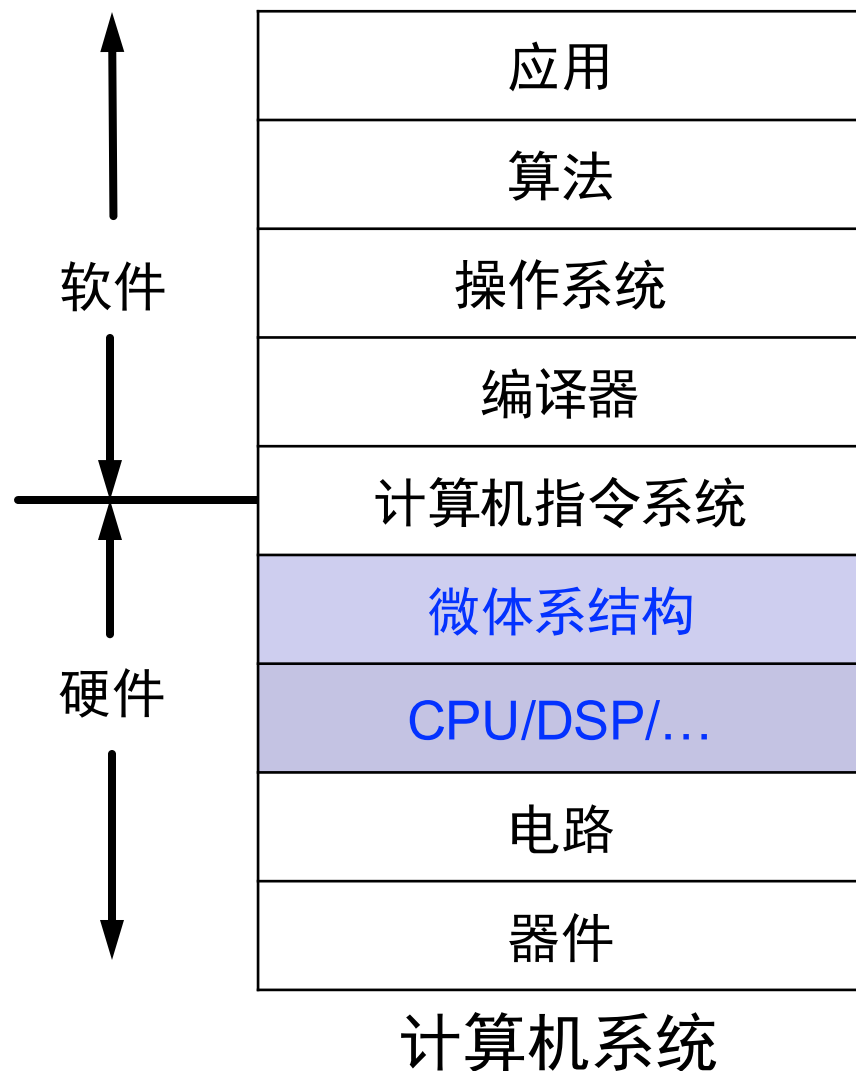
#腾讯会议： 453-1488-6548

助教： 唐文骏*/陈仲豪/曾令安

*本节内容负责答疑助教

2024年11月25日

微体系结构



周	日期	暂定主要内容
1	9/9	绪论
2	9/21	布尔代数 (中秋节调休)
3	9/23	组合逻辑
4	9/30	时序逻辑
5	10/7	国庆节放假
6	10/14	时序逻辑
7	10/21	计算机指令系统
8	11/3	期中考试
9	11/4	计算机指令系统
10	11/11	微处理器
11	11/18	微处理器
12	11/25	流水线
13	12/2	流水线
13	12/9	流水线/存储器
15	12/16	存储器
16	12/23	存储器/IO与总线/总结

本讲学习目标

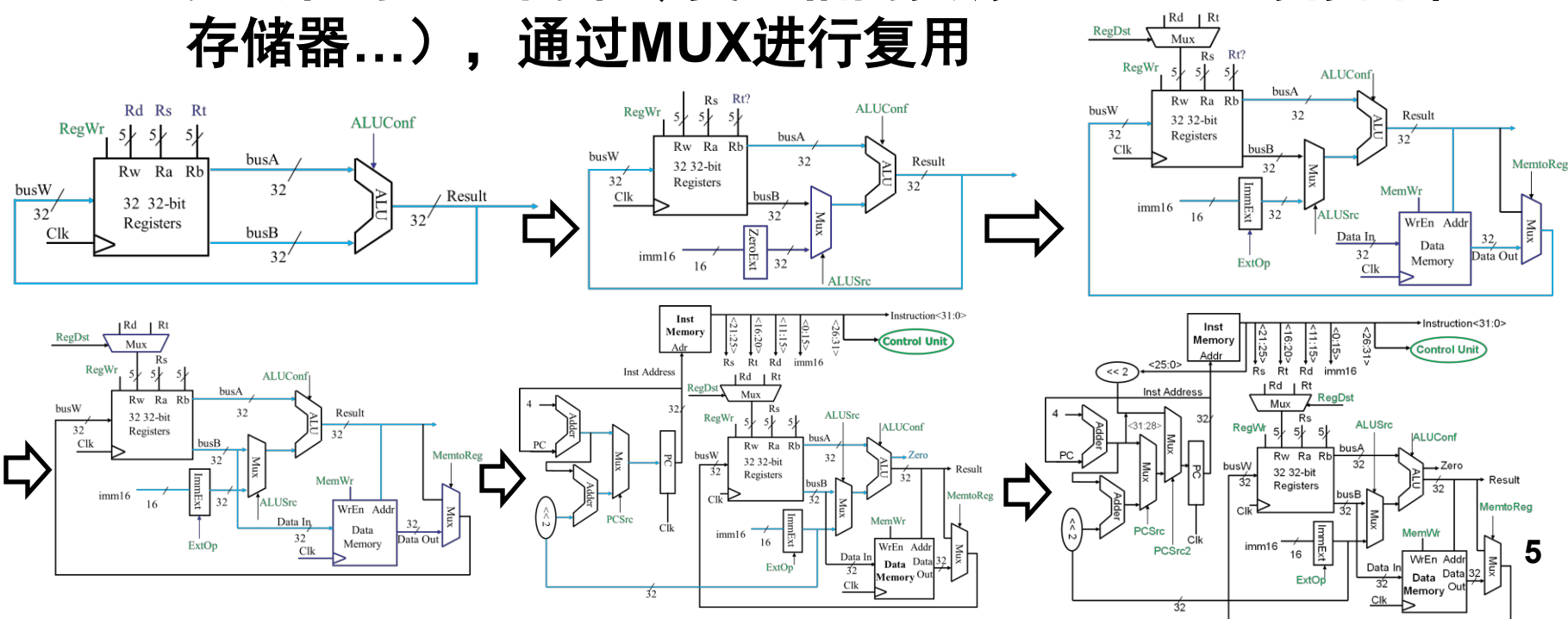
- **知识点：概念、原理和指标**
 - 单周期、多周期、流水线处理器的区别
 - 流水线的基本概念、数据通路（执行过程）
 - 流水线中的冒险起因与分类
- **能力**
 - 掌握设计和分析流水线处理器的方法
 - 掌握分析和解决流水线中冒险的方法
- **思想**
 - 时空复用
 - 并行加速

本节课目录

- 复习回顾与流水线基本概念 P5
- 流水线数据通路 P42
- 流水线的冒险 P89
 - 结构冒险
 - 数据冒险
 - 控制冒险
- 数据通路的实现（考虑冒险）

回顾：单周期处理器

- 单周期处理器的设计过程：
 - 分析指令集需求
 - 确定数据通路的组成模块：寄存器堆、存储器、ALU...
 - 根据指令组装数据通路：关键路径&资源分配
 - 控制信号：不同指令复用相同资源（ALU、寄存器堆、存储器...），通过MUX进行复用



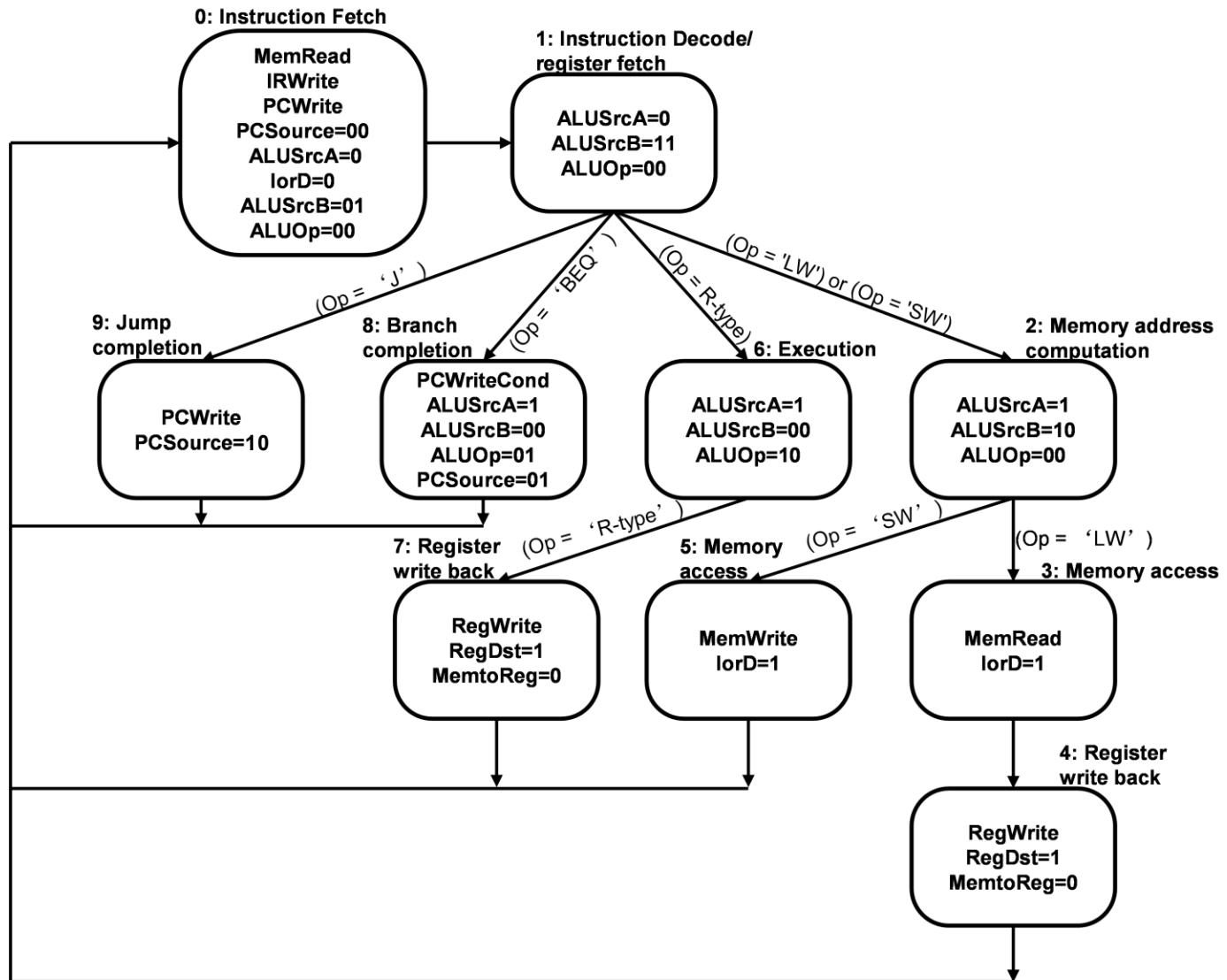
回顾：多周期实现

• 各种指令的执行步骤和控制信号

- ALU指令：4 cycles
- lw指令：5 cycles
- sw指令：4 cycles
- 分支指令：3 cycles
- 跳转指令：3 cycles

步骤/指令类型	R型指令	内存访问指令	分支指令	跳转指令
取指令	IR <= MemInst[PC] PC <= PC+4			
译码 & 寄存器堆读取	A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]] ALUOut <= PC + signext(IR[15:0]) << 2			
执行 & 分支/跳转完成	ALUOut <= A op B	ALUOut <= A + signext(IR[15:0])	if (A==B) PC <= ALUOut	PC <= {PC[31:28], IR[25:0], 2'b00}
内存访问 & R型指令完成	Reg[IR[15:11]] <= ALUOut	Load: MDR <= MemData[ALUOut] Store: MemData[ALUOut] <= B		
内存		Load: Reg[IR[20:16]] <= MDR		

回顾：多周期状态转移



回顾：多周期处理器

- 单周期 → 多周期：
 - 单周期：CPI = 1；各指令关键路径不均衡
 - 多周期：CPI > 1；指令分步骤执行，由状态机控制
- 多周期有限状态机：
 - 现态：当前的执行阶段/步骤
 - 次态方程：根据现态（以及opcode）决定次态
 - 输出方程：根据现态给出控制信号

回顾：CPU的性能

- CPU的性能：

$$CPU\text{执行时间} = \text{指令数} \times CPI \times \text{时钟周期}$$

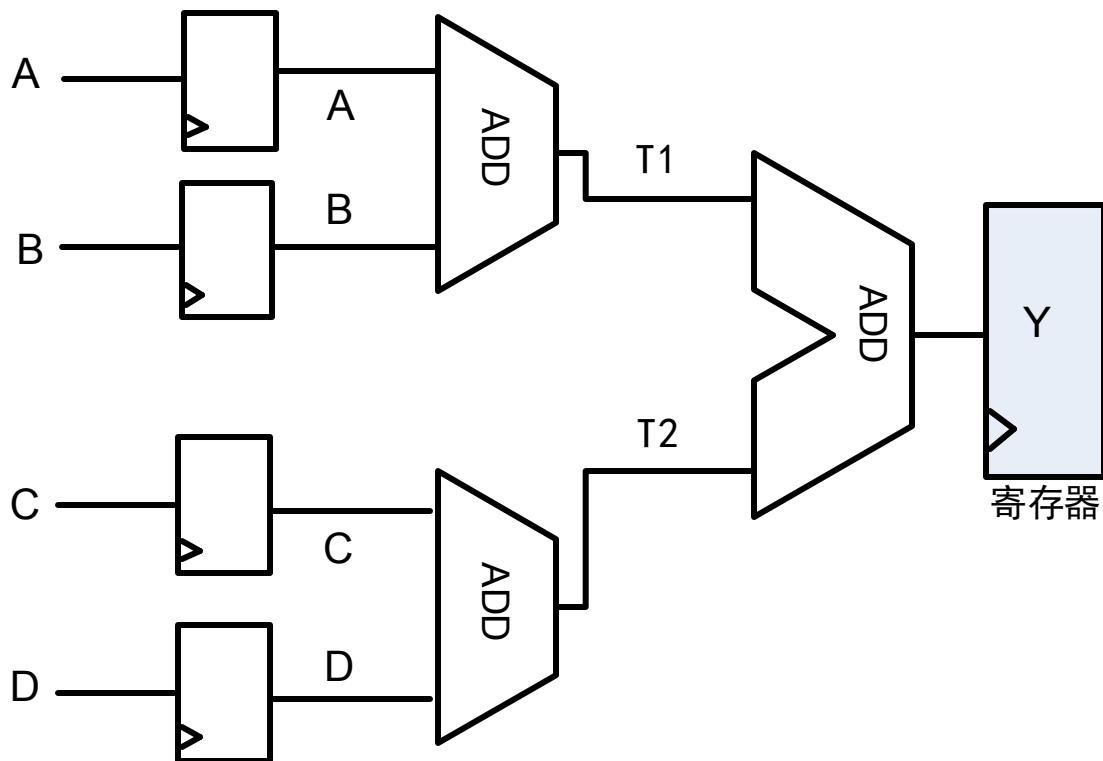
- 主要手段：

- 减小指令数
- 减小CPI x 时钟周期

工作时间/空闲时间：处理电路的“忙碌”程度

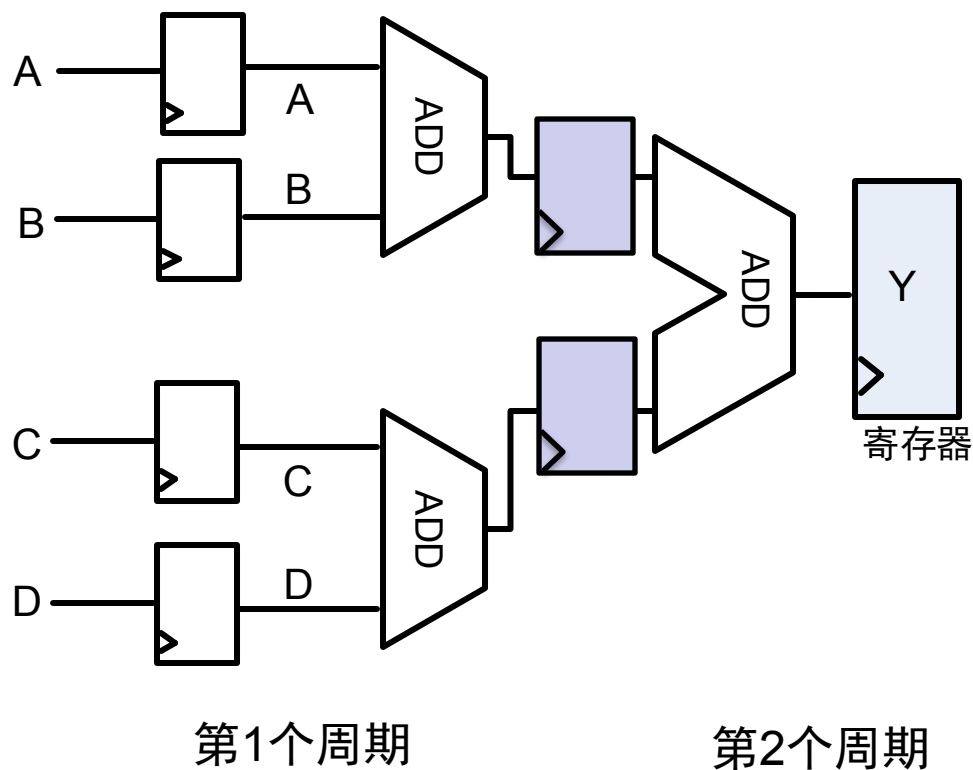
举例：单周期到多周期

- 算法： $Y=A+B+C+D$ ，4个32位无符号数的相加
- 单周期的做法：



举例：单周期到多周期

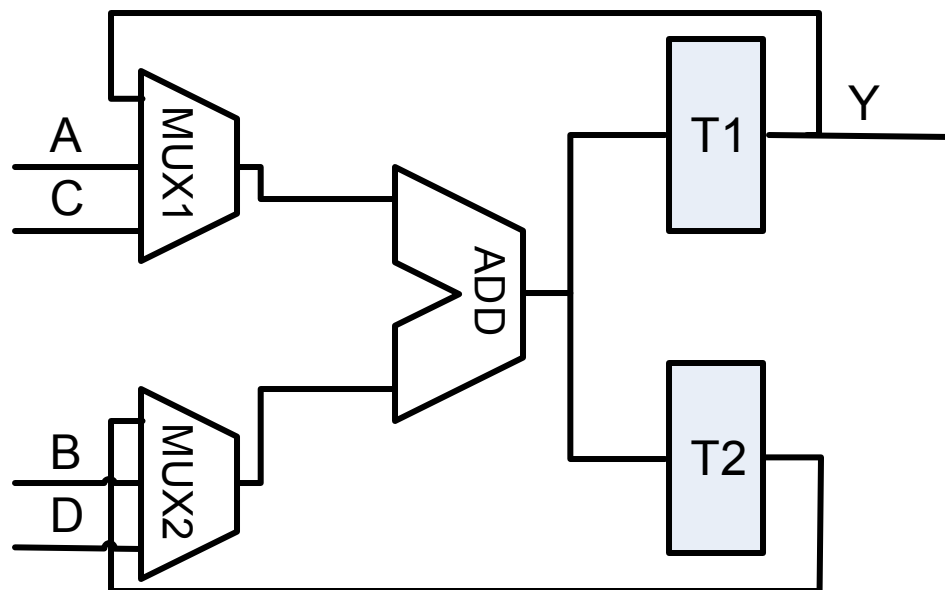
- 算法： $Y=A+B+C+D$ ，4个32位无符号数的相加
- 一种多周期的做法：



单周期到多周期

- 多周期的资源复用：
 - 周期调度
 - 资源绑定（加法器，寄存器）

周期	操作
1	$T1 = A + B$
2	$T2 = C + D$
3	$Y = T1 + T2$

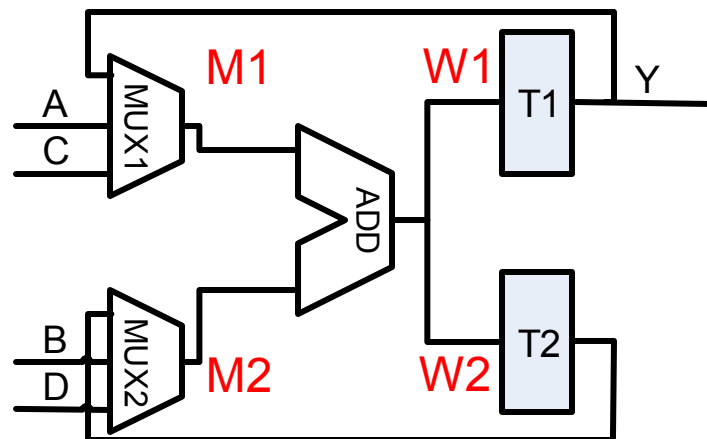


单周期到多周期

- 多周期的资源复用：
 - 周期调度
 - 资源绑定（加法器，寄存器）
 - T2只有一个周期被使用？

周期	资源绑定	控制
1	T1=Add1(A, B)	M1=01; M2=01; W1=1; W2=0
2	T2=Add1(C, D)	M1=10; M2=10; W1=0; W2=1
3	T1=Add1(T1, T2)	M1=00; M2=00; W1=1; W2=0

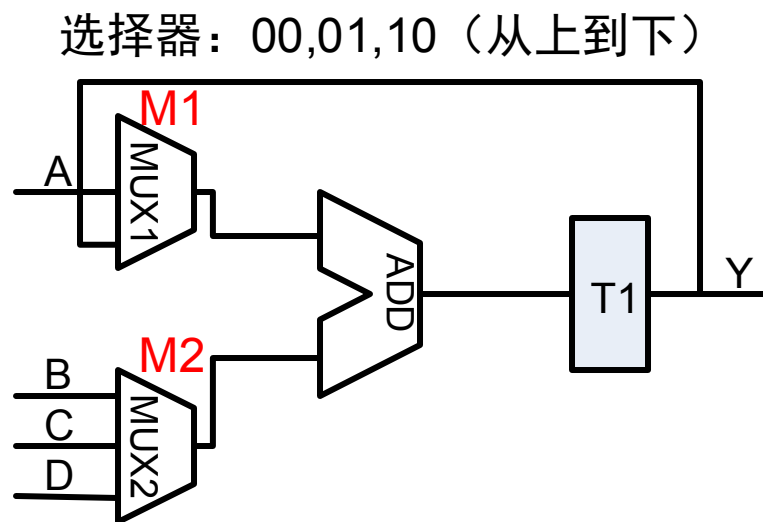
选择器：00,01,10（从上到下）
寄存器：W1,W2（分别为T1、T2的使能信号）



单周期到多周期

- 多周期的资源复用
 - 周期调度
 - 资源绑定（加法器，寄存器）
 - ✓ 寄存器数量降低至1个

周期	资源绑定	控制
1	$T1 = \text{Add1}(A, B)$	$M1=0; M2=00;$
2	$T1 = \text{Add1}(T1, C)$	$M1=1; M2=01;$
3	$T1 = \text{Add1}(T1, D)$	$M1=1; M2=10;$



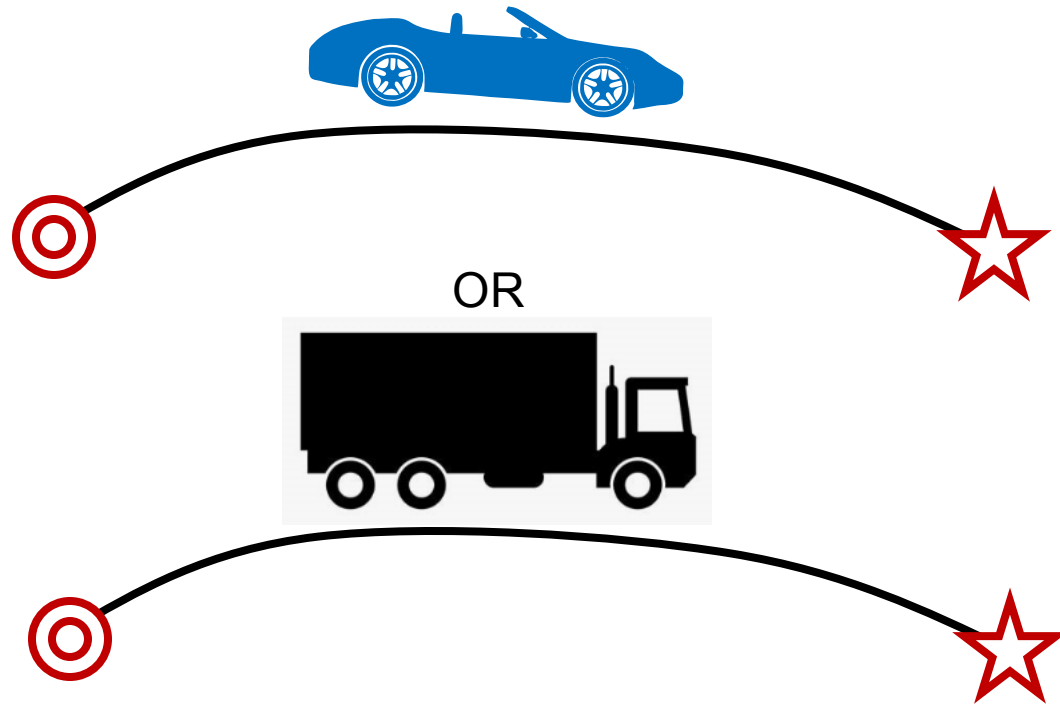
回顾：单周期与多周期方案的对比

$$CPU\text{执行时间} = \text{指令数} \times CPI \times \text{时钟周期}$$

- **CPI**：单周期处理器CPI=1，多周期处理器CPI>1
- **时钟周期**：单周期周期较长，多周期周期较短
- **单周期与多周期的共性**：一条指令完全执行结束，才能执行下一条指令，指令难以并行
- **如何进一步提高处理器性能？**

影响性能的因素

- 高速公路带来的启发

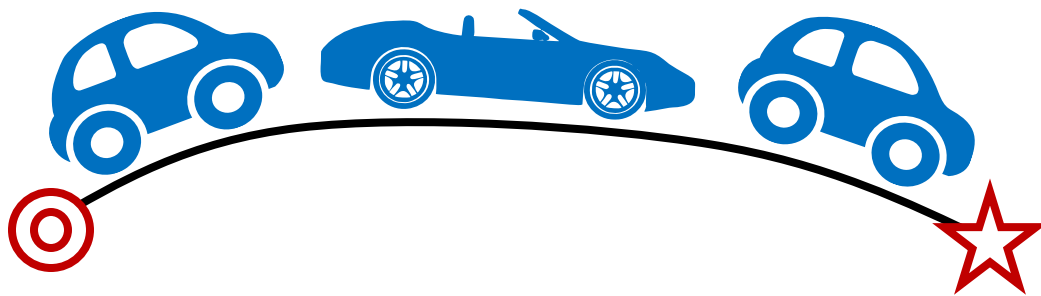


一次执行一条指令

- 一条路一辆车
- 采用固定的上路间隔

影响性能的因素

- 高速公路带来的启发



流水线执行指令

- 一条路多辆车
- 优点：效率提高
- 不同车速度不同？
- 依赖性？

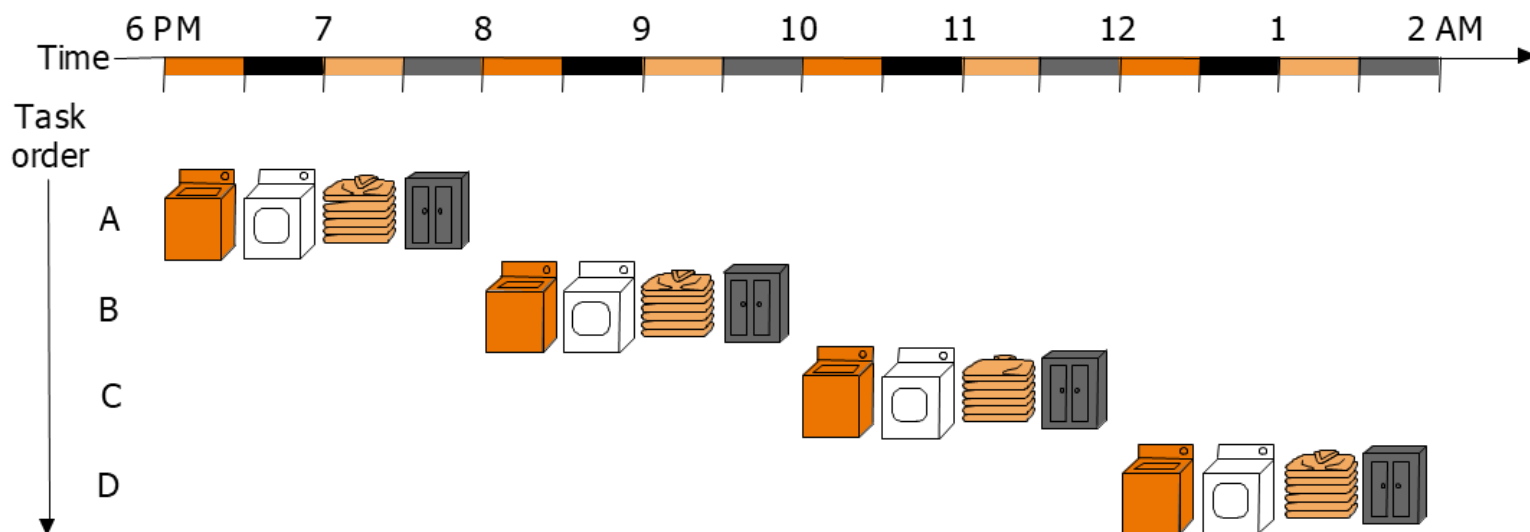
流水线的基本概念

• 生活中的实例——洗衣店

– 洗衣过程包括4个步骤：每个步骤需要30分钟

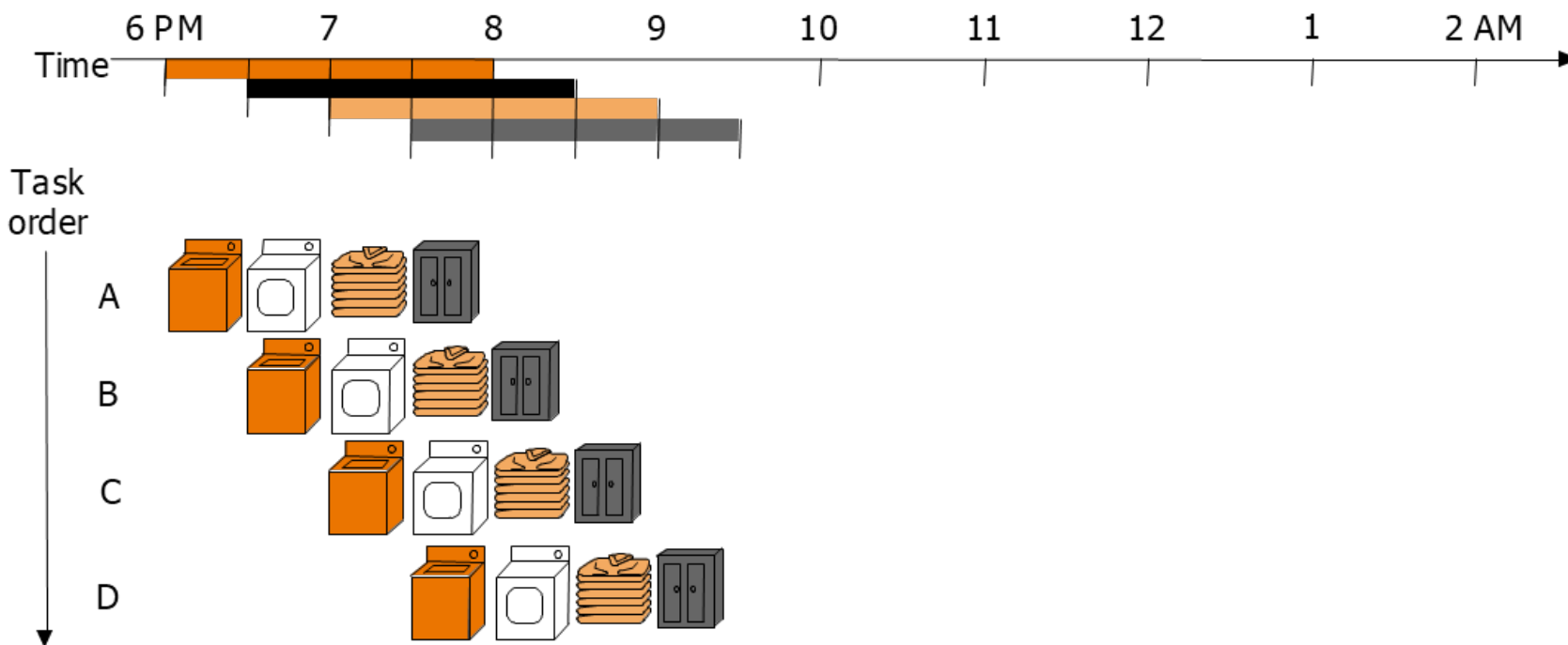
洗衣	脱水	折叠	储存
----	----	----	----

– 非流水线方式，洗4批衣服需要花费8个小时的时间



流水线的基本概念

- 生活中的实例——洗衣店
 - 采用流水线的方式，洗4批衣服只需要花费3.5小时



流水线的基本概念

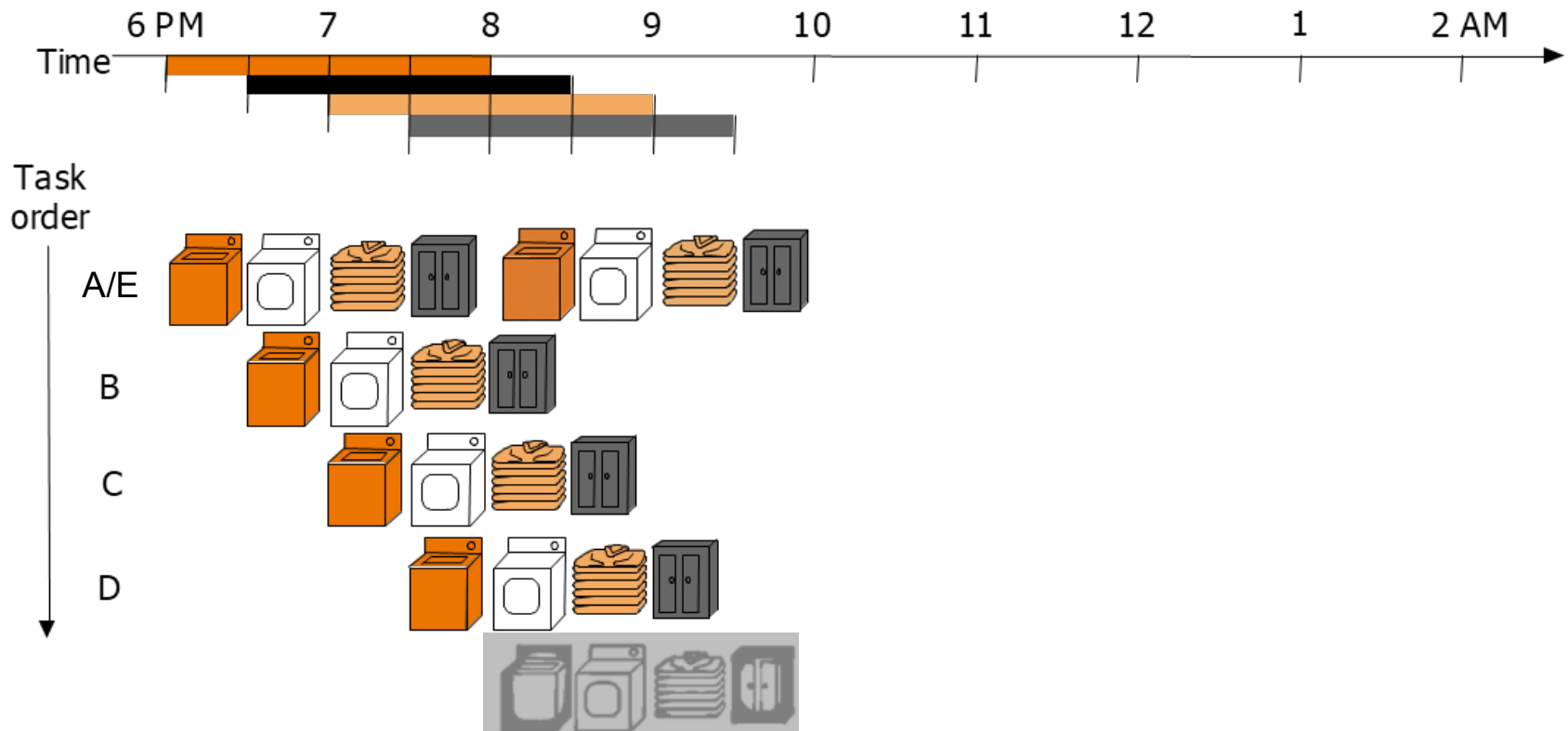
- 生活中的实例——洗衣店
 - 采用流水线的方式，洗4批衣服只需要花费3.5小时
 - 洗更多衣服？



流水线的基本概念

• 生活中的实例——洗衣店

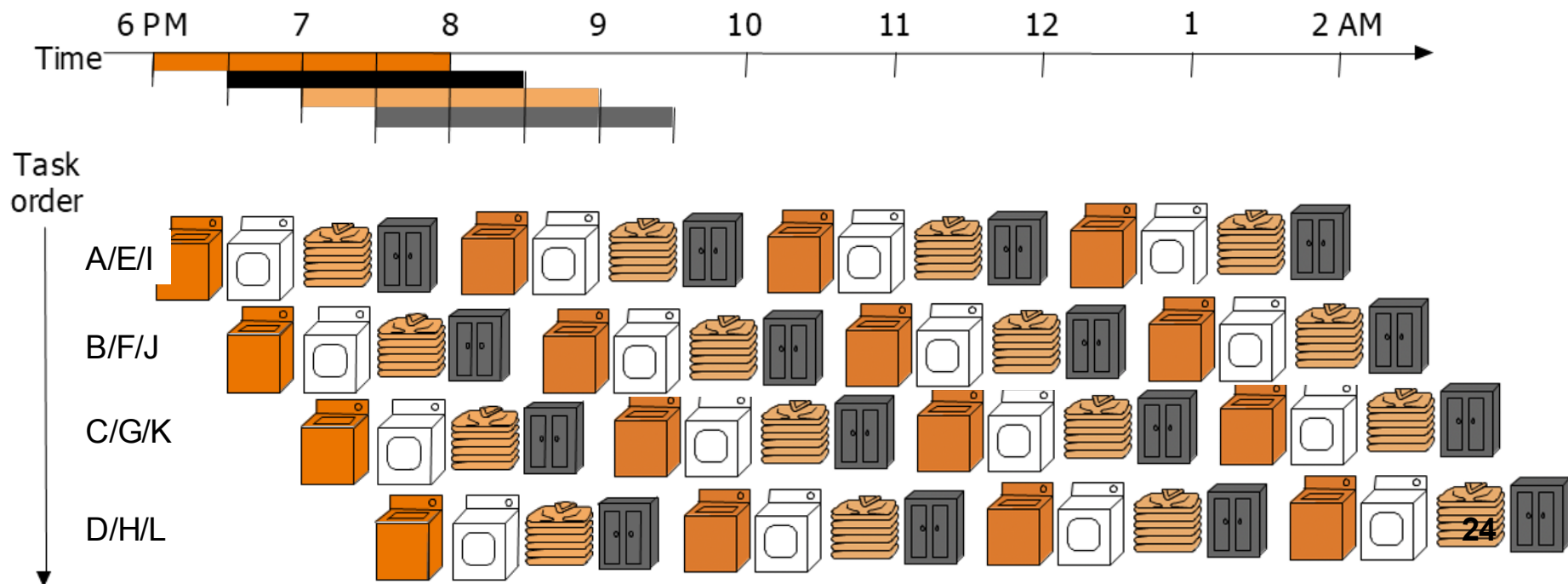
- 采用流水线的方式，洗4批衣服只需要花费3.5小时
- 采用流水线的方式，洗16批衣服只需要花费9.5小时



流水线的基本概念

• 生活中的实例——洗衣店

- 采用流水线的方式，洗4批衣服只需要花费3.5小时
- 采用流水线的方式，洗16批衣服只需要花费9.5小时
- 横向来看，洗完一批衣服的时间并没有降低
- 纵向来看，最多同时洗4批衣服，效率提高了



流水线的基本概念

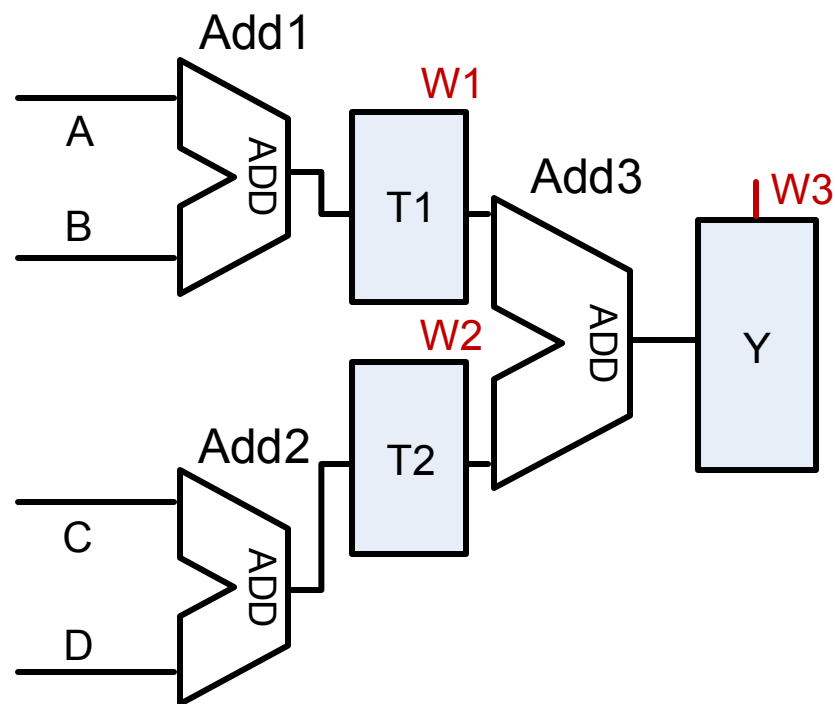
- 正如流水线可以加快洗衣店的工作一样，流水线也可以加快指令的总体执行时间
- 流水线（Pipeline）：时空复用
 - 将一个复杂的任务分解成若干个先后执行的子任务，每个子任务分配给专门的设施处理
 - 多个任务的不同子任务可以同时执行来提高处理器性能

流水线的基本概念

- 流水线:

- 第2个周期时, 第一级 (Add1, Add2) 可以处理下一次加法

流水级	资源绑定
1	$T1 = \text{Add1}(A, B)$ $T2 = \text{Add2}(C, D)$
2	$Y = \text{Add3}(T1, T2)$



流水线的基本概念

- 单周期→多周期→流水线
 - 延时与吞吐率：周期数与时钟周期
 - 调度（Scheduling）：操作分配到特定的周期
 - 资源绑定（Binding）：操作分配到特定的资源
 - 寄存器
 - 运算单元
 - 资源共享方式：
 - 时分复用：同一资源在多个时间重复使用
 - 空分复用：多个资源在同一时间使用

流水线的基本概念

- 提高处理器运算速度是处理器设计基本任务之一：

$$CPU\text{执行时间} = \text{指令数} \times CPI \times \text{时钟周期}$$

- 主要手段：
 - 减小指令数
 - 减小时钟周期
 - 减小CPI → 指令级并行处理

流水线的基本概念

- 指令级并行处理(ILP, Instruction-Level-Parallelism)
 - 并行执行多条指令
 - 目前和将来提高处理器指令执行速度的主要途径之一
- 基本技术：
 - 流水线(Pipeline): 把一条指令的执行过程分解为若干个子过程, 每个子过程与其他指令的不同子过程同时进行
 - 超标量(Superscale): 设置多个独立的功能部件并行执行指令, 确保处理器可以在同一时钟周期内执行多条指令
 - 超长指令字(VLIW): 在一条指令中设置多个独立的字段, 每个字段可以分别独立地控制各个功能部件并行工作

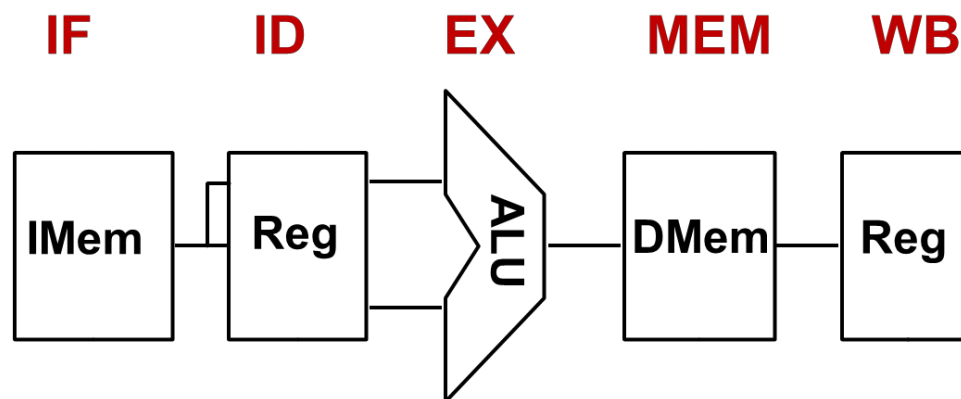
流水线的基本概念

- 每条指令的执行过程，可以划分为若干步骤
 - 参考多周期的多个步骤

步骤\指令类型	R型指令	内存访问指令	分支指令	跳转指令
取指令	IR <= MemInst[PC] PC <= PC+4			
译码 & 寄存器堆 读取	A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]] ALUOut <= PC + signext(IR[15:0]) << 2			
执行 & 分支/跳转完成	ALUOut <= A op B	ALUOut <= A + signext(IR[15:0])	if (A==B) PC<=ALUOut	PC <= {PC[31:28], IR[25:0], 2'b00}
内存访问 & R型指令完成	Reg[IR[15:11]] <= ALUOut	Load: MDR<=MemData[ALUOut] Store: MemData[ALUOut] <= B		
内存		Load: Reg[IR[20:16]]<=MDR		

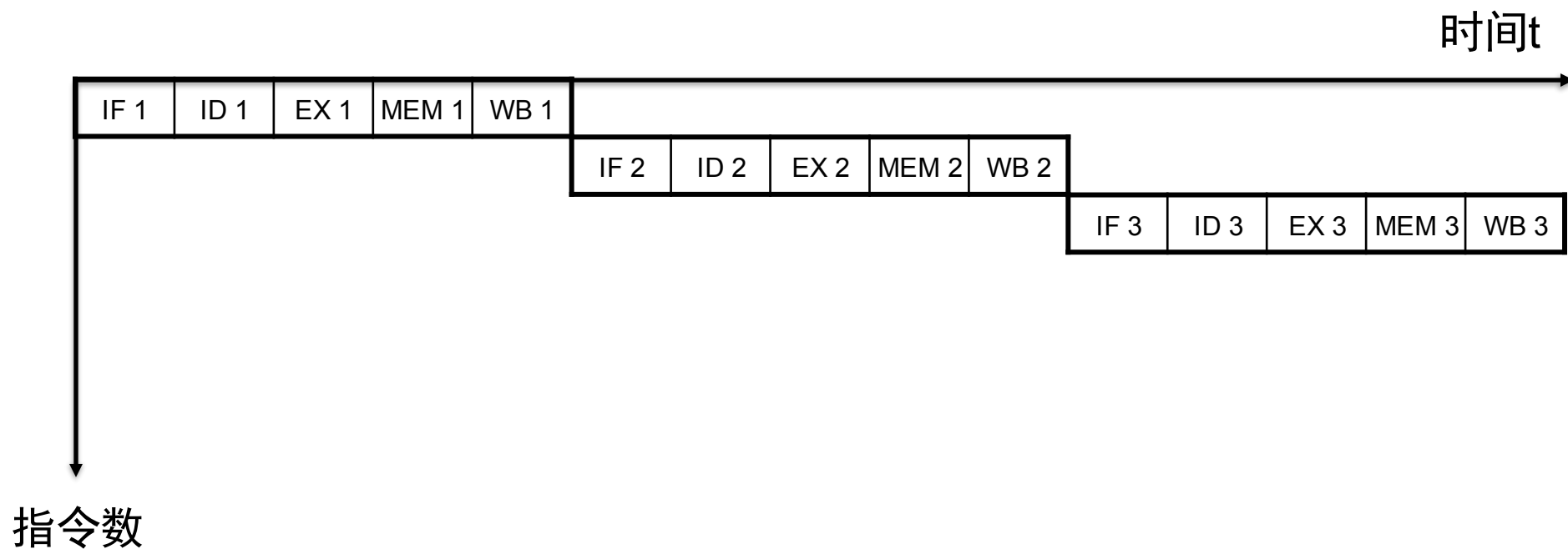
流水线的基本概念

- 每条指令的执行过程，可以划分为若干步骤
- 通常，一条MIPS指令包含5个先后处理步骤：
 - **Instruction Fetch (IF)**: 取指，从存储器读取指令
 - **Instruction Decode / Register File (ID/RF)**: 指令译码、读取寄存器
 - **ALU Execution (EX)**: 执行运算或计算存储器地址
 - **Memory Data Access (MEM)**: 访问存储器
 - **Register Write-Back (WB)**: 将结果写回寄存器



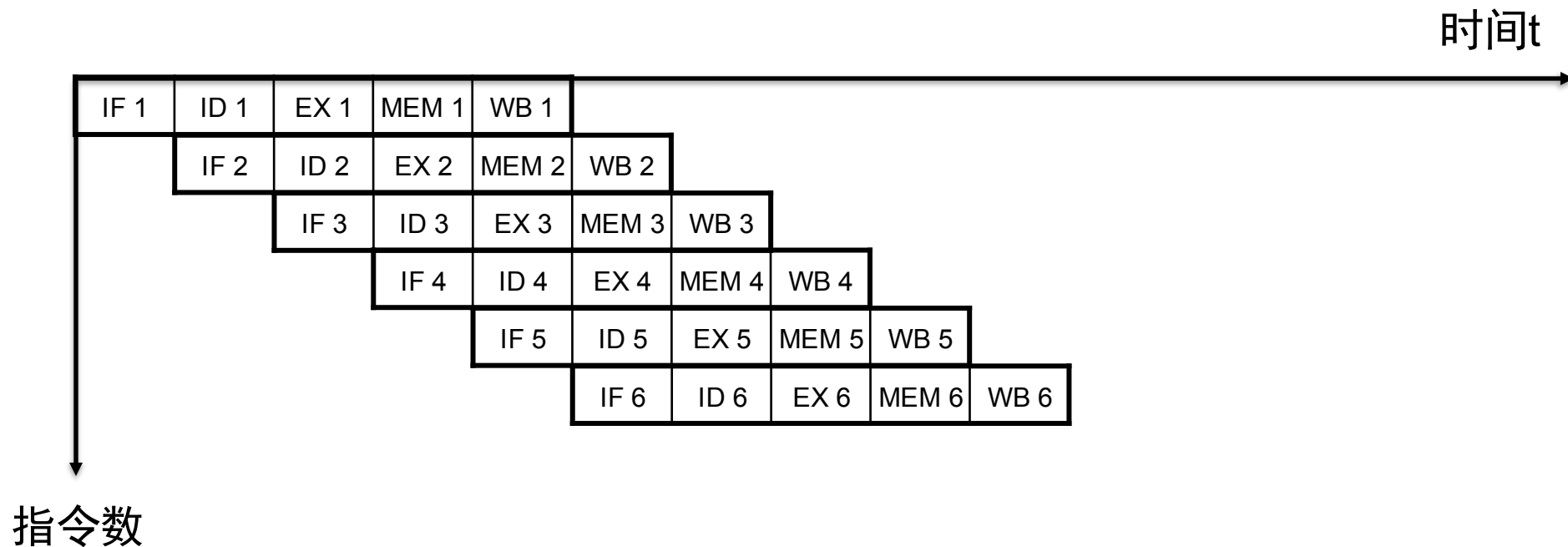
流水线的基本概念

- 单周期处理器的指令执行方式：



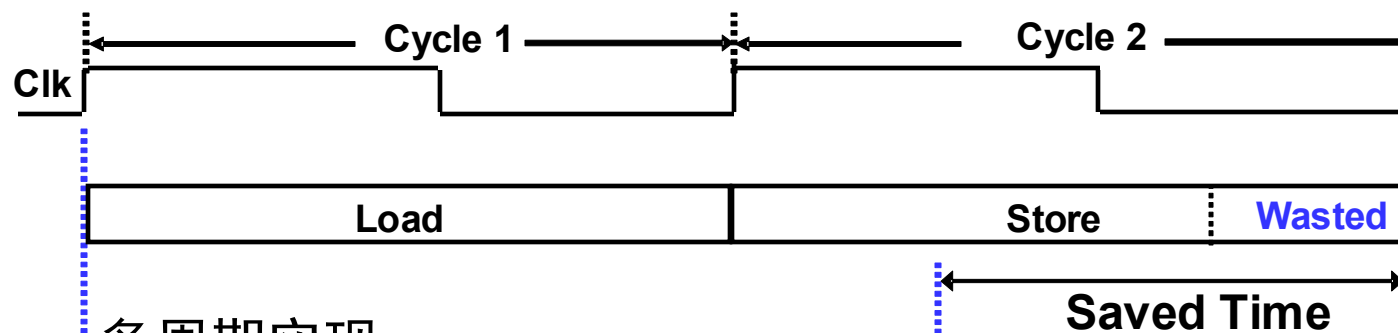
流水线的基本概念

- 流水线处理器的指令执行方式
- 流水线并不能减少单条指令的延时，但是可以提高一段时间内处理的总指令数（吞吐率）

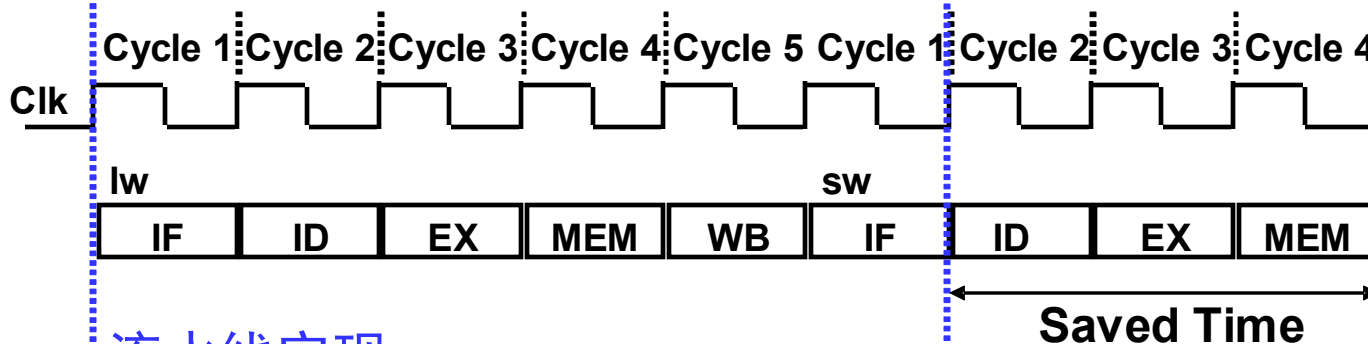


单周期 vs 多周期 vs 流水线

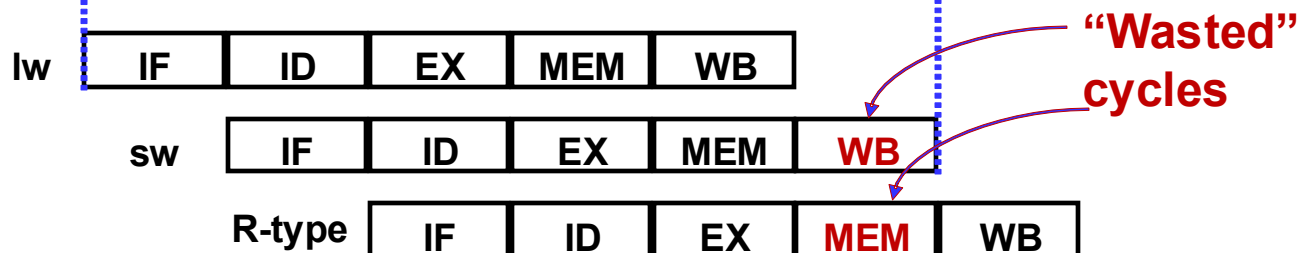
单周期实现



多周期实现



流水线实现



流水线的瓶颈

- 流水线中各段的时间应尽量**相等**，为什么？
 - 流水线处理时间受限于**最慢**的处理资源
- 类比多周期：
 - 各个状态的延时应该尽量相等

流水线的性能

- **吞吐率 (Throughput, TP)**
 - 单位时间内流水线处理的指令数
- **最大吞吐率**
 - 在连续指令流输入下流水线达到稳态后的吞吐率
- **实际吞吐率**
 - 运行有限任务实际能达到的吞吐率

流水线的性能

- 吞吐率

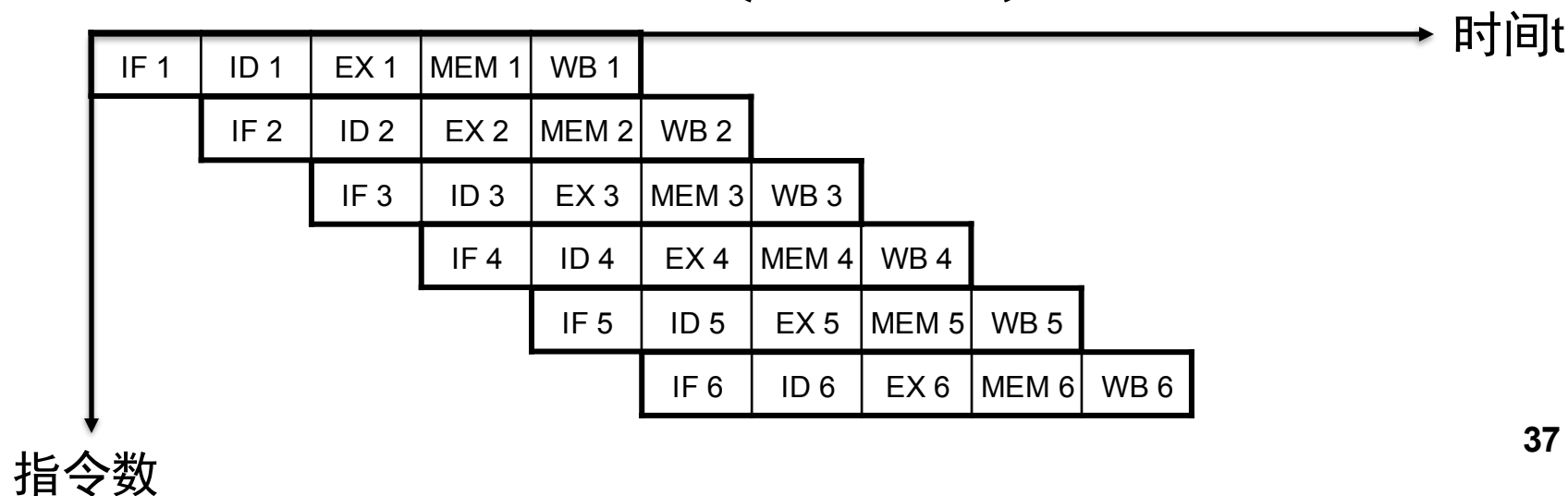
n 条指令, k 级流水线, 级间延时(周期) Δt
 T_k 为 k 级流水线处理 n 条指令所用的时间

- 实际吞吐率:

$$TP = \frac{n}{T_k} = \frac{n}{(k + n - 1)\Delta t}$$

- 最大吞吐率:

$$TP_{max} = \lim_{n \rightarrow \infty} \frac{n}{(k + n - 1)\Delta t} = \frac{1}{\Delta t}$$



流水线的性能

- 加速比（speedup）

- 不使用流水线所用的时间，与使用流水线所用的时间的比值。处理 n 条指令， T_0 为不使用流水线所用的时间， T_k 为使用 k 级流水线所用的时间（假设每一段操作所用时间相同）

$$S = \frac{T_0}{T_k}$$

- 实际加速比：

$$S = \frac{nk\Delta t}{(k+n-1)\Delta t} = \frac{kn}{k+n-1}$$

- 最大加速比：

$$S_{max} = \lim_{n \rightarrow \infty} \frac{kn}{k+n-1} = k$$

本节课目录

- 复习回顾与流水线基本概念 P5
- 流水线数据通路 P42
- 流水线的冒险 P89
 - 结构冒险
 - 数据冒险
 - 控制冒险
- 数据通路的实现（考虑冒险）

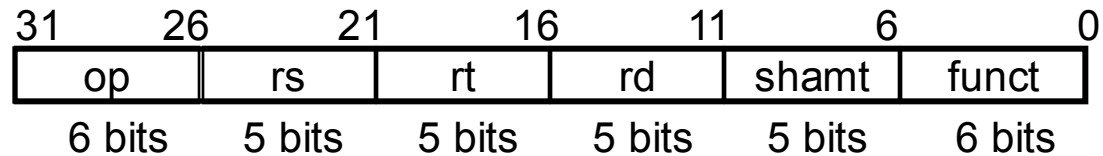
针对流水线结构的指令集设计

- **MIPS指令集是专为指令执行的流水线化而设计的**
 1. **MIPS为定长指令格式**
 - 简化IF和ID的过程
 2. **MIPS只有很少的指令格式，并且每种指令格式中源寄存器的字段都相同**
 - ID阶段译码的同时可以读取寄存器
 3. **内存操作只出现在装入和存储指令中**
 - 在EX阶段可以计算内存地址
 4. **内存中的操作数必须对齐**
 - 一条数据传输指令只需访问一次内存

流水线MIPS子集

- **ADD, SUB, AND, SLT**

- addu rd, rs, rt
- subu rd, rs, rt
- and rd, rs, rt
- sltu rd, rs, rt

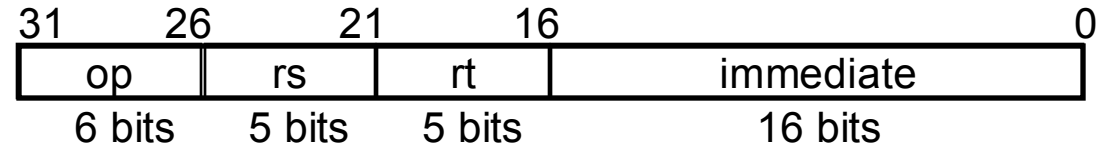


- **OR Immediate**

- ori rt, rs, imm16

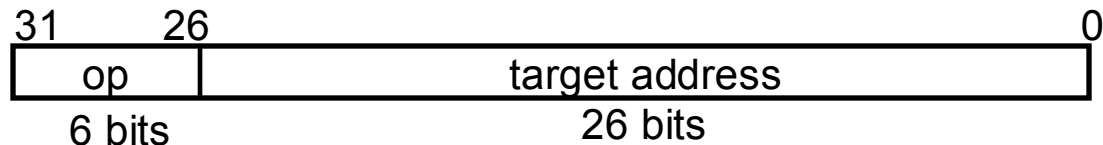
- **LOAD and STORE Word**

- lw rt, imm16(rs)
- sw rt, imm16(rs)



- **BRANCH:**

- beq rs, rt, imm16

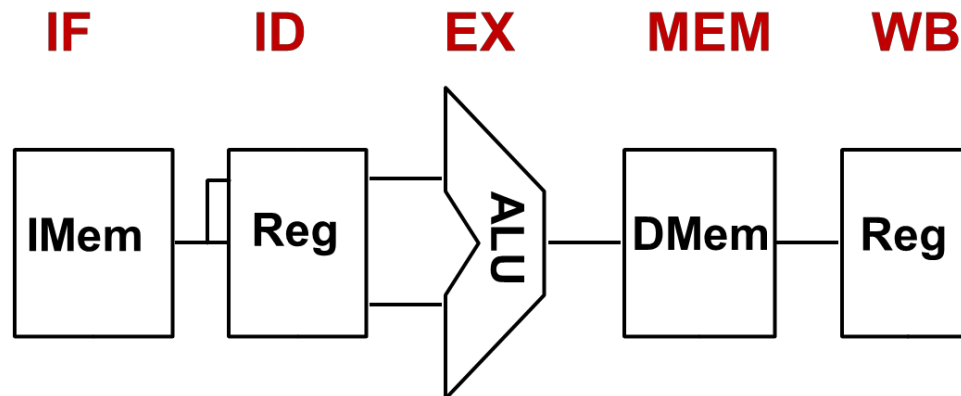


针对流水线结构的指令集设计

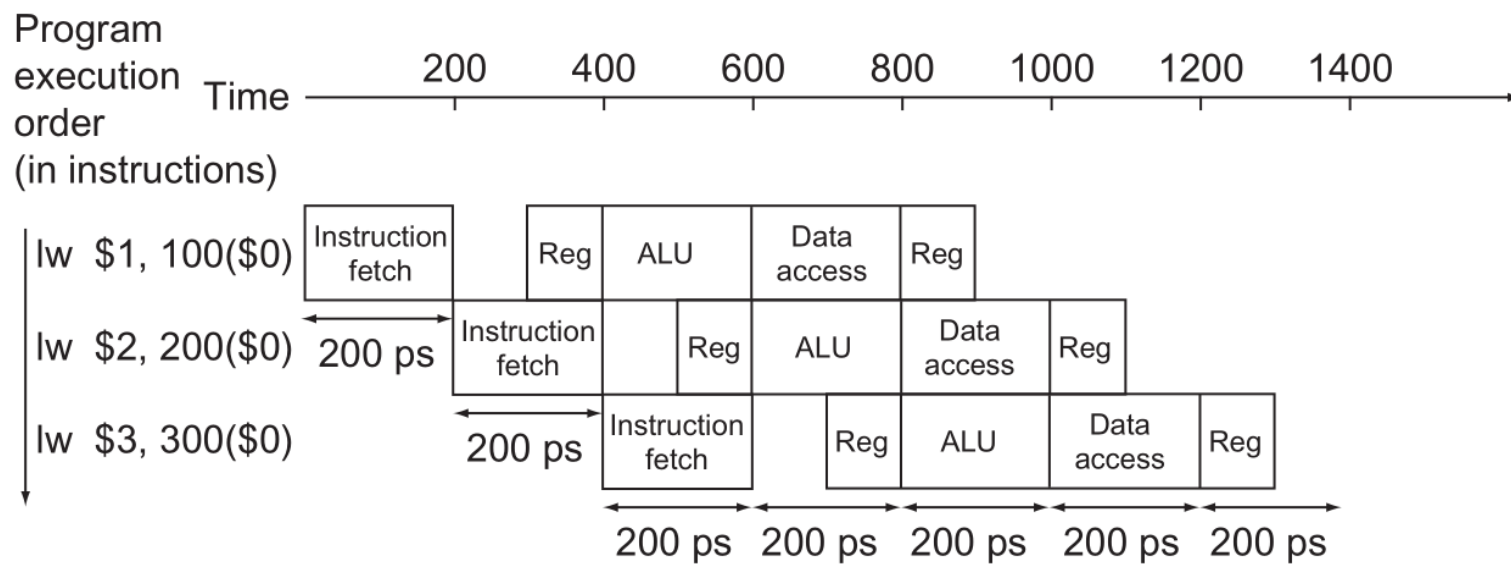
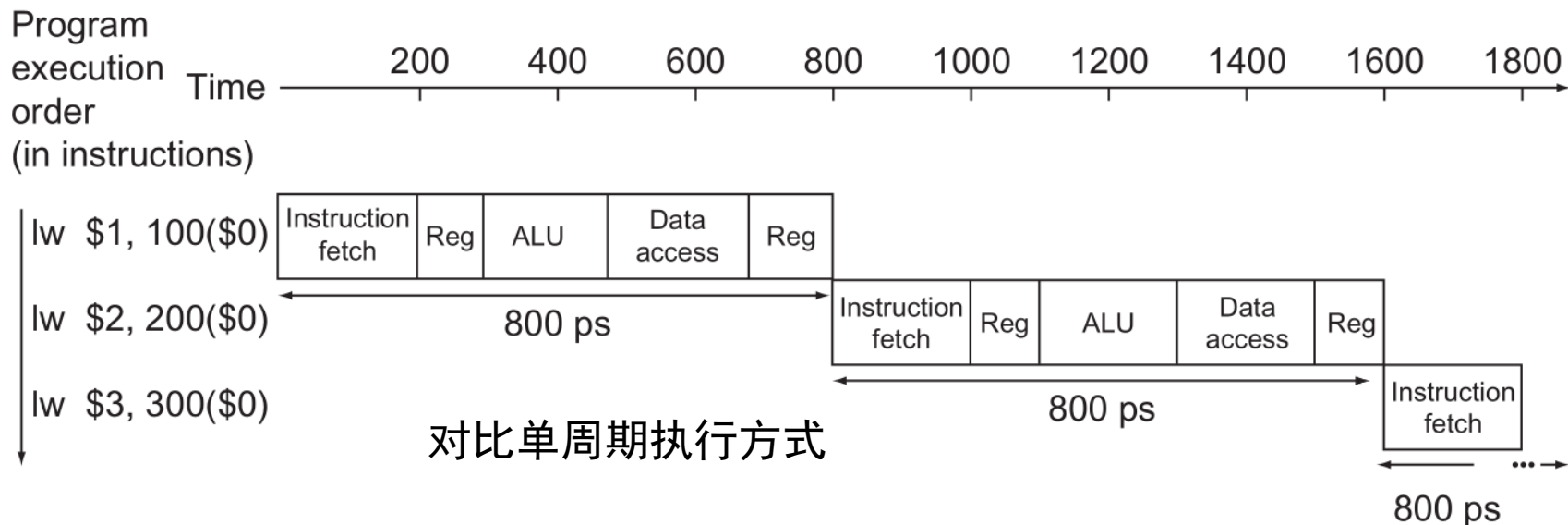
指令类型	取指令	读寄存器	ALU操作	数据访问	写寄存器	总执行时间
lw	1	1	1	1	1	5
sw	1	1	1	1		4
R型 add, sub, etc.	1	1	1		1	4
ori	1	1	1		1	4
beq	1	1	1			3

流水线的数据通路

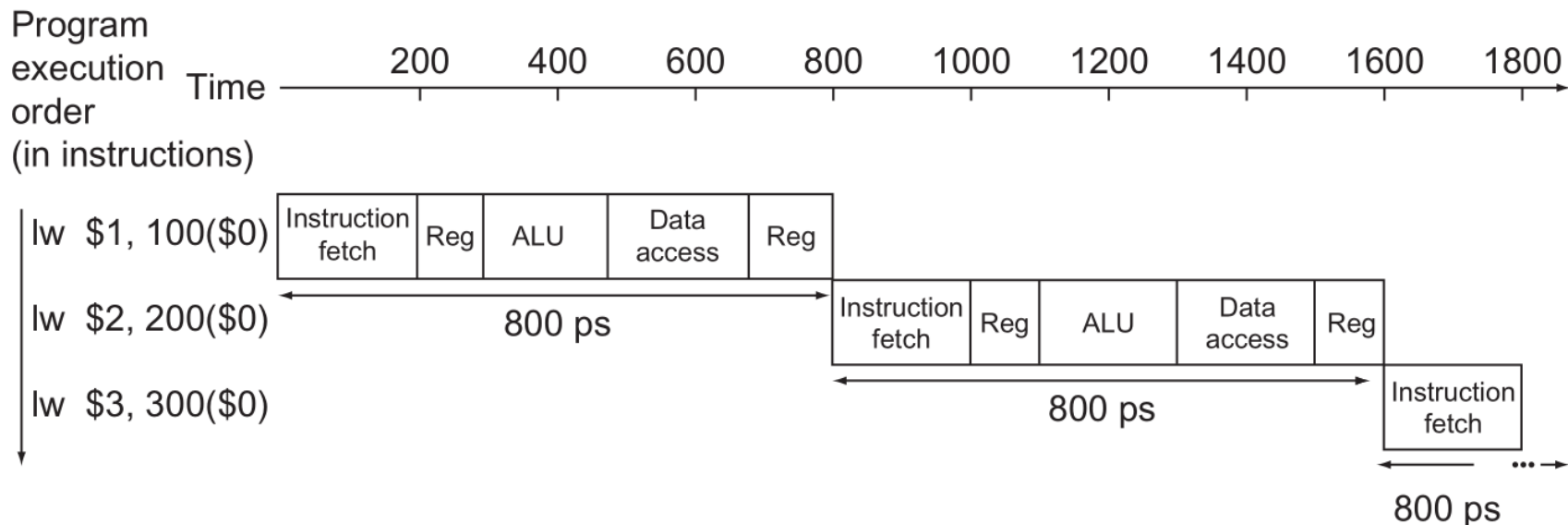
- 目标：流水线的CPI下限是1，希望能够达到“几乎”1.0的CPI和更高的时钟频率
- 方法：在多周期基础上，将处理器变为多级流水线
 - **Instruction Fetch (IF)**: 取指，从存储器读取指令
 - **Instruction Decode / Register File (ID/RF)**: 指令译码、读取寄存器
 - **ALU Execution (EX)**: 执行运算或计算存储器地址
 - **Memory Data Access (MEM)**: 访问存储器
 - **Register Write-Back (WB)**: 将结果写回寄存器



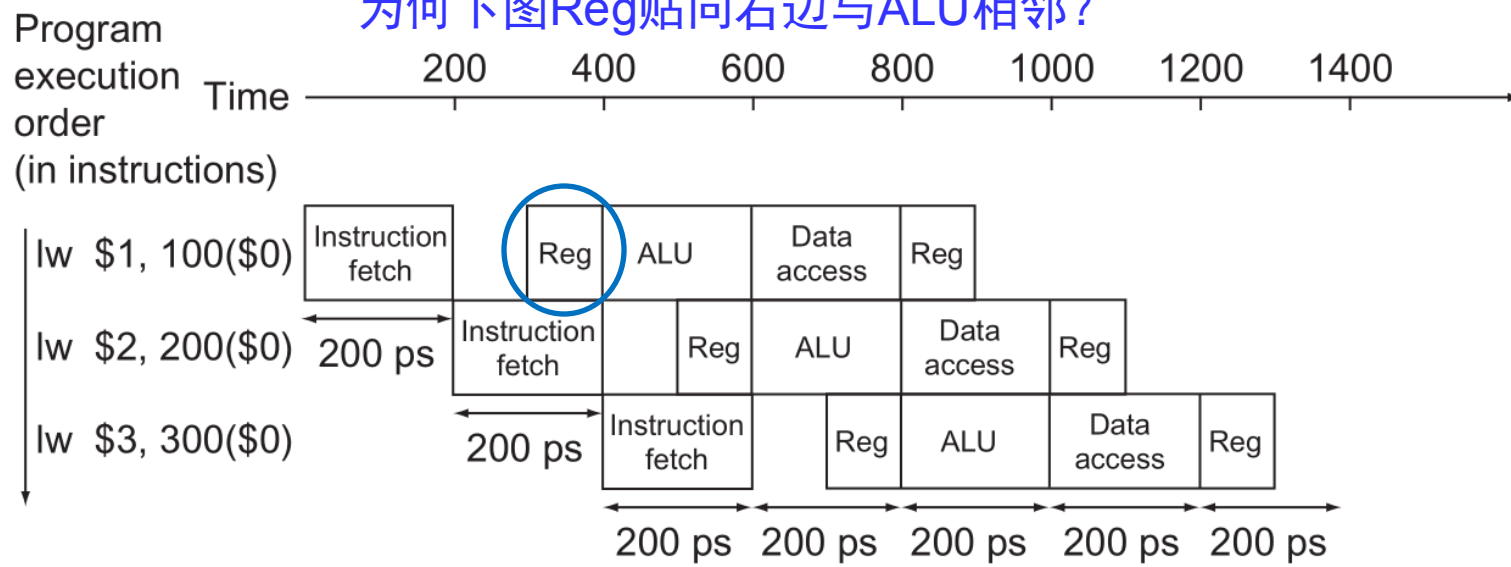
流水线的数据通路



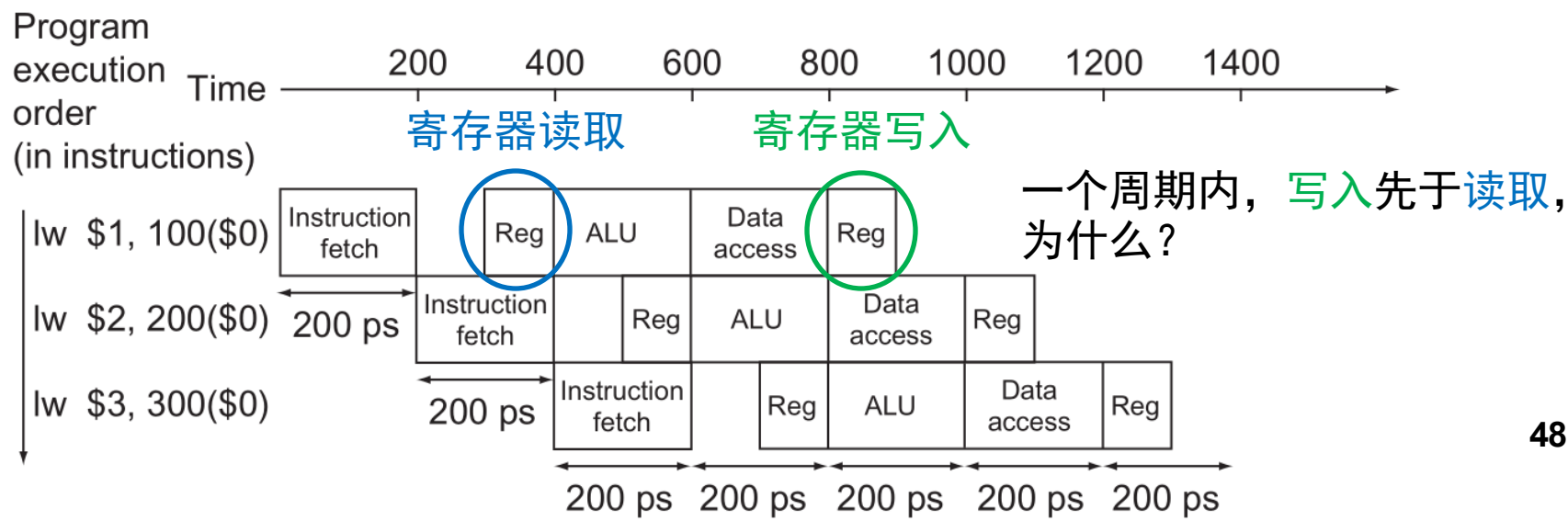
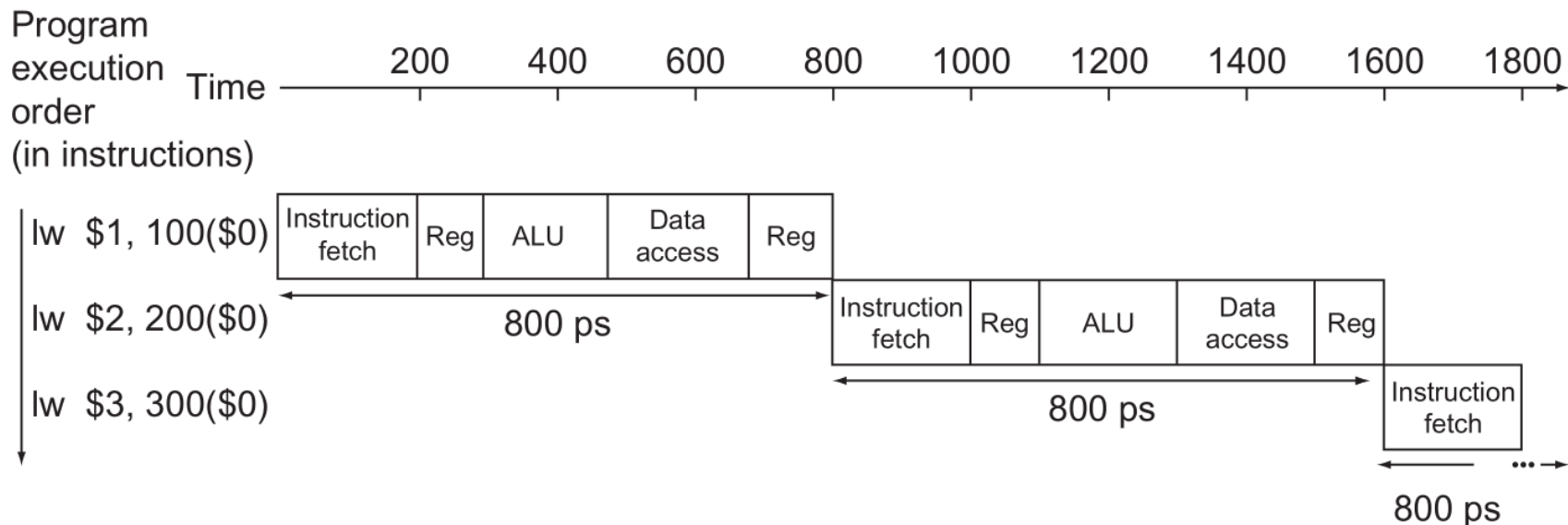
流水线的数据通路



为何下图Reg贴向右边与ALU相邻?



流水线的数据通路



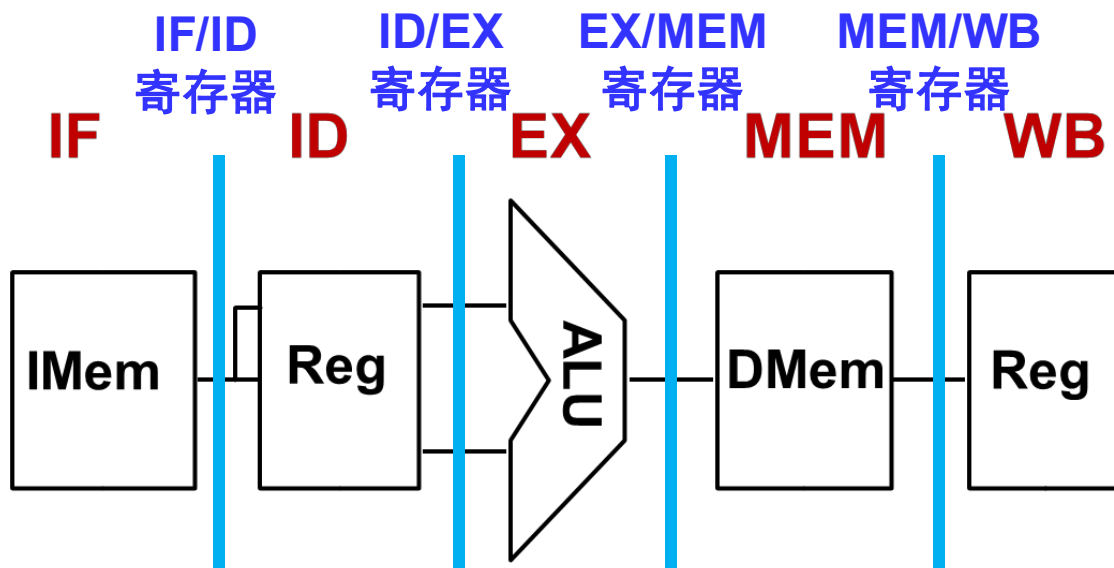
流水线的数据通路：级间寄存器

- 5个步骤

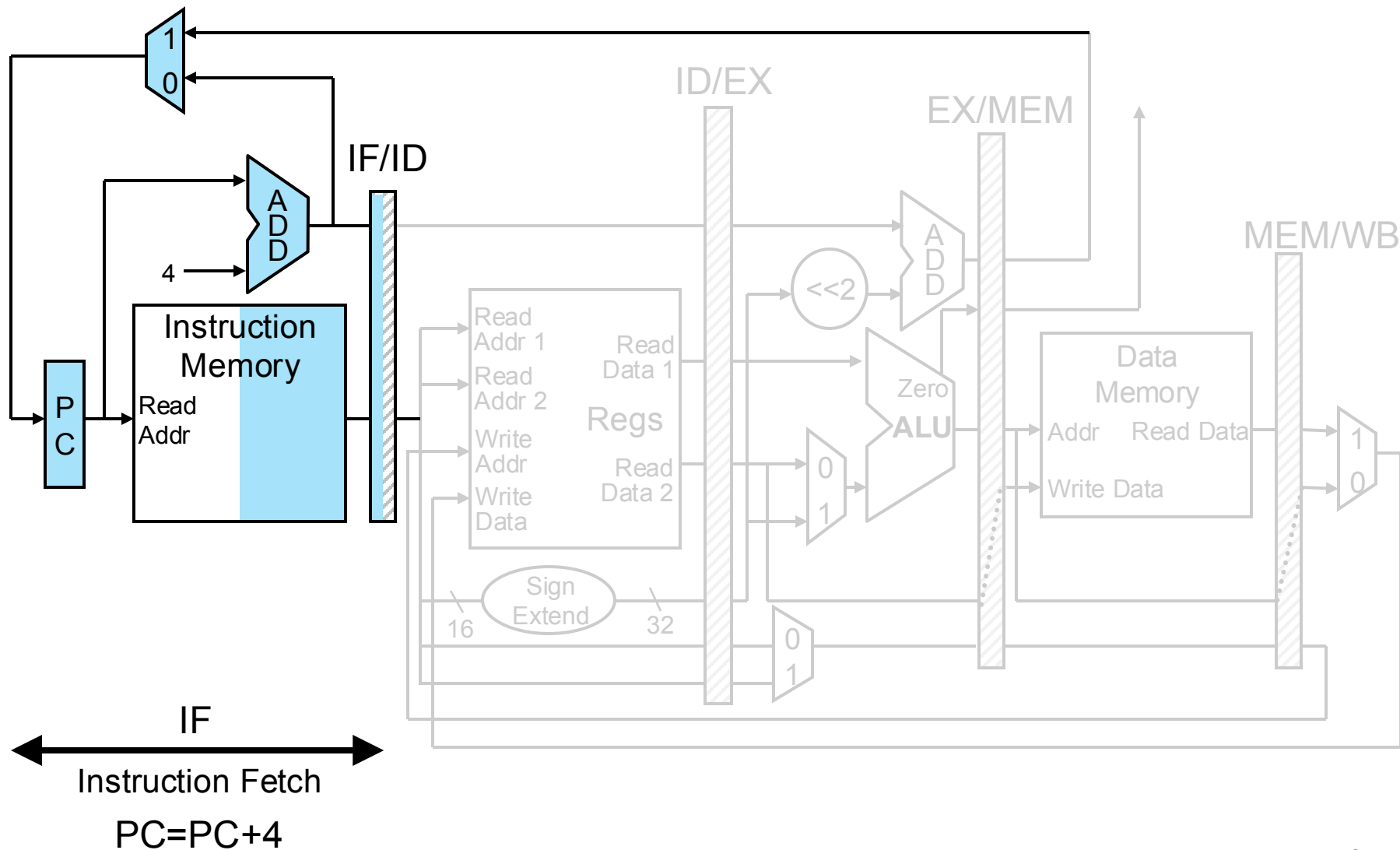
- 输入：来自于上一步骤上一周期
- 输出：被下一步骤在下一周期使用

- 数据在不同周期间传递

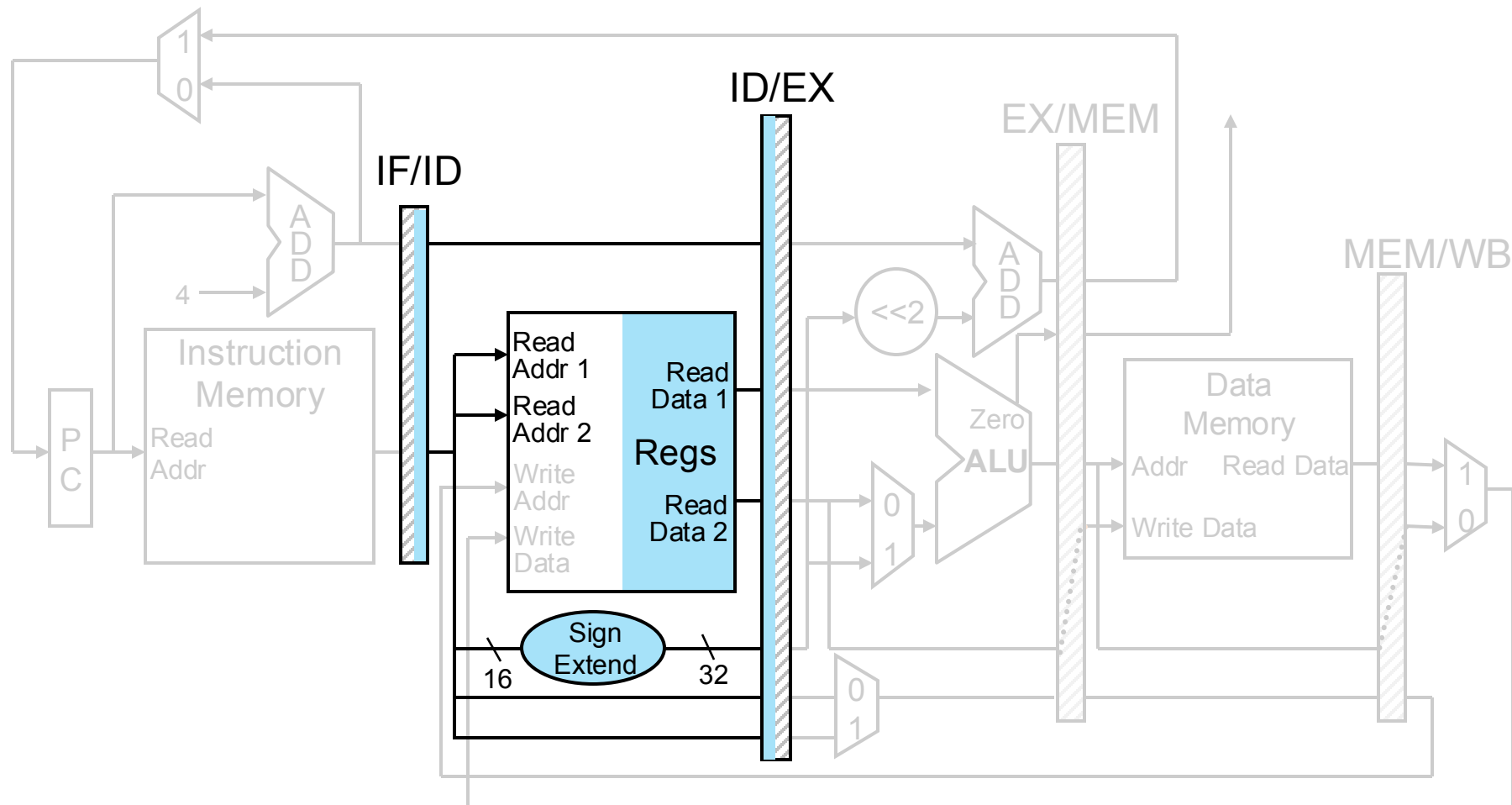
- 需要寄存器保持中间数据



流水线的数据通路



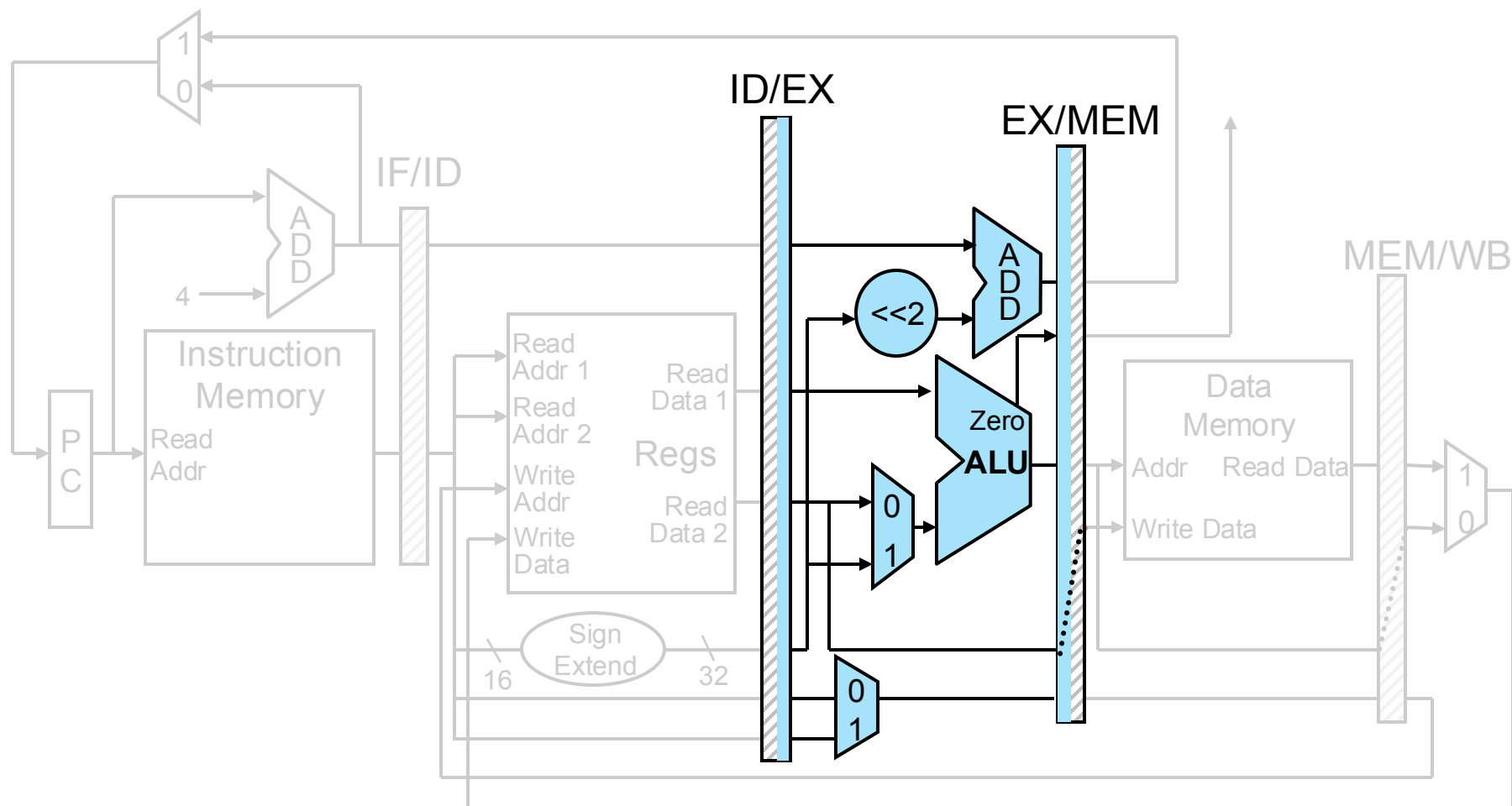
流水线的数据通路



ID
Instruction Decode/
Register Fetch

寄存器读取 &
立即数扩展

流水线的数据通路

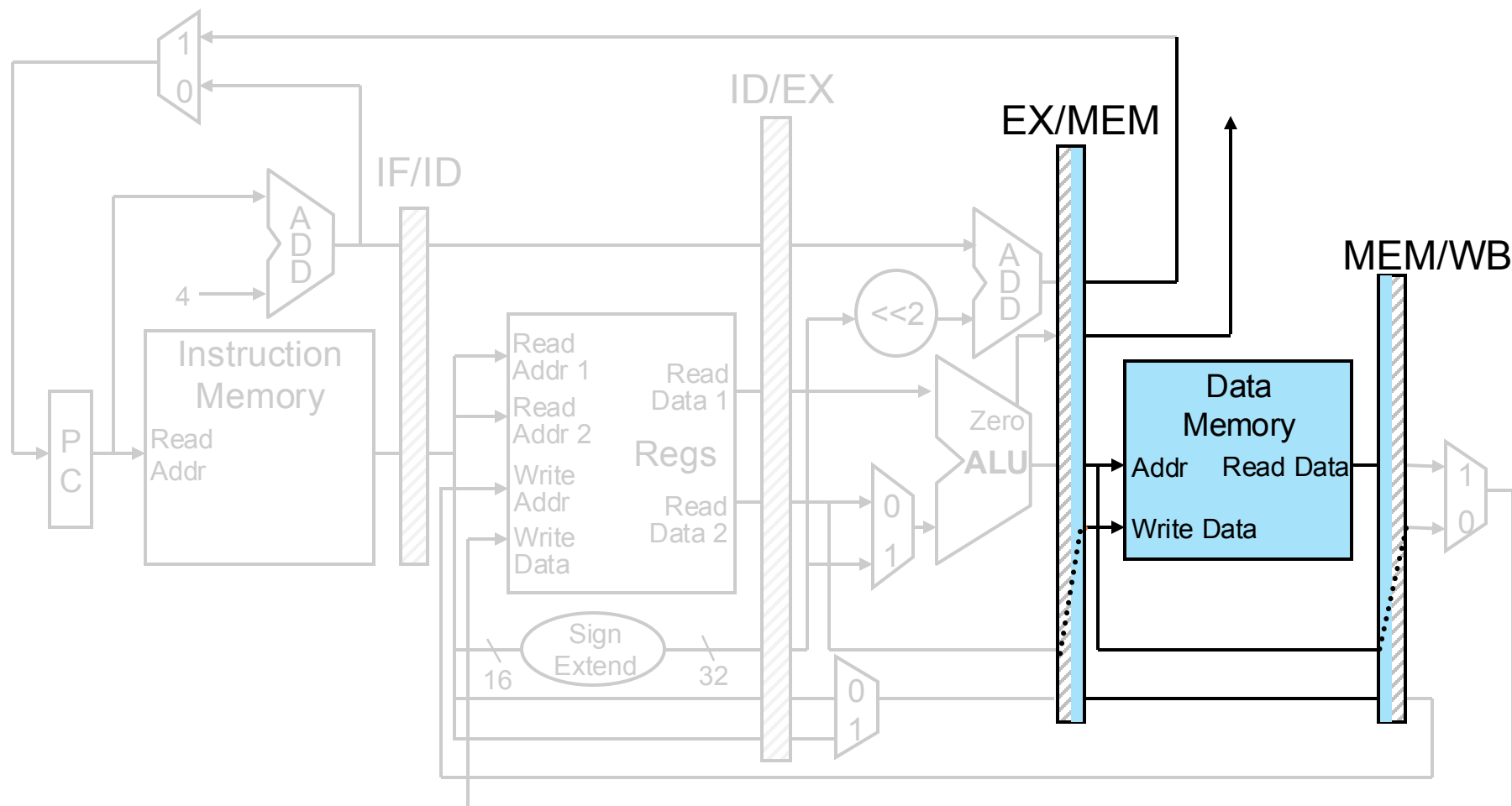


EX
ALU execution
Jump address

ALU计算访存地址
地址跳转

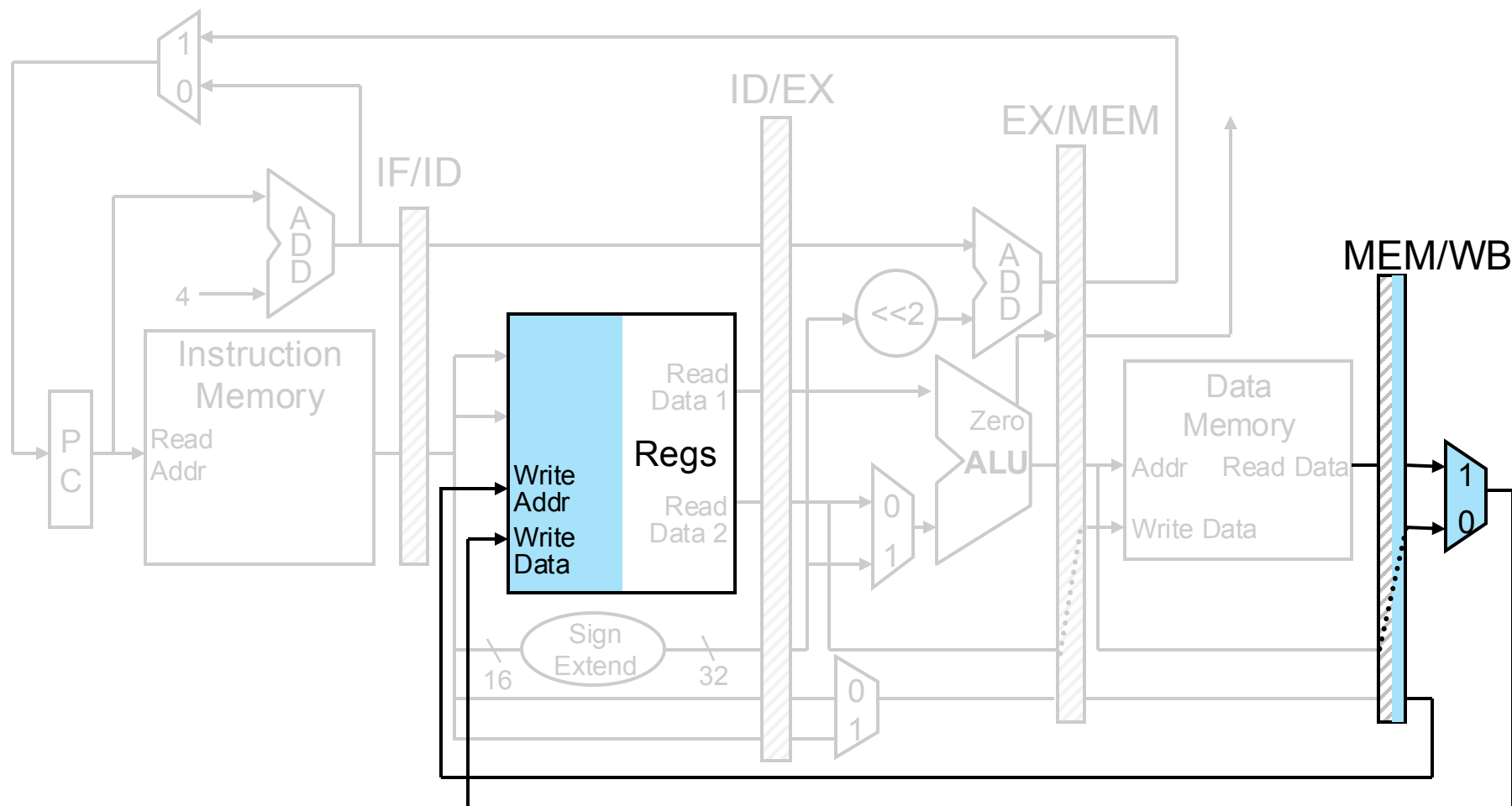
53

流水线的数据通路



Memroy数据读取 **MEM** **Memory Access**

流水线的数据通路



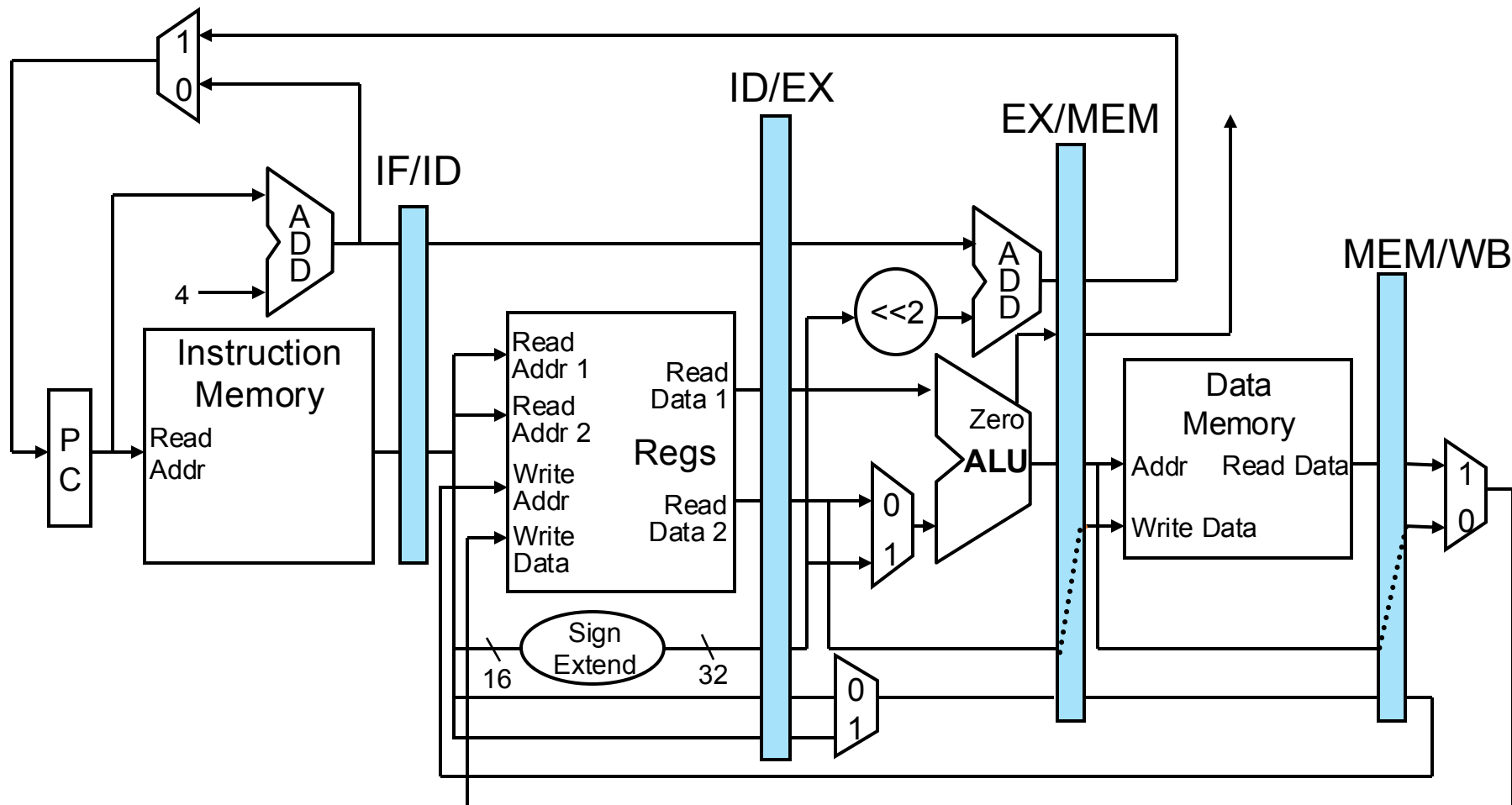
注意： 回写阶段的后面没有流水线寄存器，所有的指令都会更新机器中的一些状态，如寄存器堆、存储器、PC等

Write Back
数据写回寄存器堆

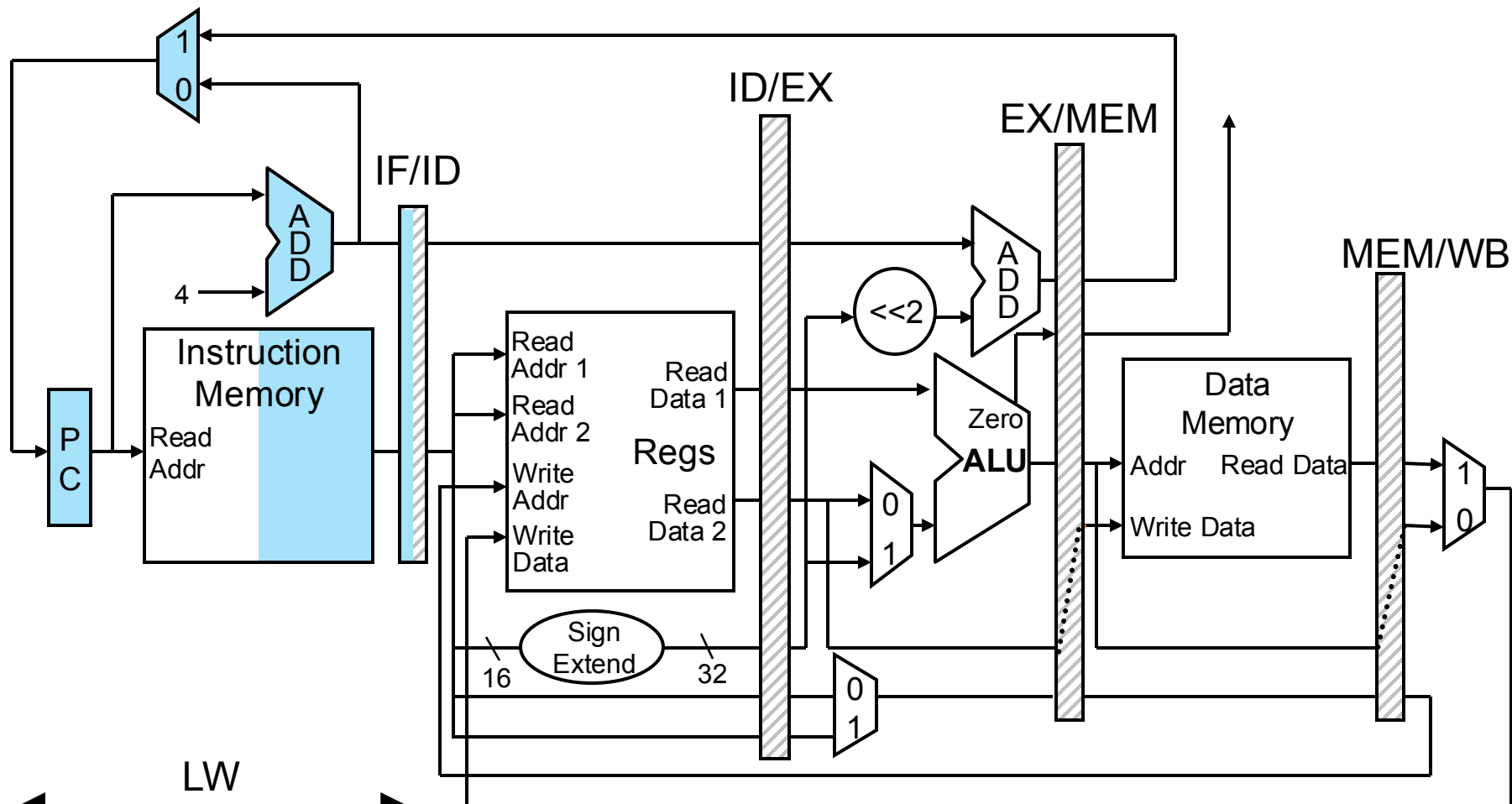
WB

55

流水线的数据通路：例子

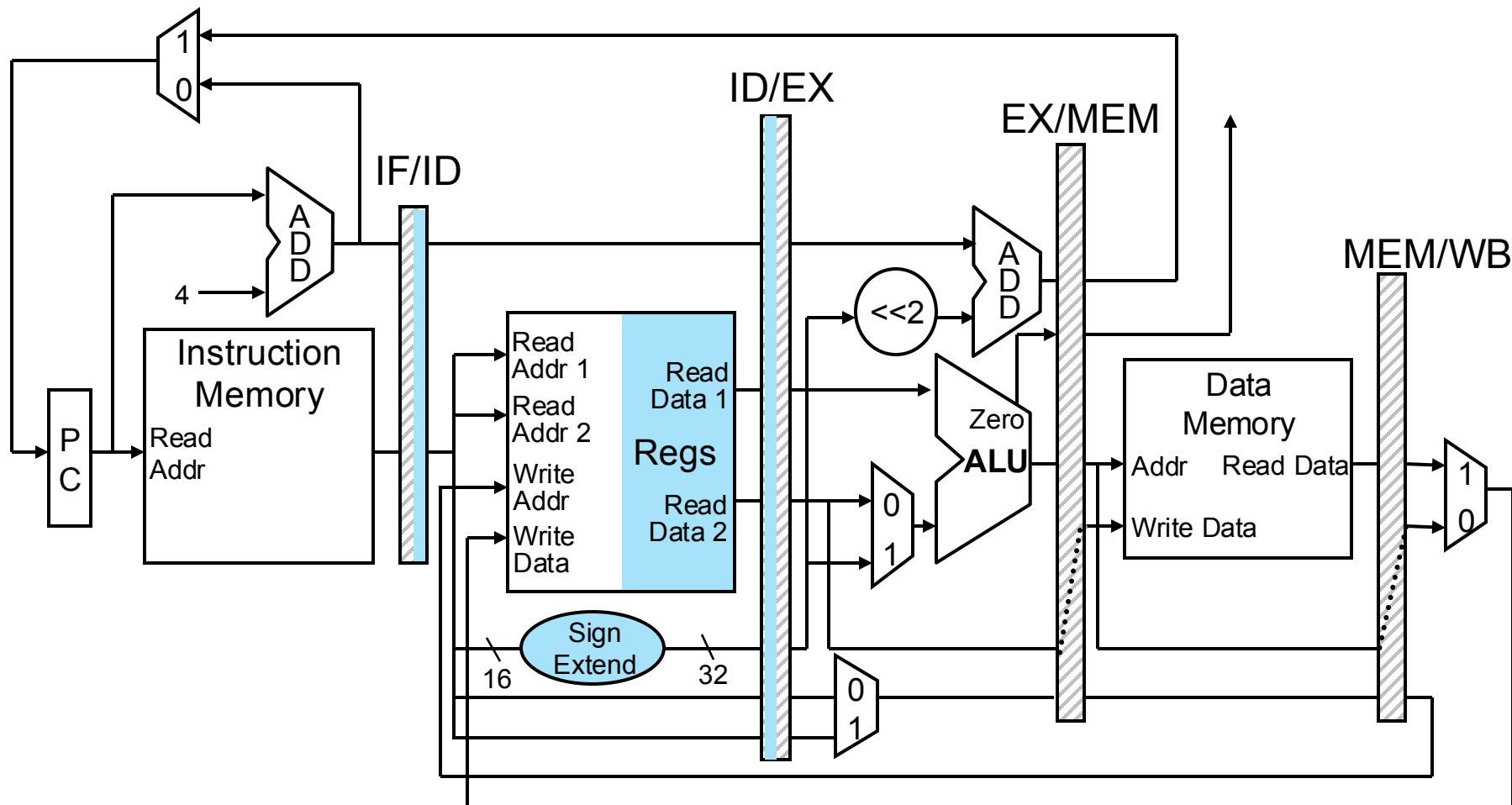


流水线的数据通路：LW指令



LW
Instruction Fetch
取指令并存入IF/ID
PC=PC+4

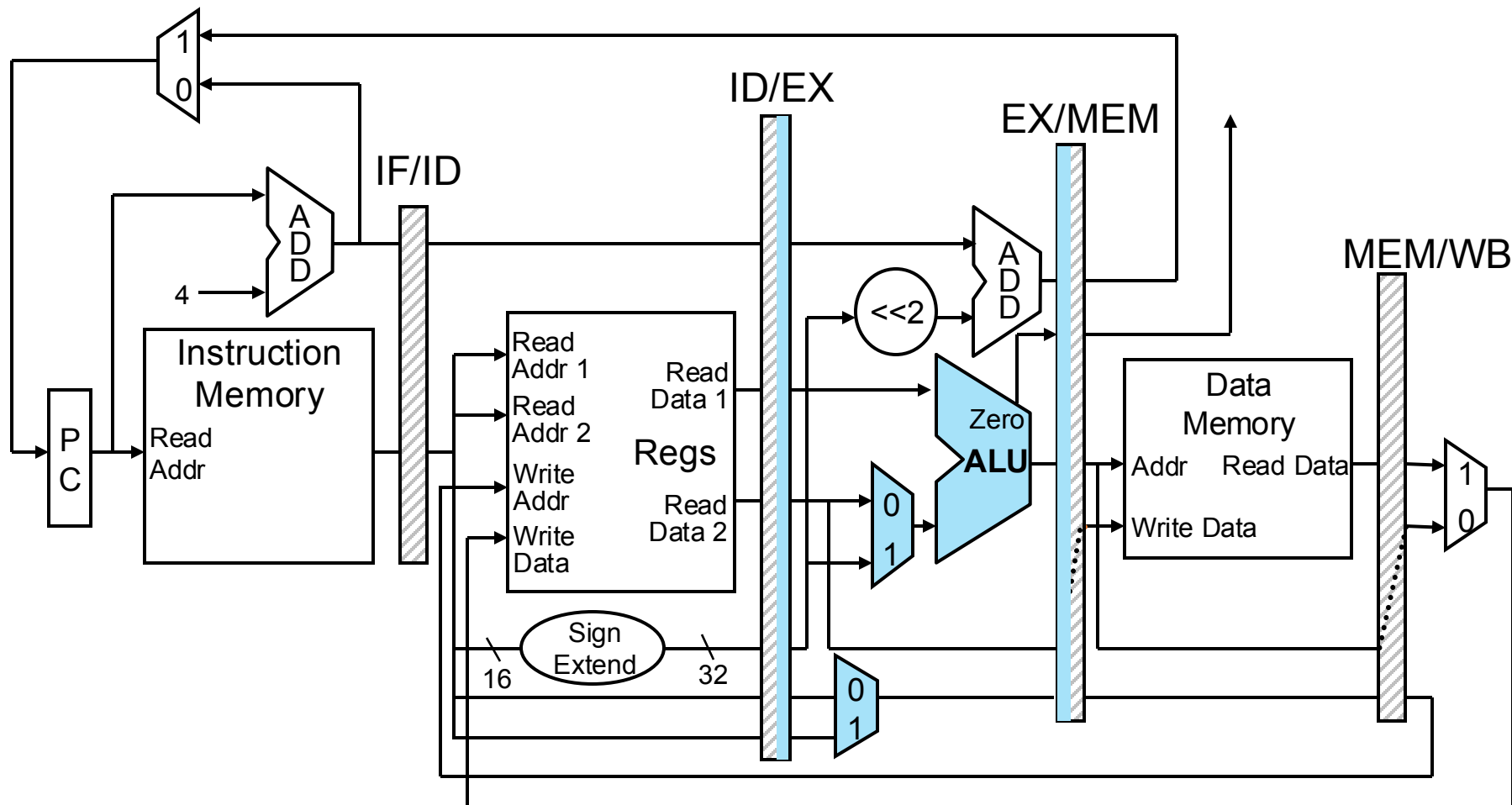
流水线的数据通路：LW指令



LW
Instruction Decode/
Register Fetch

寄存器读取
立即数扩展
存入ID/EX

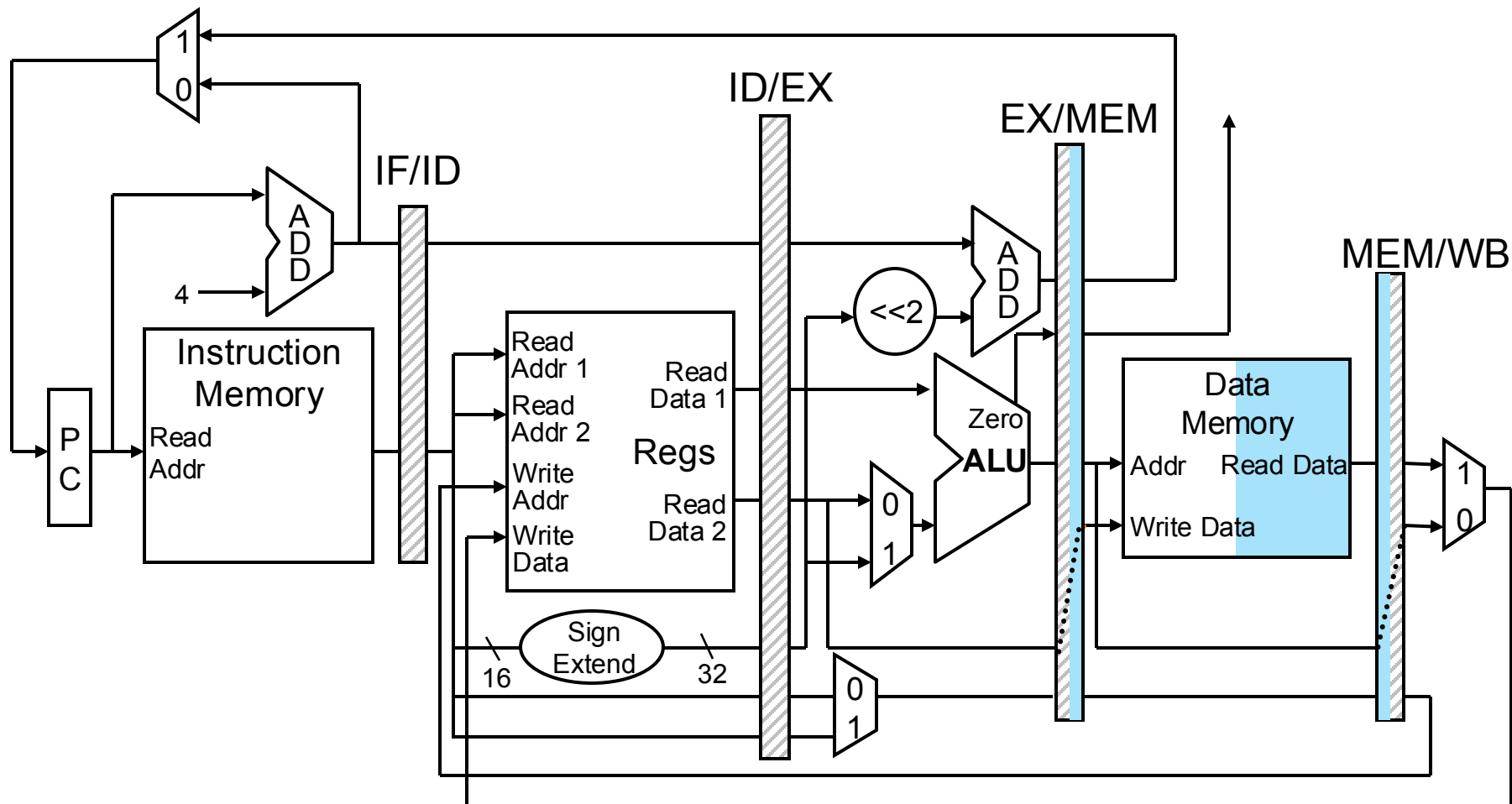
流水线的数据通路：LW指令



LW
Execution

ALU计算访存地址
存入EX/MEM

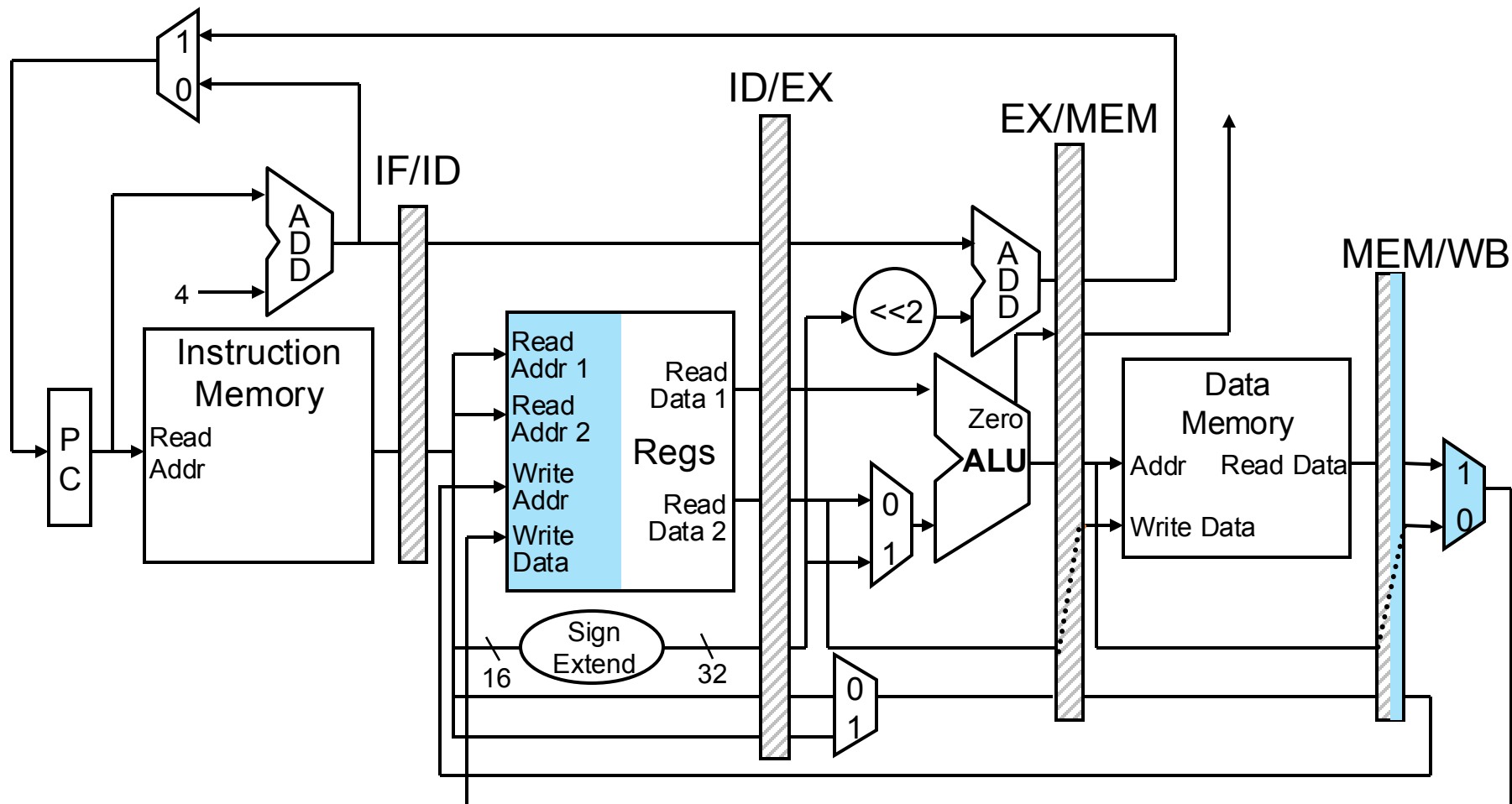
流水线的数据通路：LW指令



数据内存读取，存入MEM/WB & MDR

← LW →
Memory Access

流水线的数据通路：LW指令

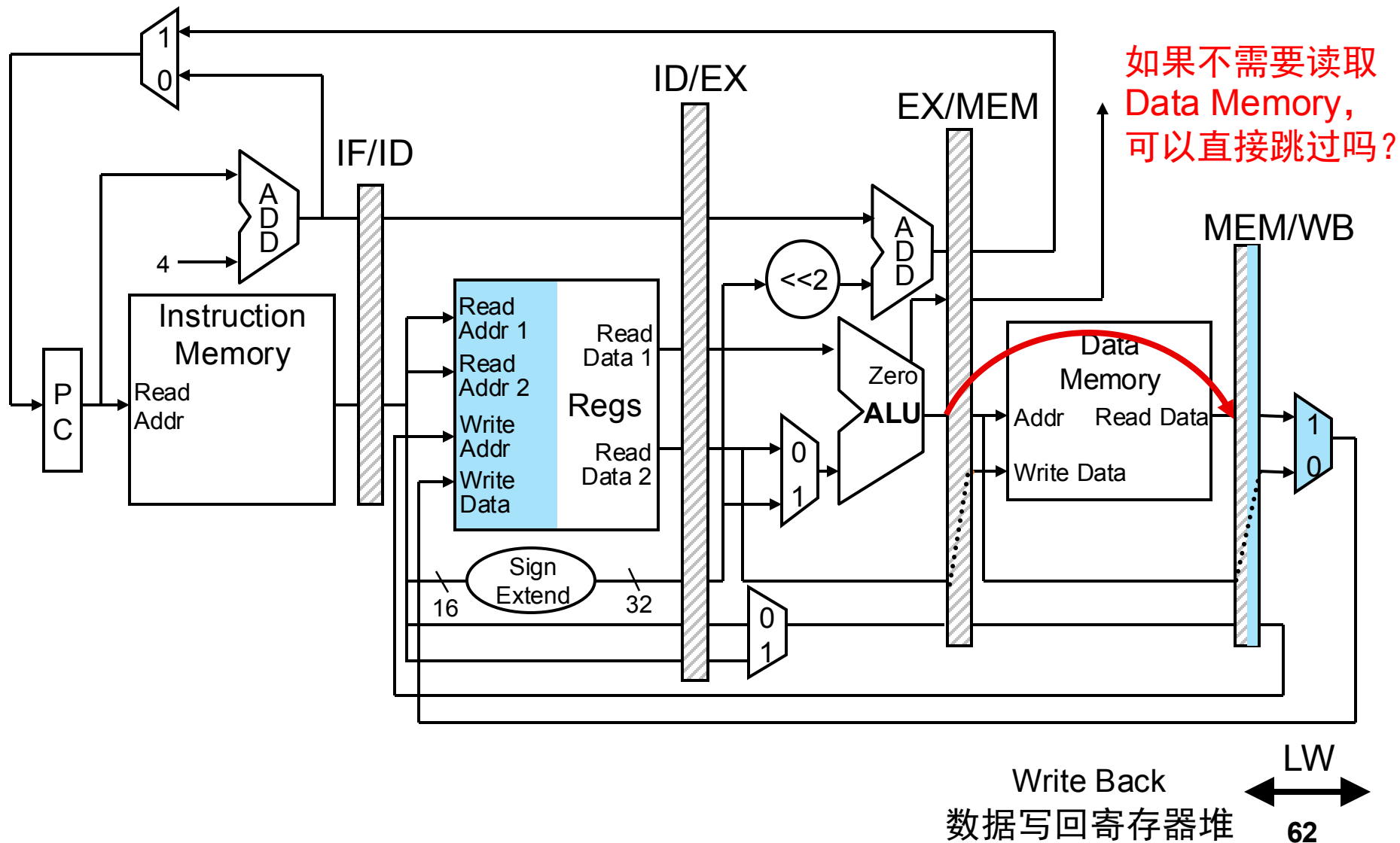


注意： WB是流水线的最后一个阶段

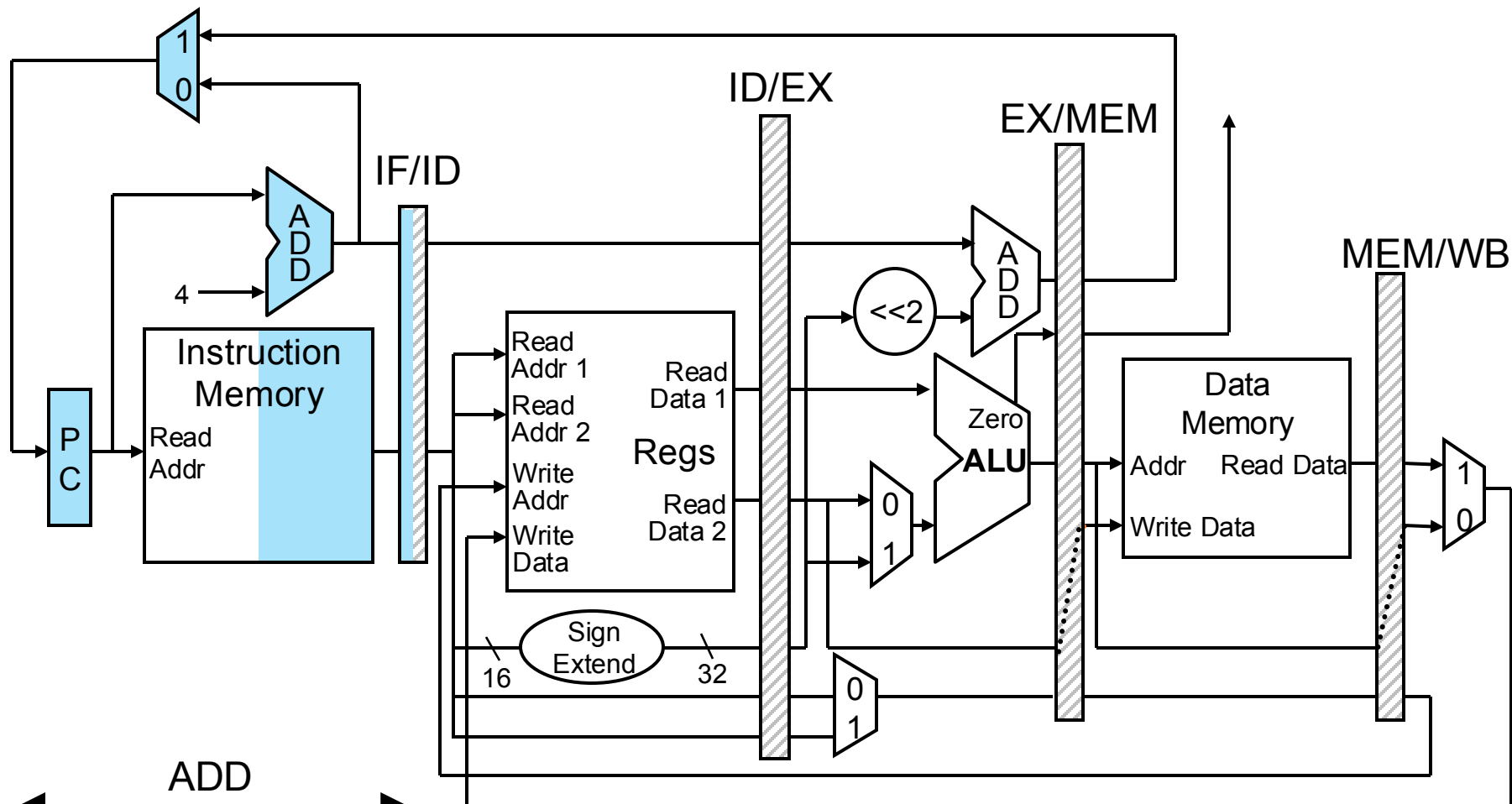
Write Back
数据写回寄存器堆

LW
61

流水线的数据通路：LW指令



流水线的数据通路：ADD指令



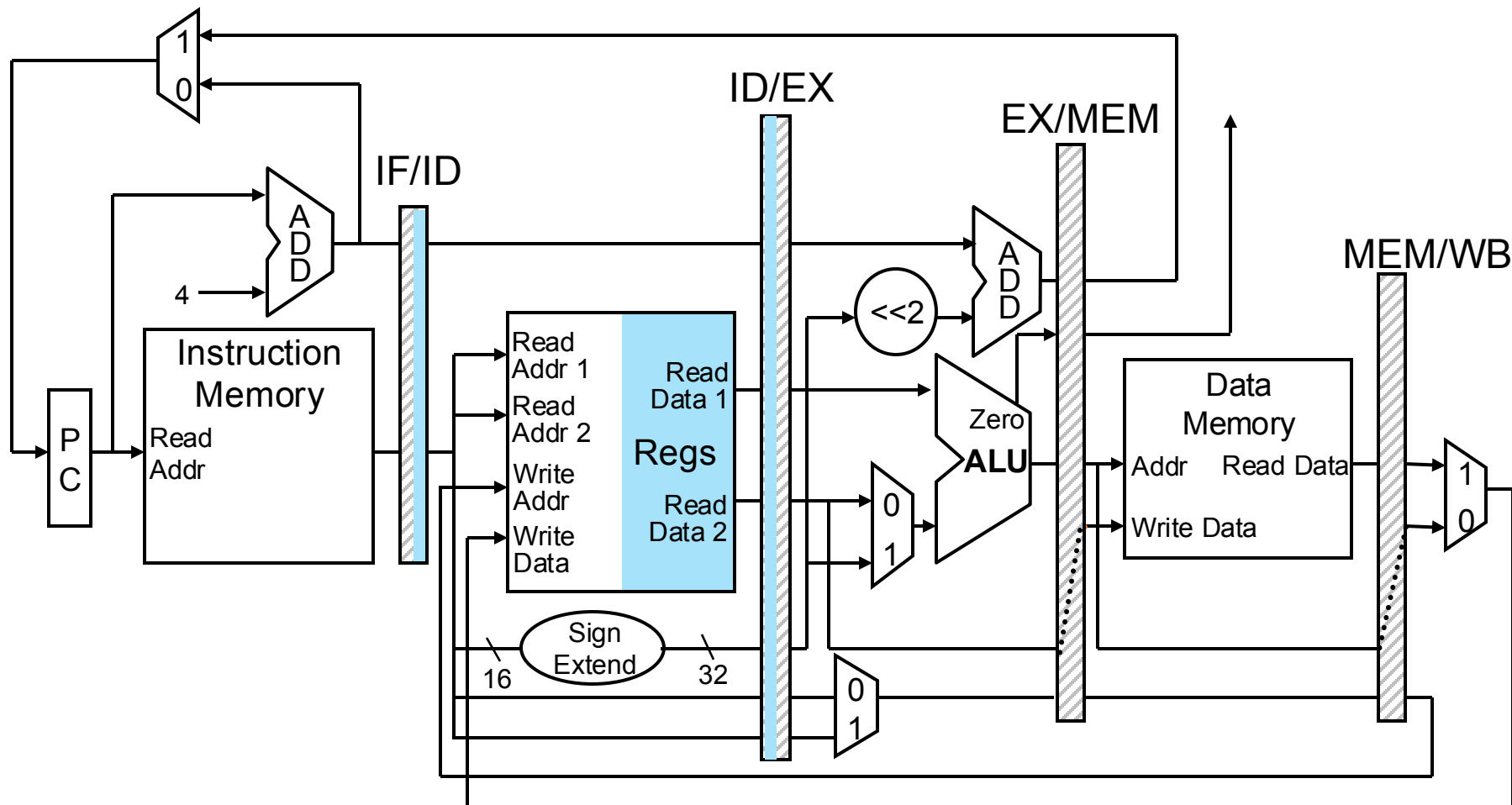
ADD

Instruction Fetch

取指令并存入IF/ID

PC=PC+4

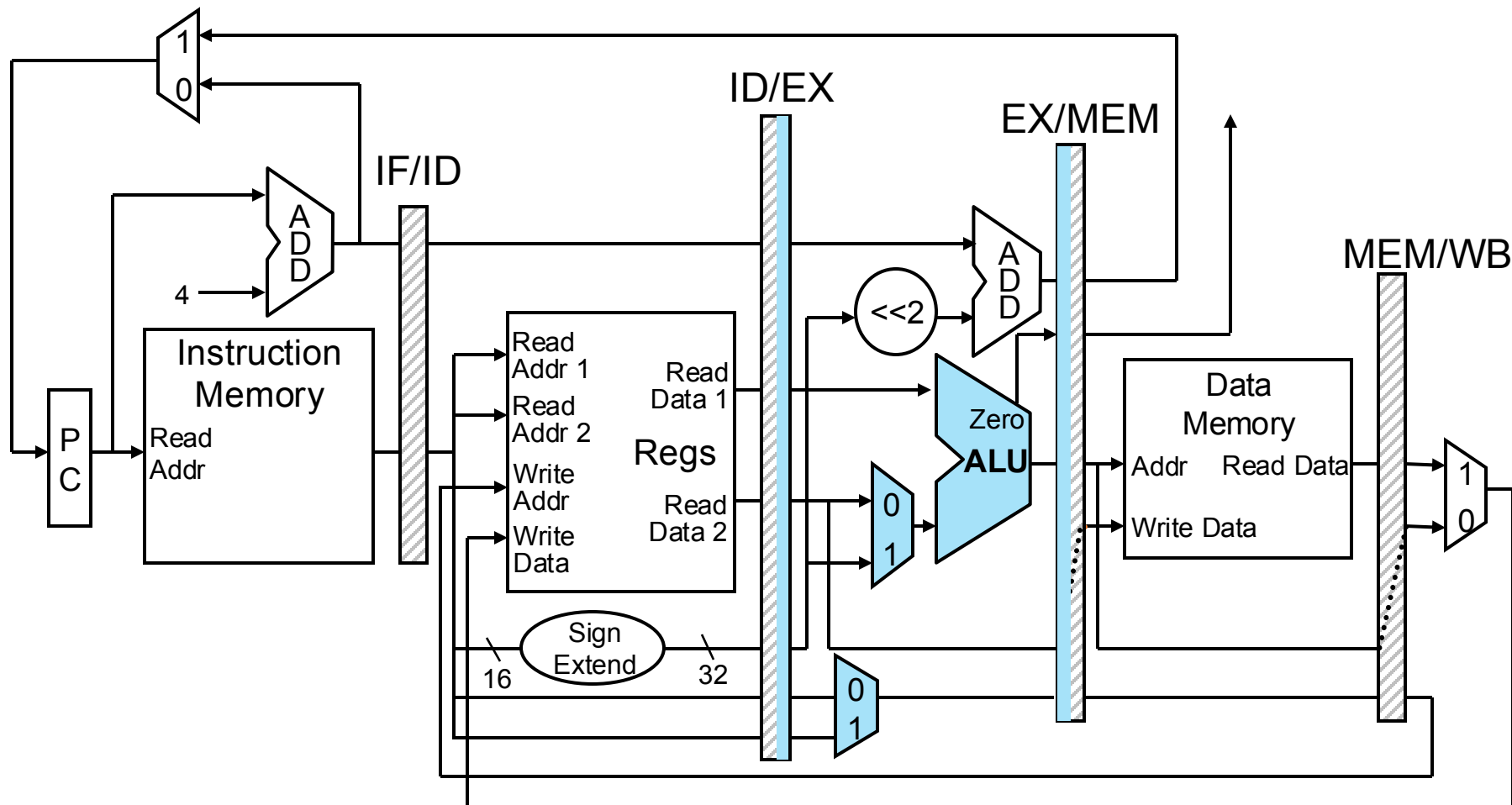
流水线的数据通路：ADD指令



ADD
Instruction Decode/
Register Fetch

寄存器读取
不存在立即数
存入ID/EX

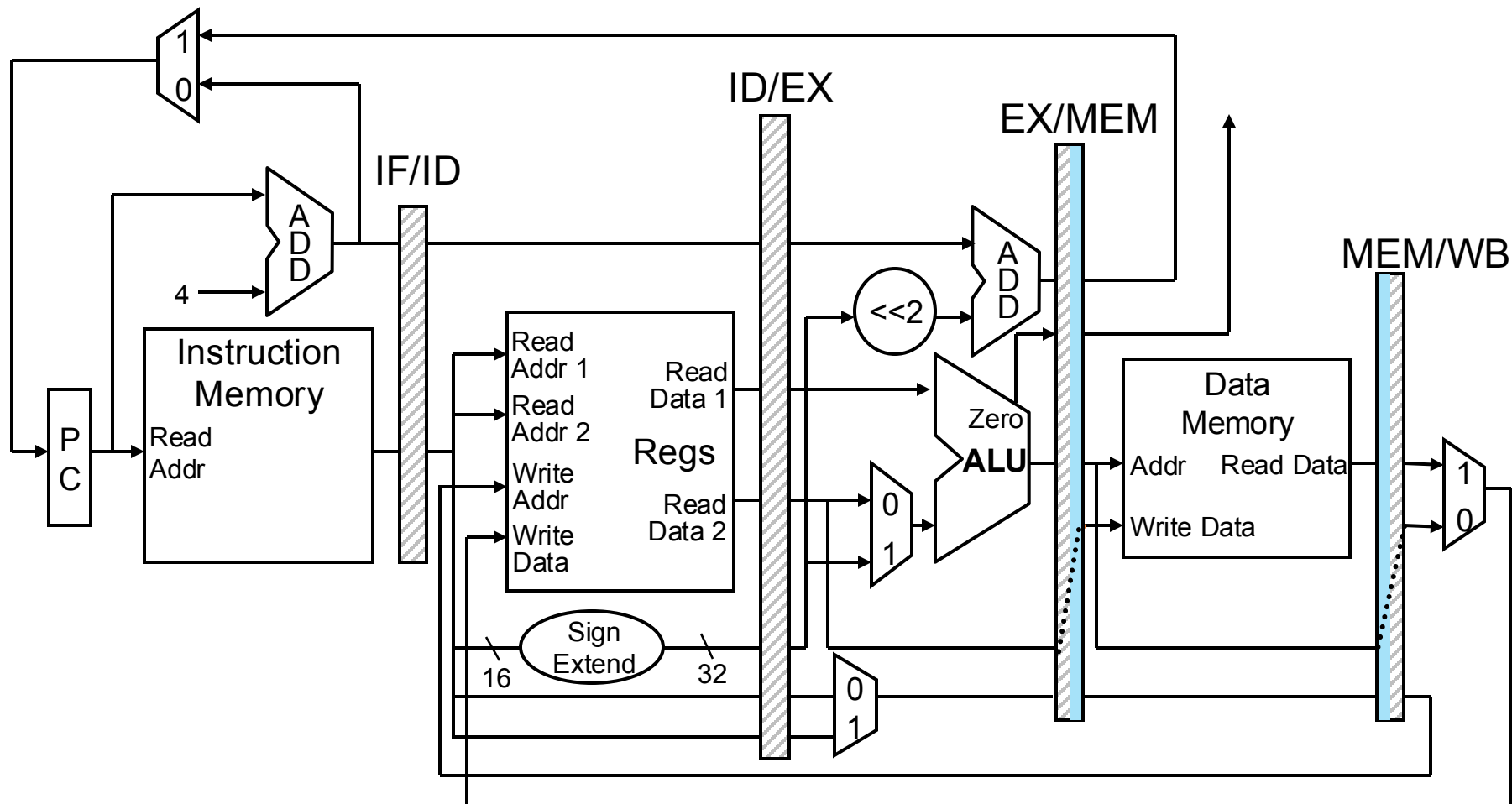
流水线的数据通路：ADD指令



ADD
Execution

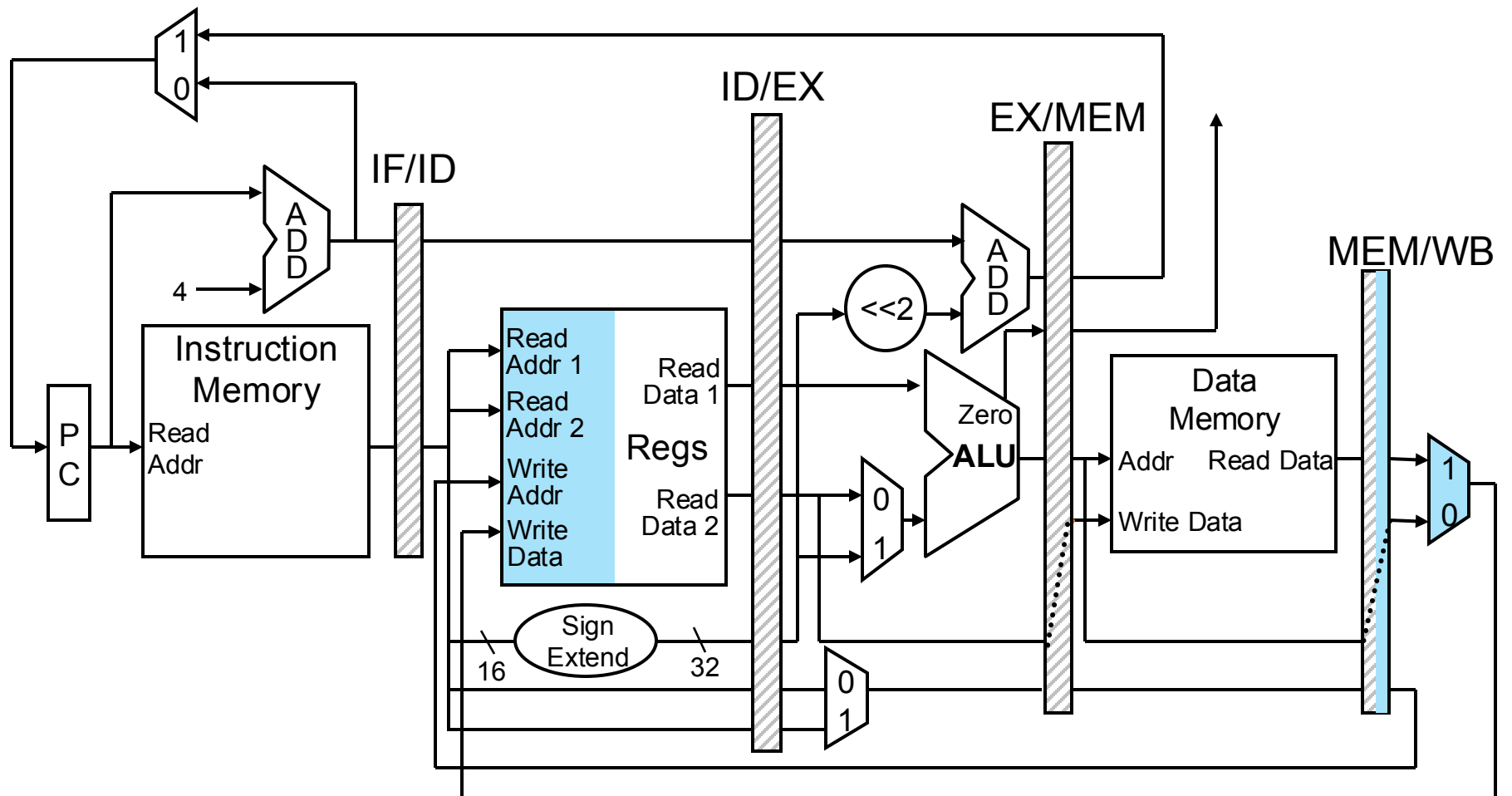
ALU计算结果
存入EX/MEM

流水线的数据通路：ADD指令

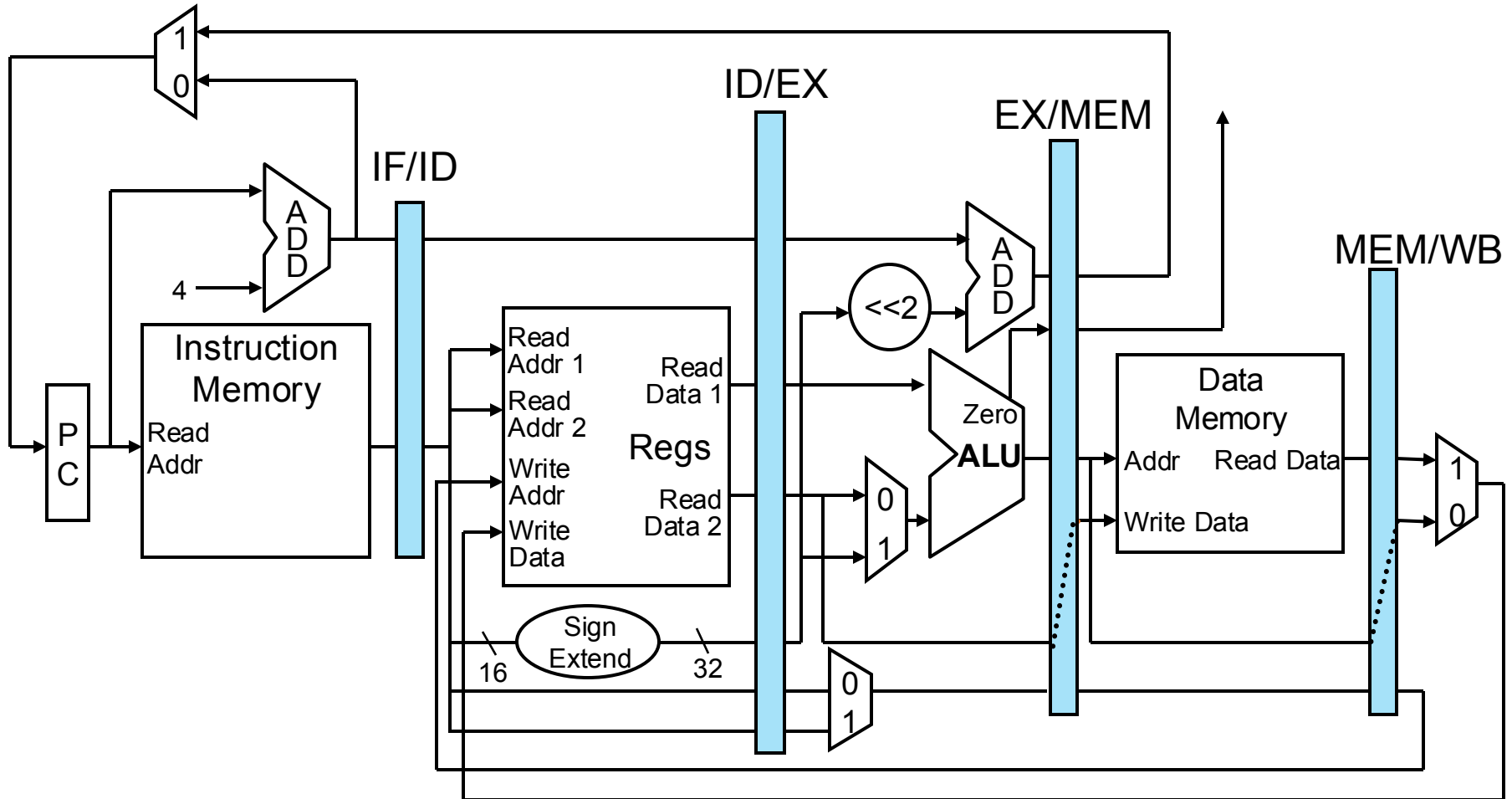


ADD无需读取Memory! $\longleftrightarrow$ ADD
Memory Access

流水线的数据通路：ADD指令

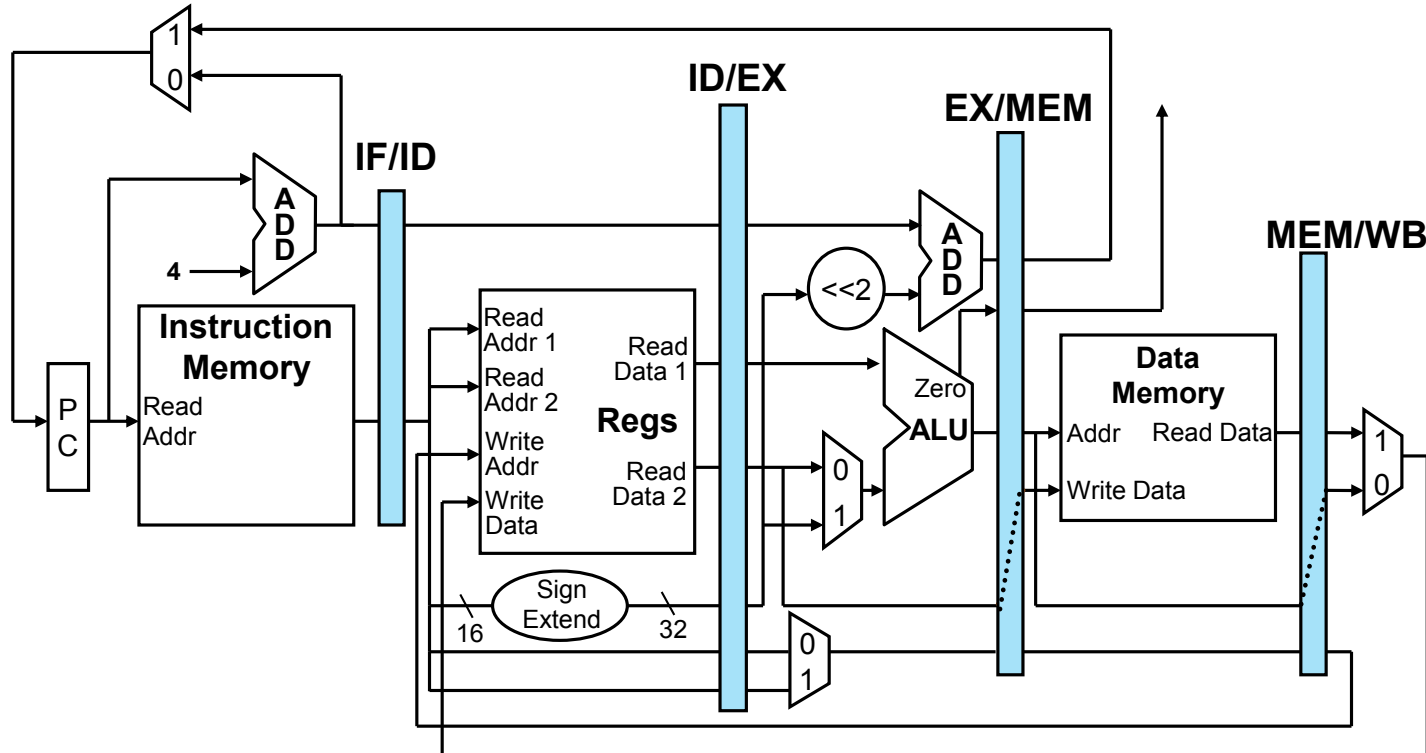


问题



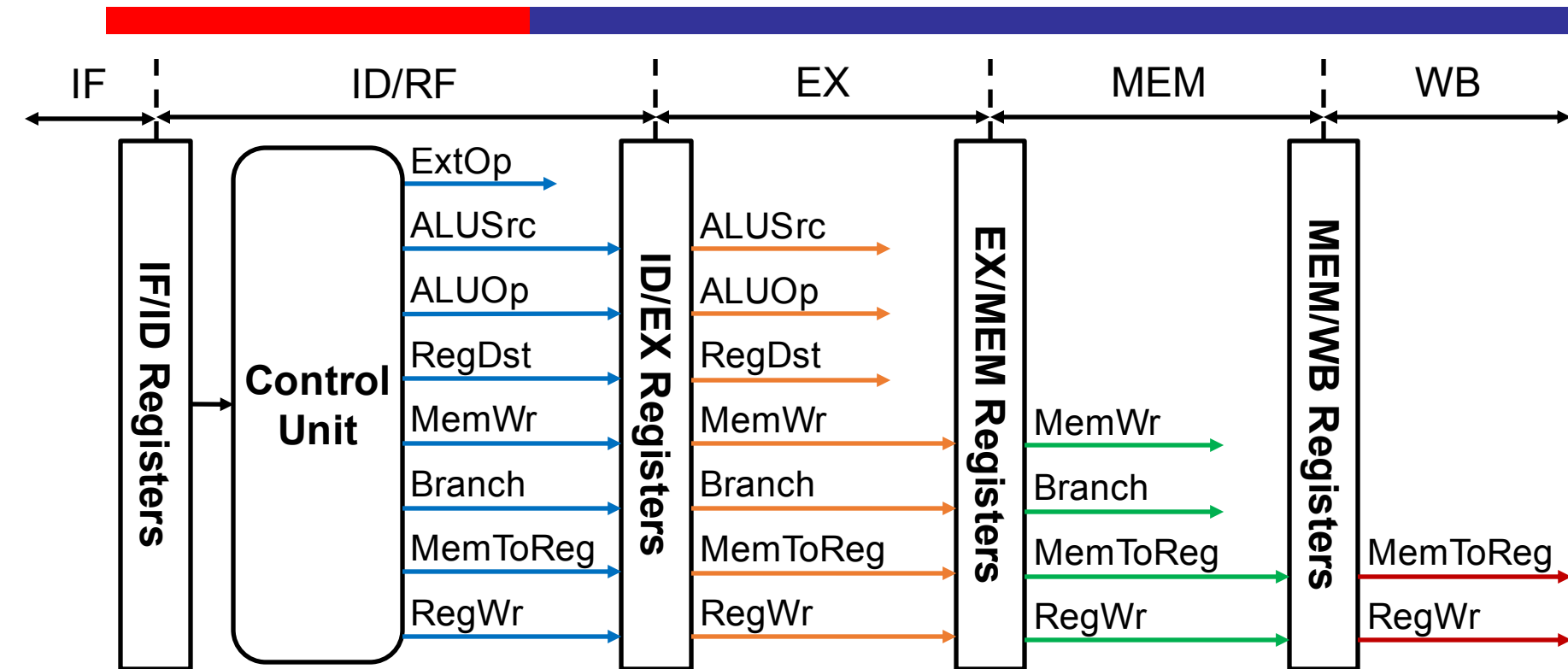
- 如何获得每个阶段的控制信号？

问题



- 控制信号在IF之后的 **ID/RF** 阶段产生
 - 由控制逻辑电路译码生成
- 控制信号在流水线中流动
 - 需要时使用，后面阶段需要时保留，不需要时丢弃

流水线的控制信号

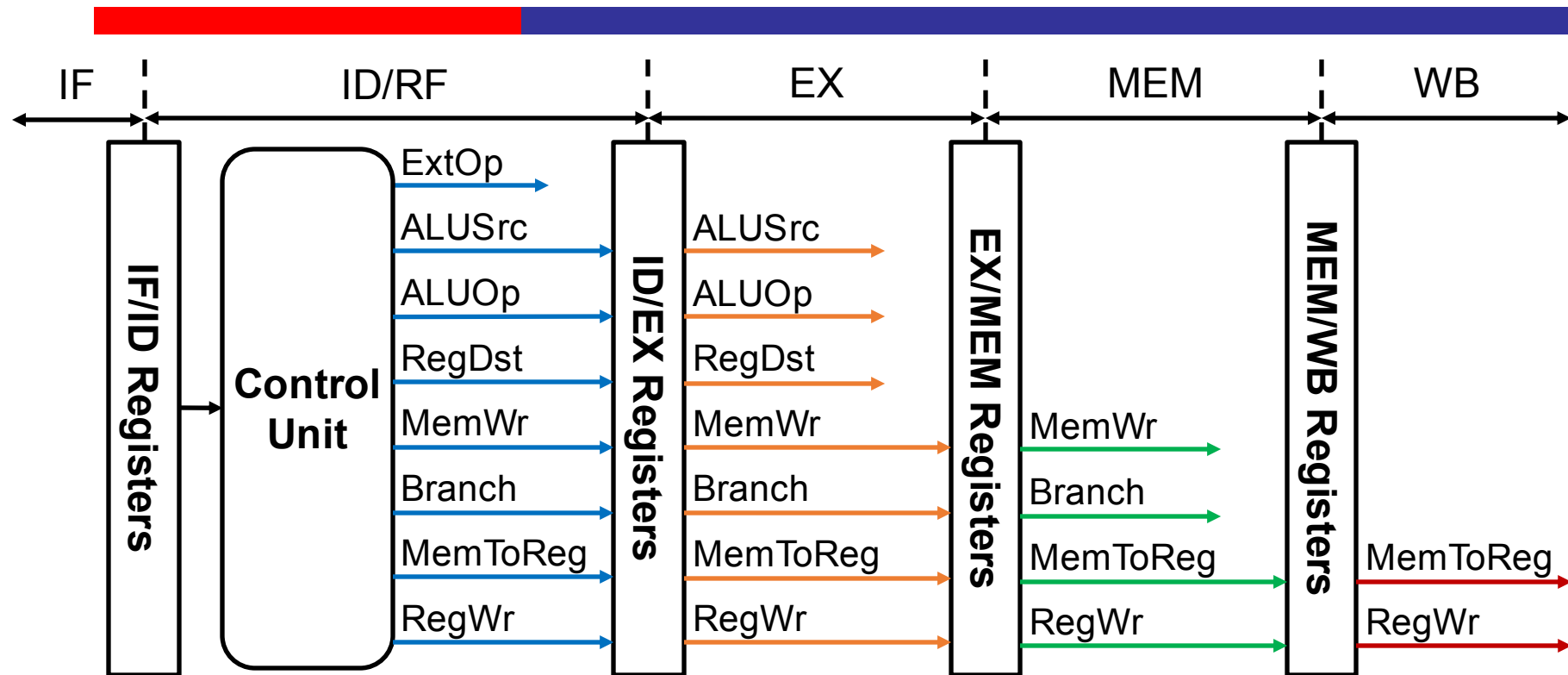


例子程序

I1: ADD R4, R3, R2
I2: AND R7, R6, R5
I3: LW R1, R9(10)

ExtOp=0
ALUSrc=0
ALUOp=10
RegDst=0
MemWr=0
Branch=0
MemToReg=0
RegWr=1
I1-ADD

流水线的控制信号



例子程序

I1: ADD R4, R3, R2

I2: AND R7, R6, R5

I3: LW R1, R9(10)

ExtOp=0

ALUSrc=0

ALUOp=10

RegDst=0

MemWr=0

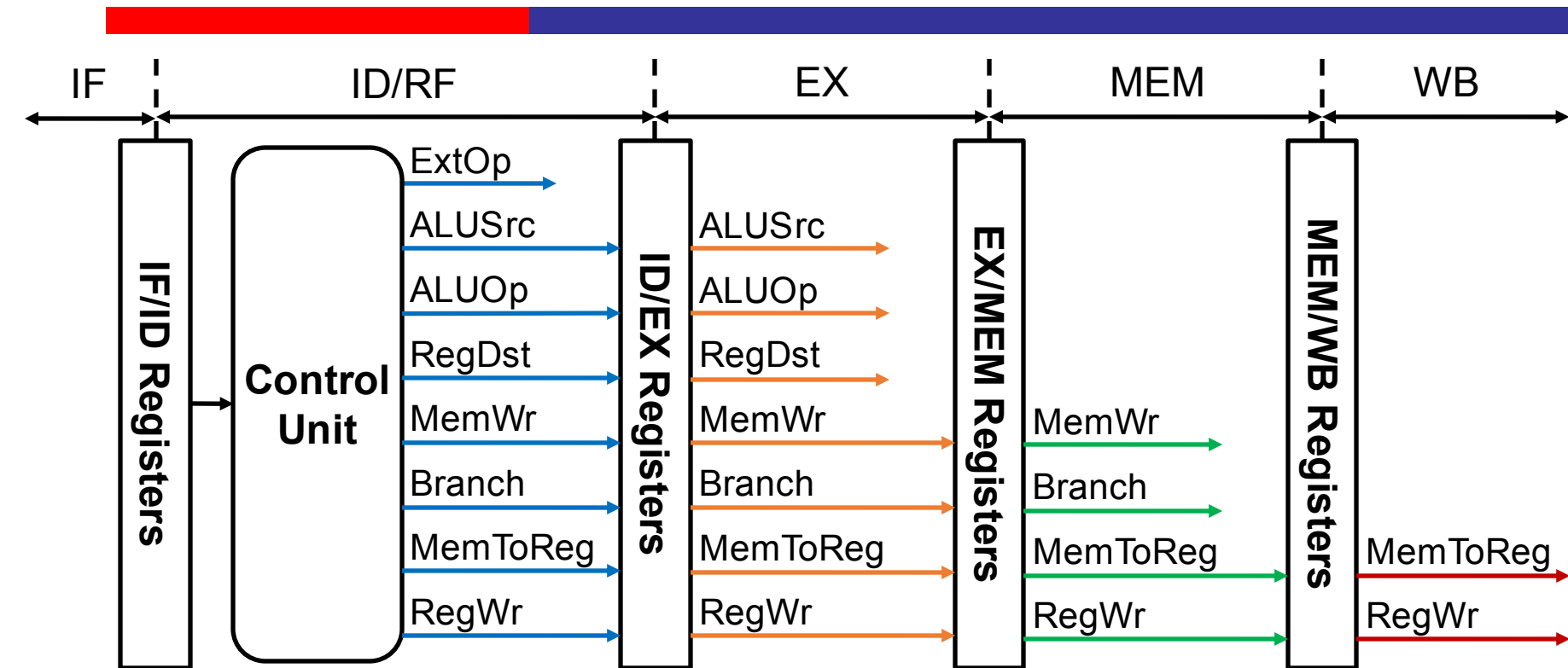
Branch=0

MemToReg=0

RegWr=1

I1-ADD

流水线的控制信号



例子程序

I1: ADD R4, R3, R2

I2: AND R7, R6, R5

I3: LW R1, R9(10)

ExtOp=0

ALUSrc=0

ALUOp=10

RegDst=0

MemWr=0

Branch=0

MemToReg=0

RegWr=1

I2-AND

ALUSrc=0

ALUOp=10

RegDst=0

MemWr=0

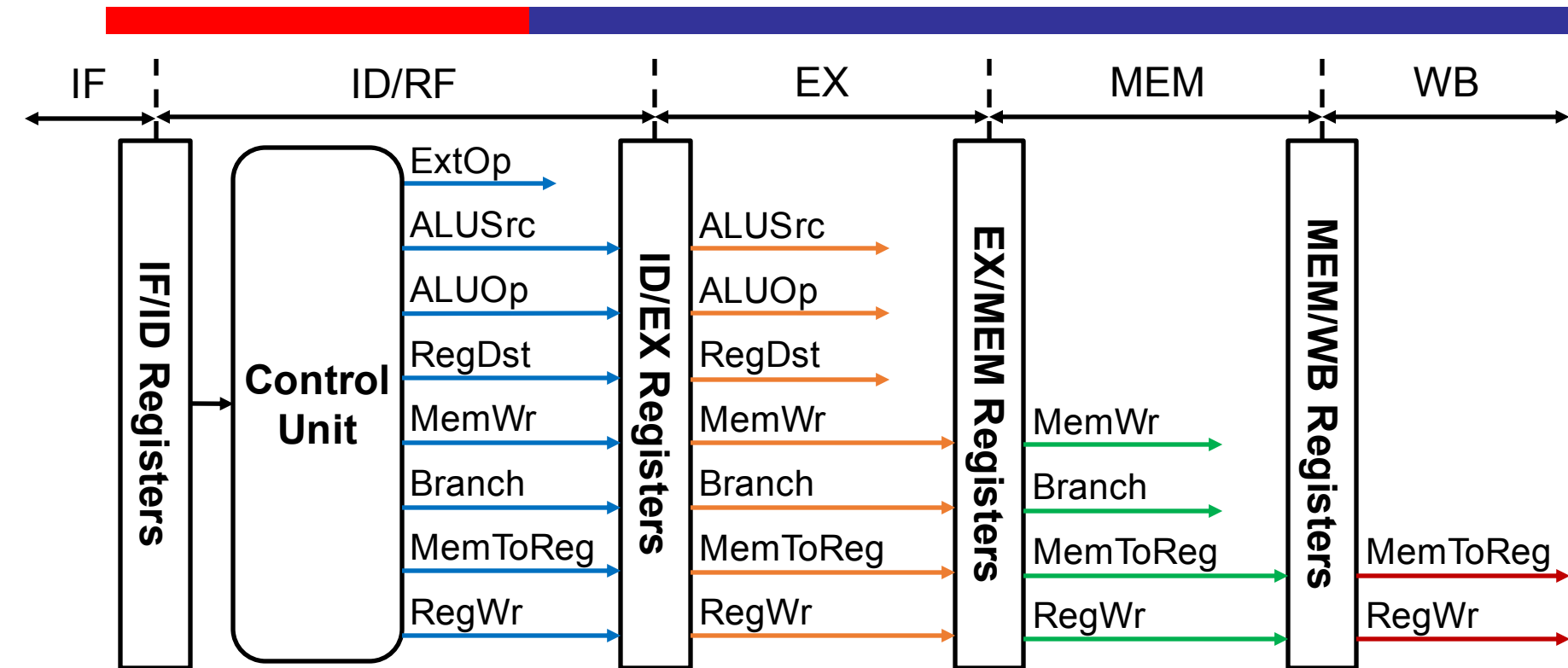
Branch=0

MemToReg=0

RegWr=1

I1-ADD

流水线的控制信号



例子程序

I1: ADD R4, R3, R2

I2: AND R7, R6, R5

I3: LW R1, R9(10)

ExtOp=0

ALUSrc=0

ALUOp=10

RegDst=0

MemWr=0

Branch=0

MemToReg=0

RegWr=1

I2-AND

ALUSrc=0

ALUOp=10

RegDst=0

MemWr=0

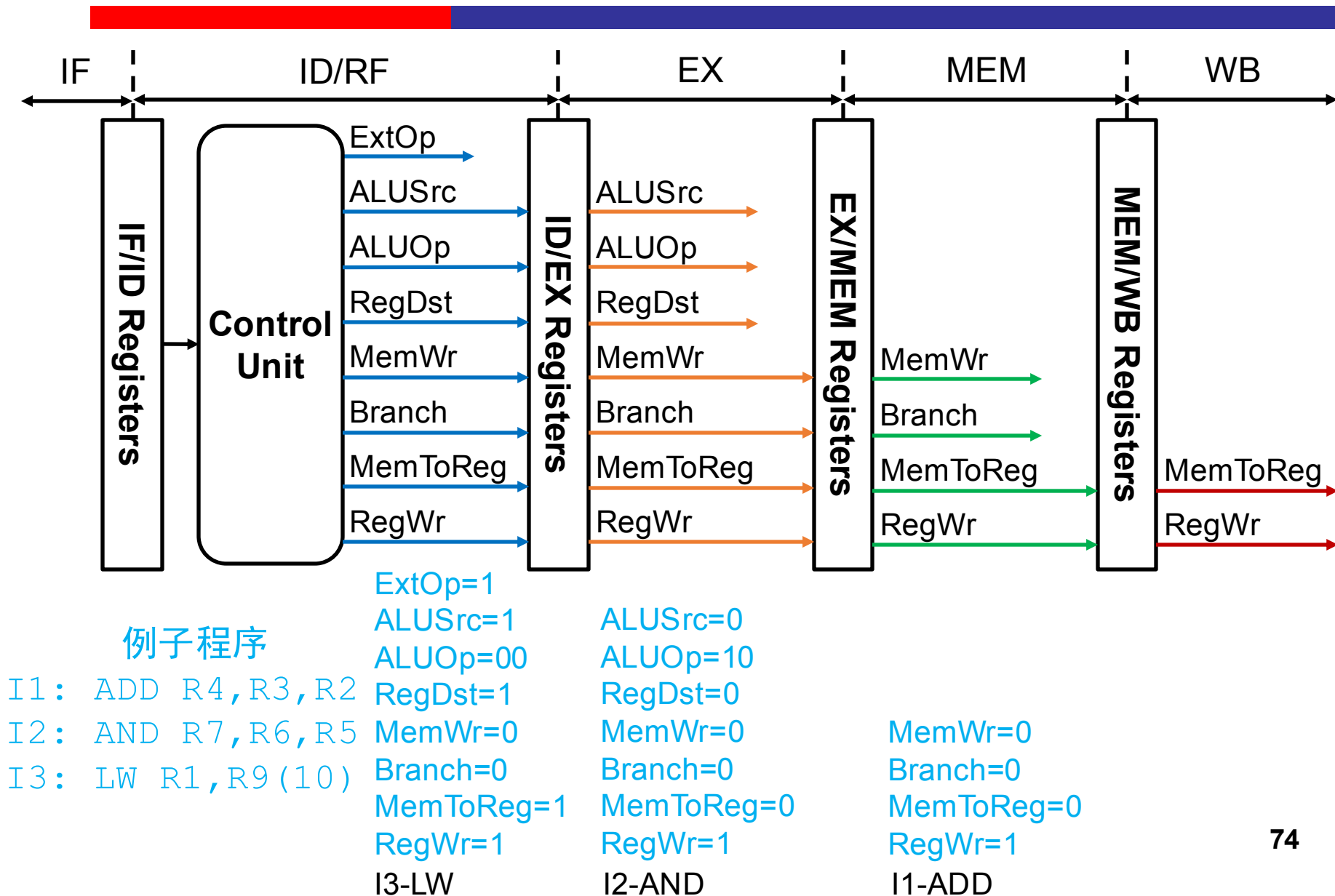
Branch=0

MemToReg=0

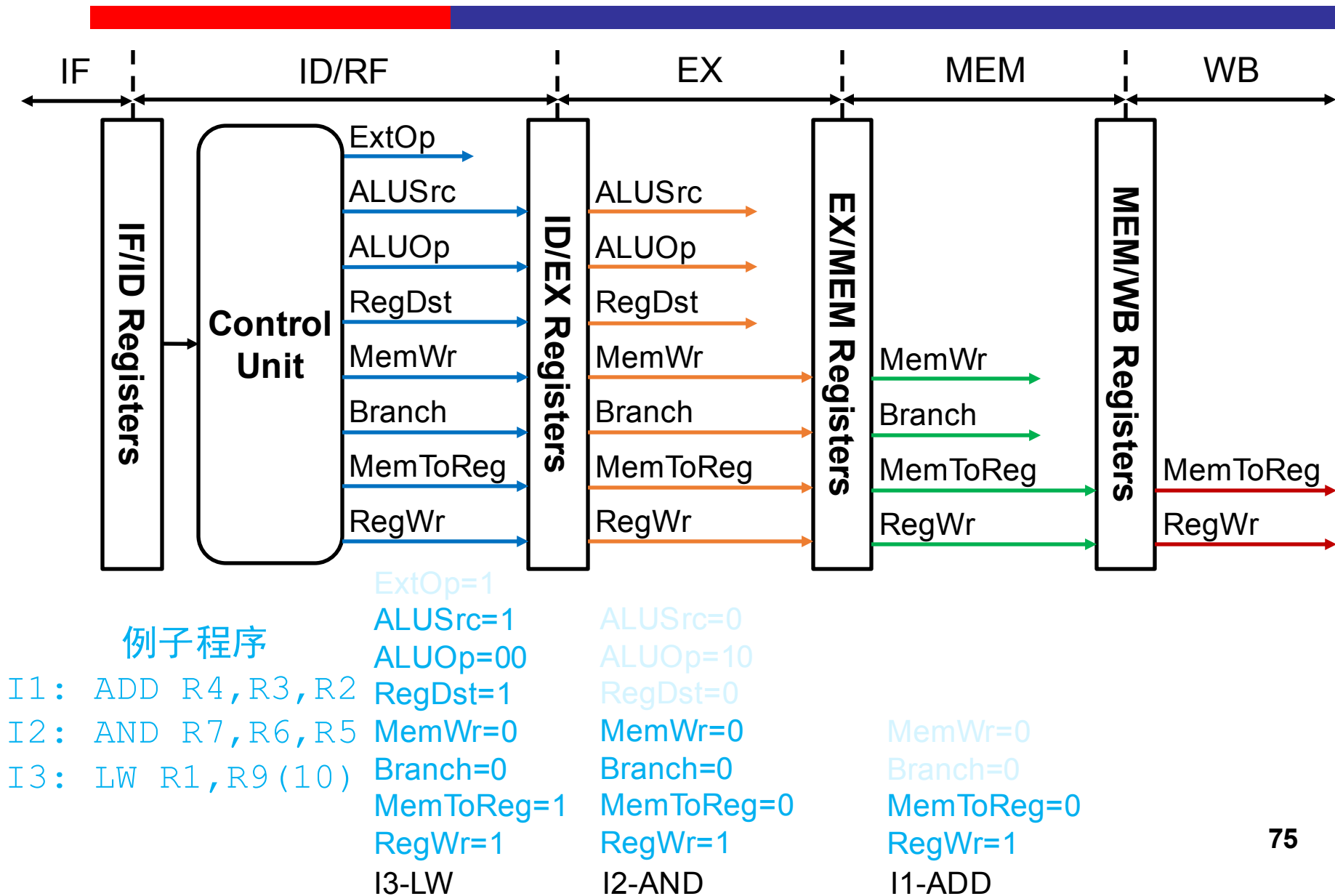
RegWr=1

I1-ADD

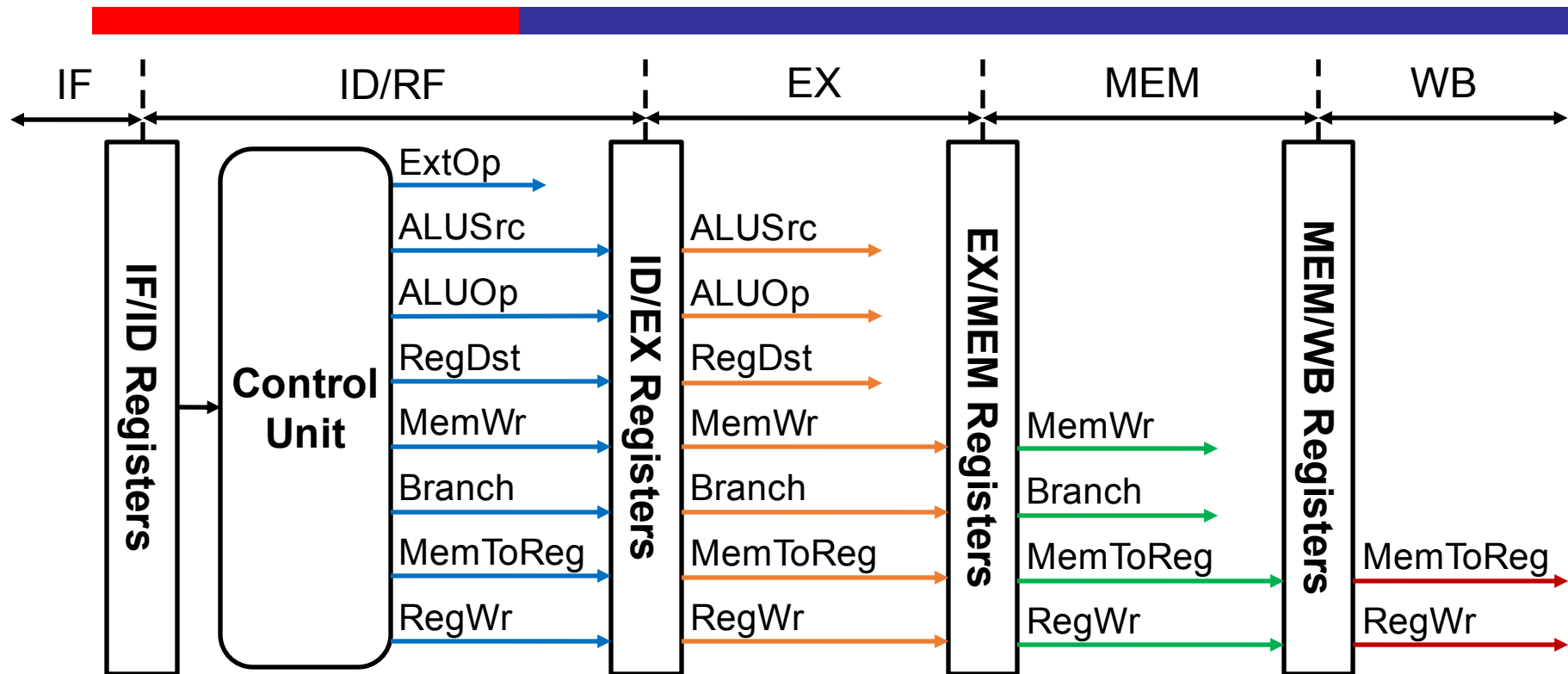
流水线的控制信号



流水线的控制信号



流水线的控制信号



例子程序

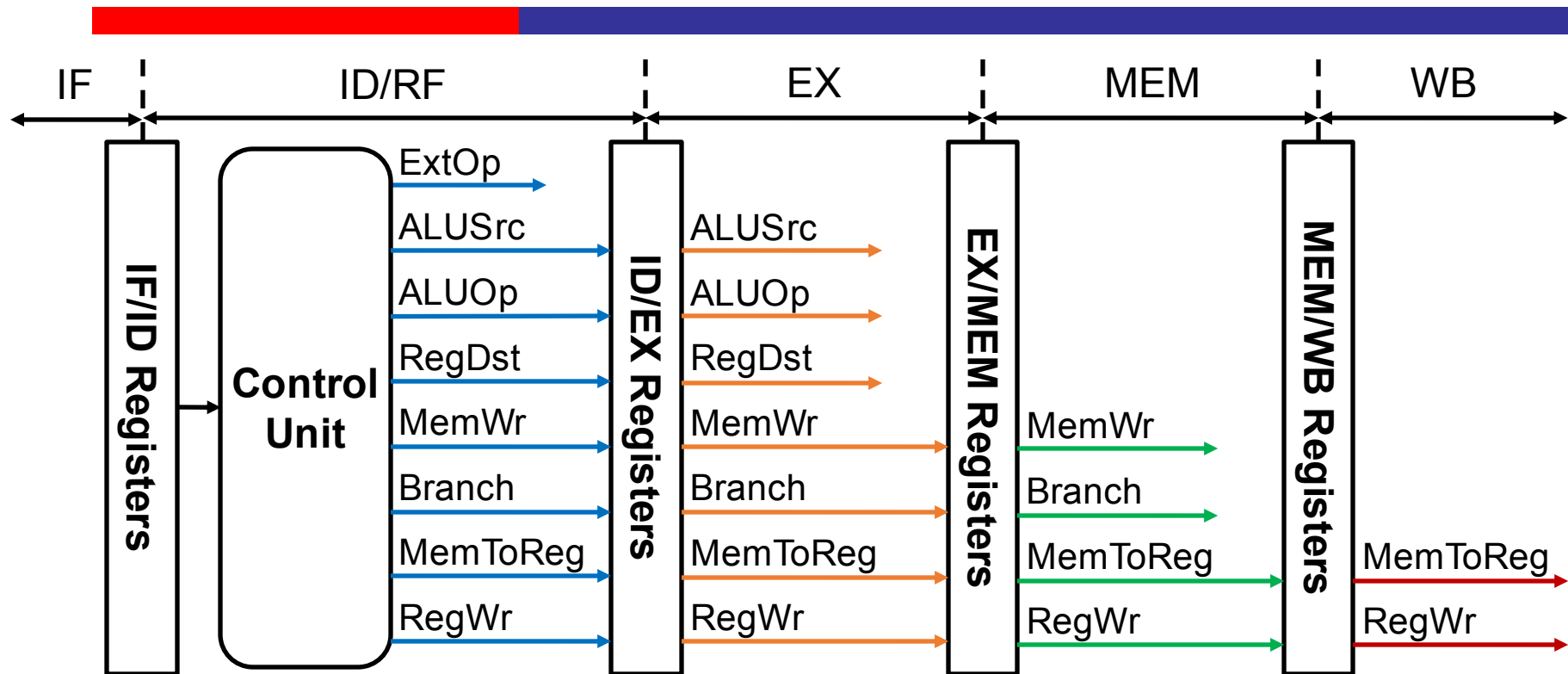
I1: ADD R4,R3,R2
I2: AND R7,R6,R5
I3: LW R1,R9(10)

ALUSrc=1
ALUOp=00
RegDst=1
MemWr=0
Branch=0
MemToReg=1
RegWr=1
I3-LW

MemWr=0
Branch=0
MemToReg=0
RegWr=1
I2-AND

MemToReg=0
RegWr=1
I1-ADD

流水线的控制信号



例子程序

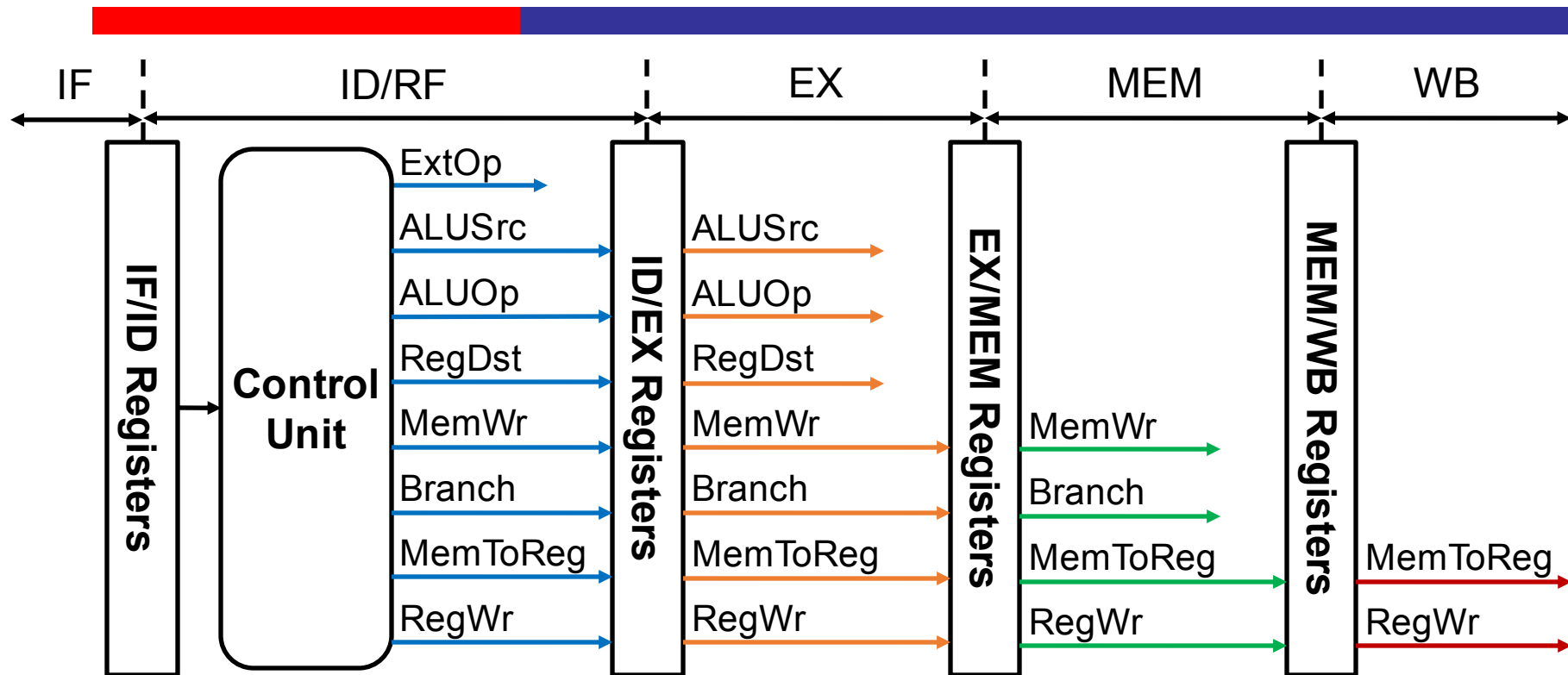
I1: ADD R4,R3,R2
I2: AND R7,R6,R5
I3: LW R1,R9(10)

ALUSrc=1
ALUOp=00
RegDst=1
MemWr=0
Branch=0
MemToReg=1
RegWr=1
I3-LW

MemWr=0
Branch=0
MemToReg=0
RegWr=1
I2-AND

MemToReg=0
RegWr=1
I1-ADD

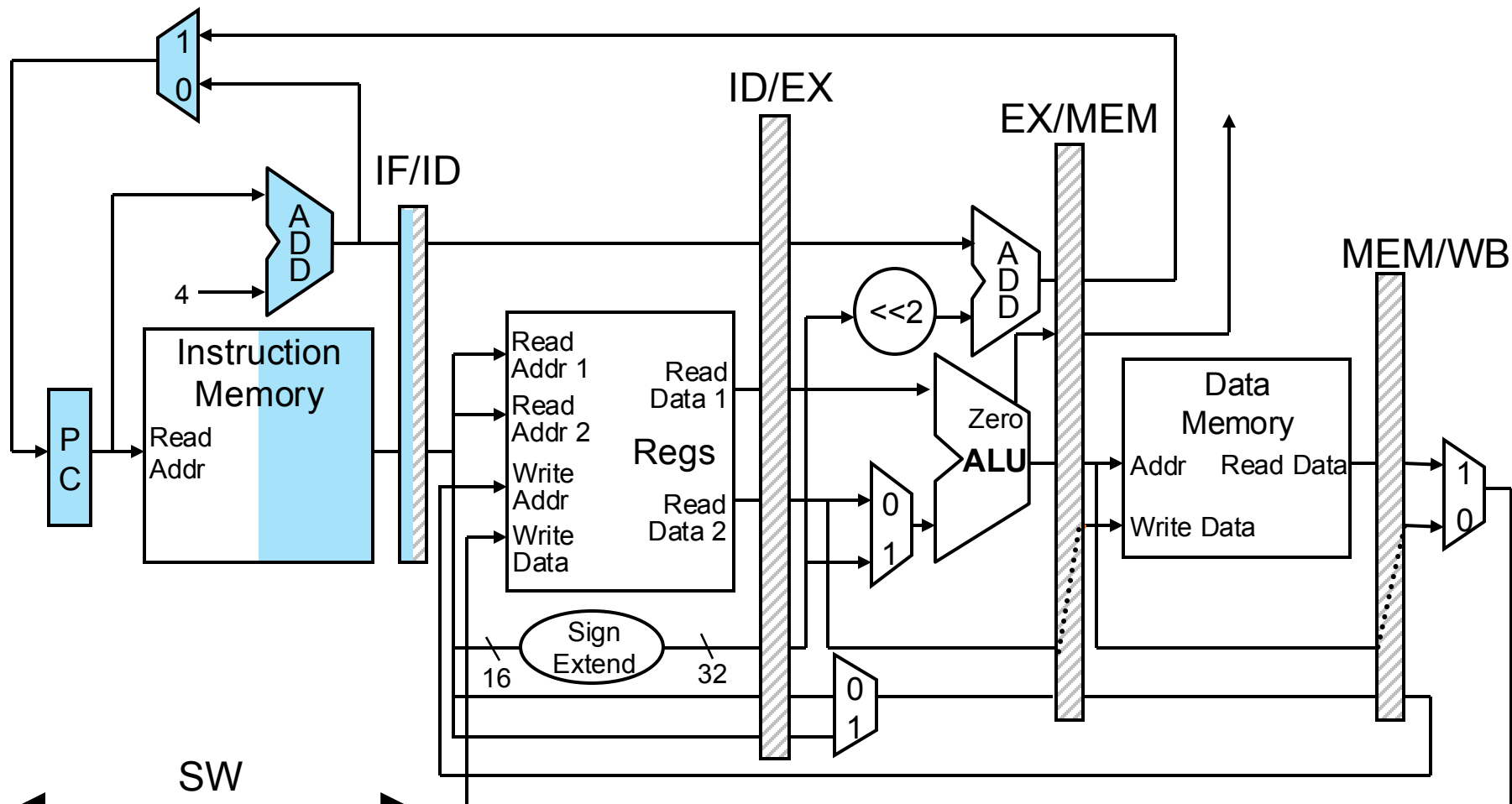
流水线的控制信号



控制信号寄存器
ID/EX 寄存器
EX/MEM 寄存器
MEM/WB 寄存器

记：MEM/WB.RegWr 代表访问
MEM/WB寄存器中的RegWr控制信号

流水线的数据通路：SW指令



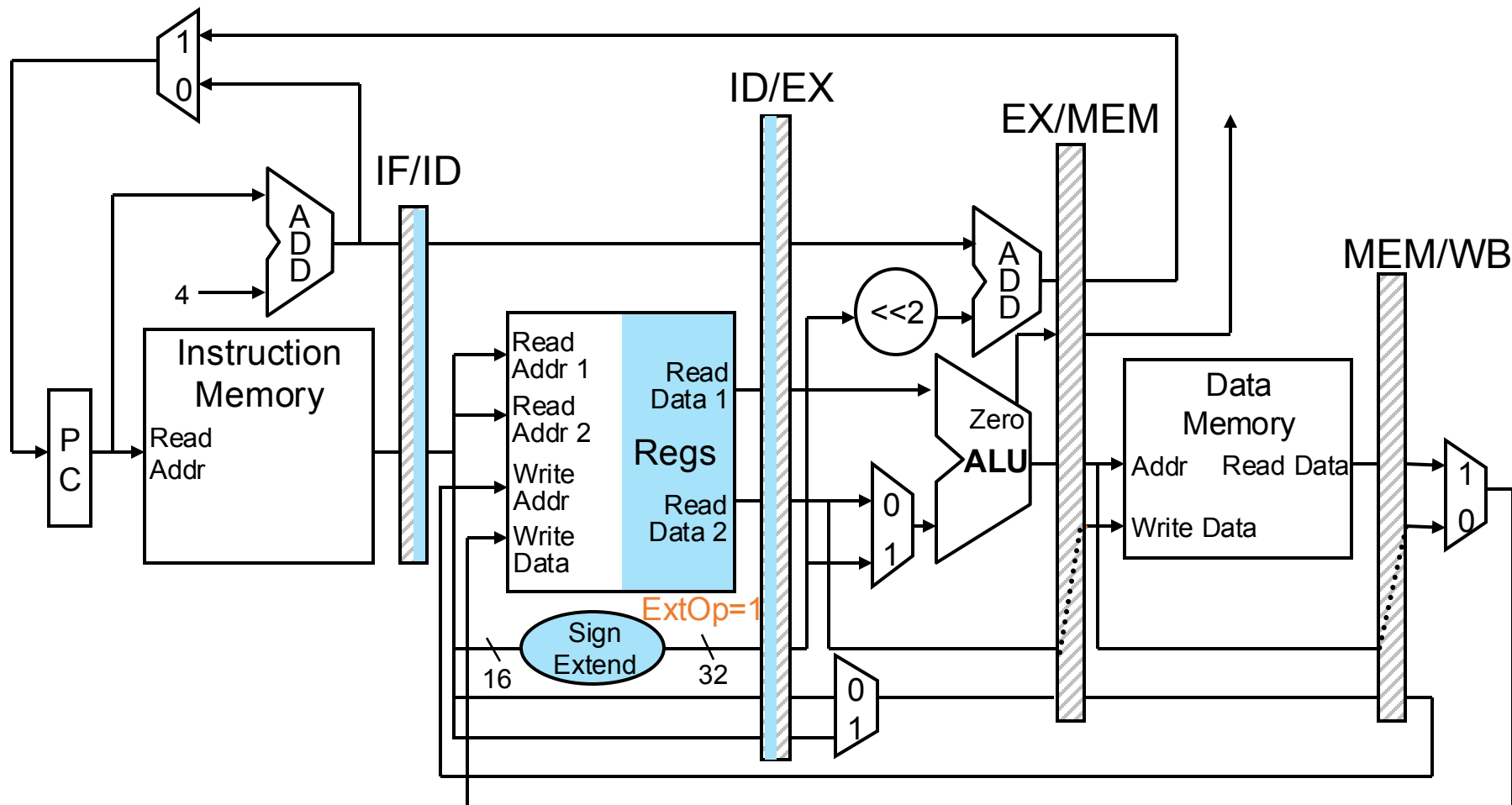
SW

Instruction Fetch

取指令并存入IF/ID

PC=PC+4

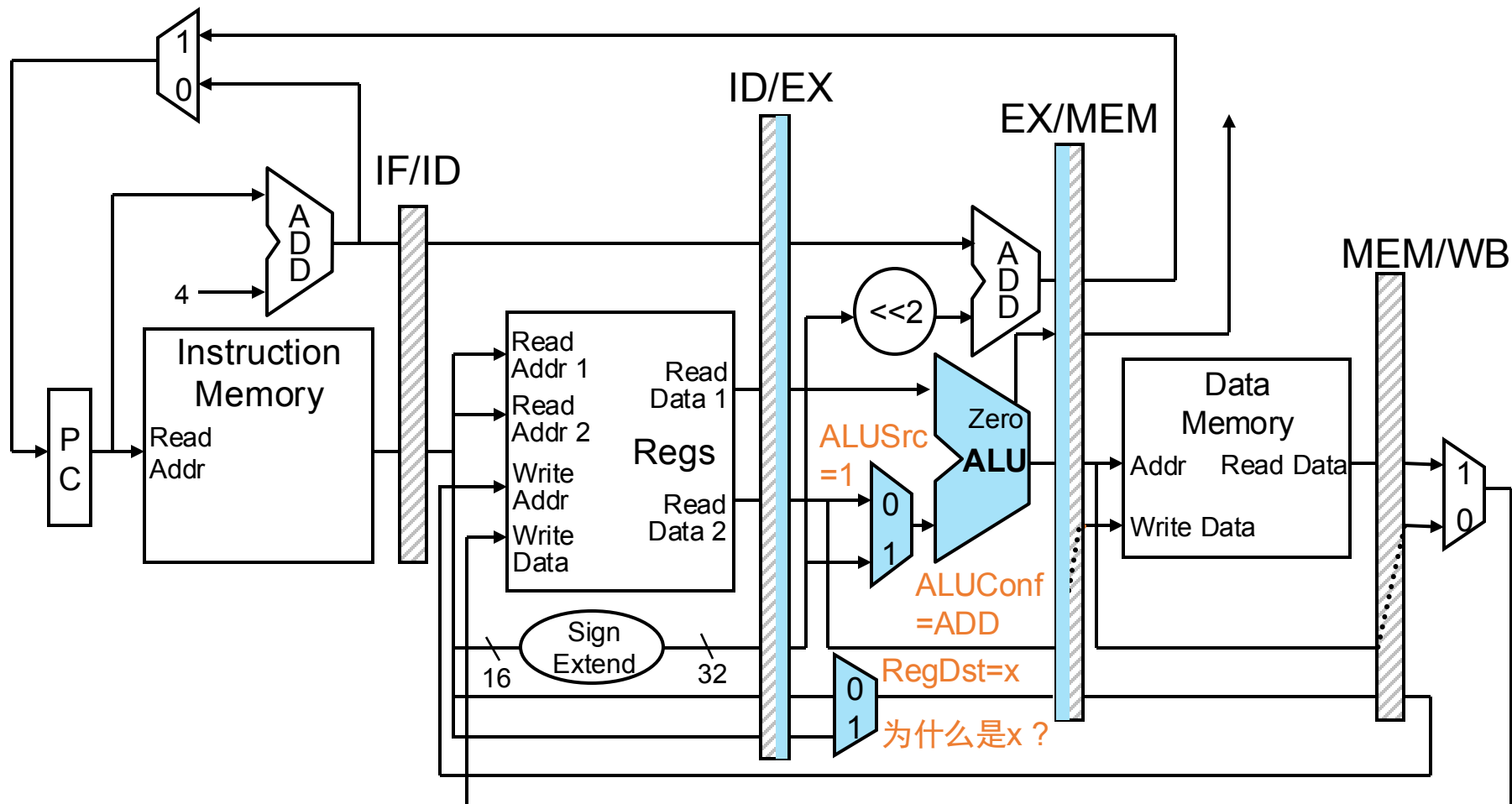
流水线的数据通路：SW指令



SW
Instruction Decode/
Register Fetch

寄存器读取 &
立即数扩展
存入ID/EX

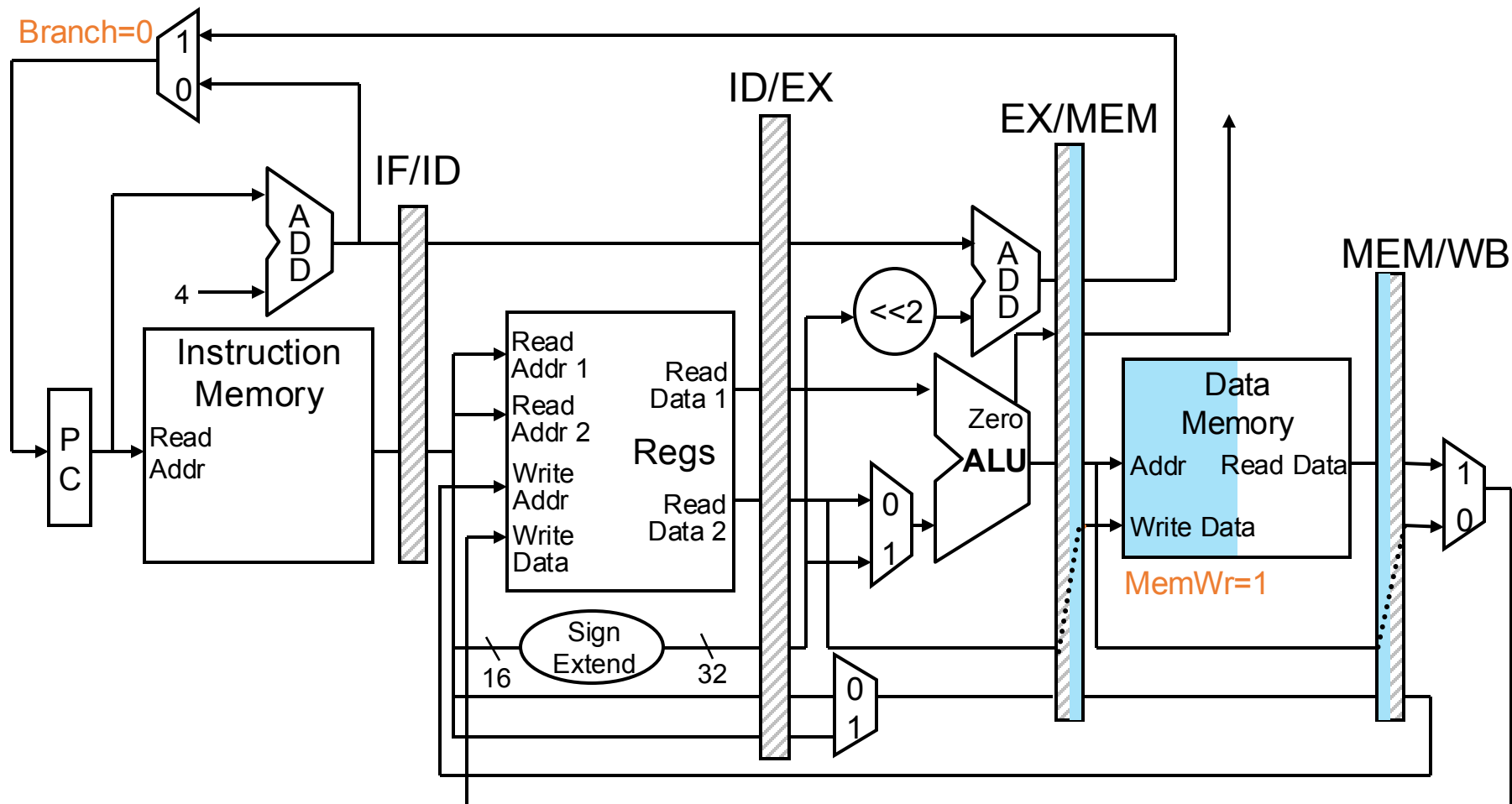
流水线的数据通路：SW指令



SW
Execution

ALU计算访存地址
存入EX/MEM

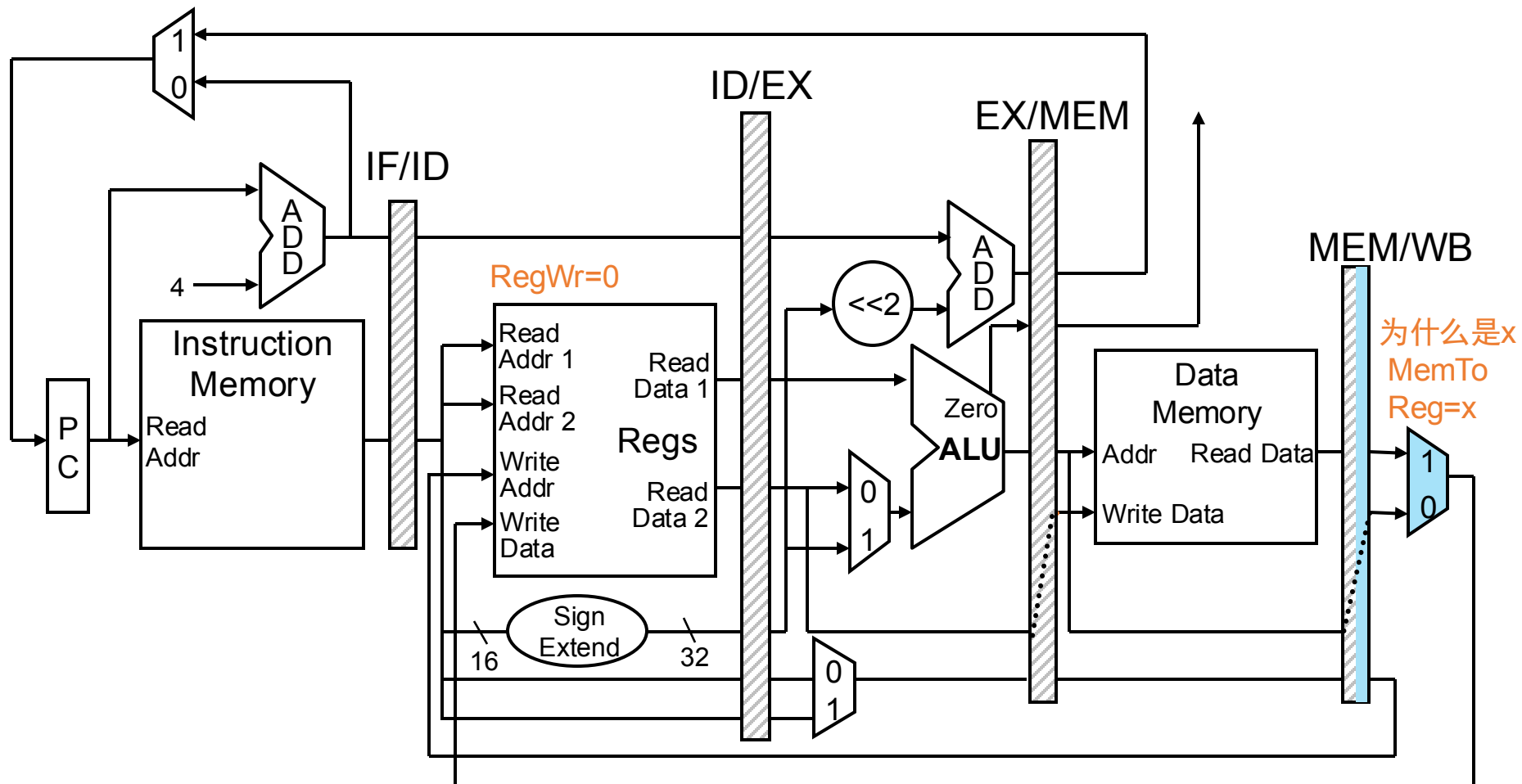
流水线的数据通路：SW指令



数据内存写入

SW
Memory Access

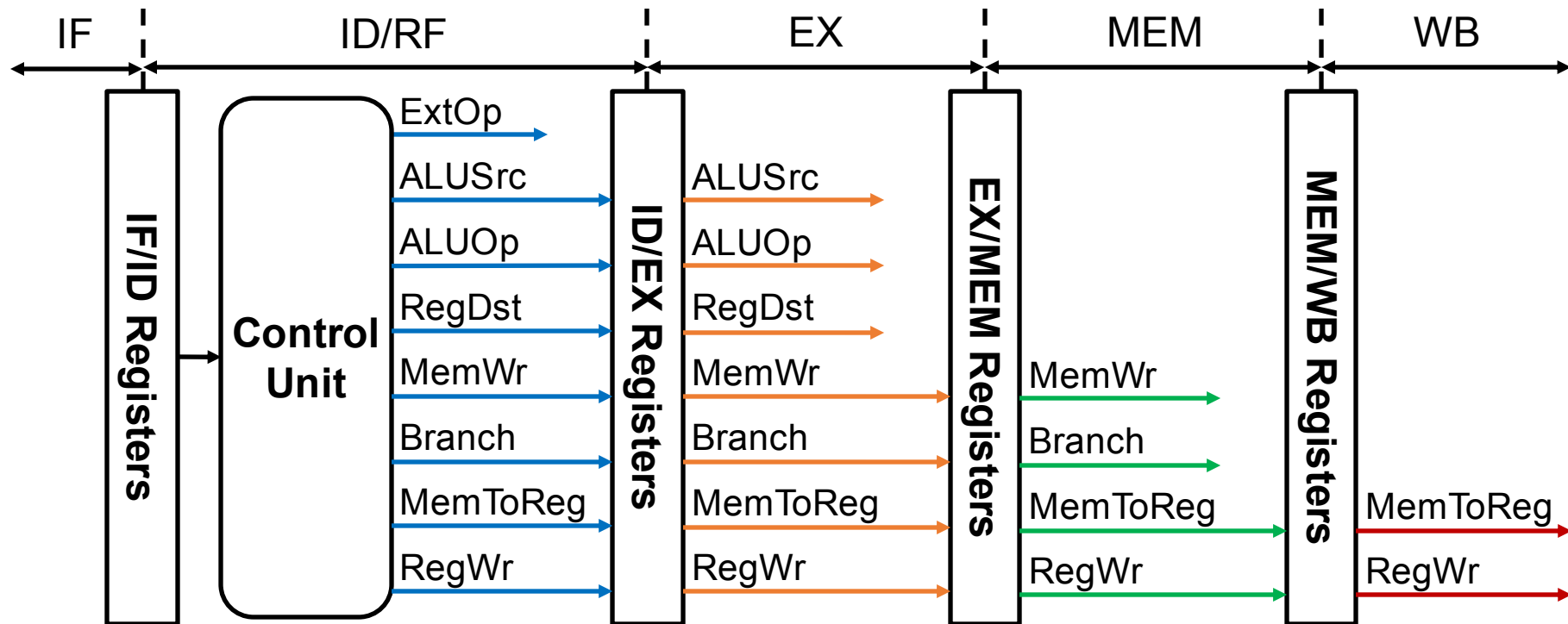
流水线的数据通路：SW指令



Write Back
无数据写回寄存器堆

SW
83

另一个例子：结果正确吗？



例子程序

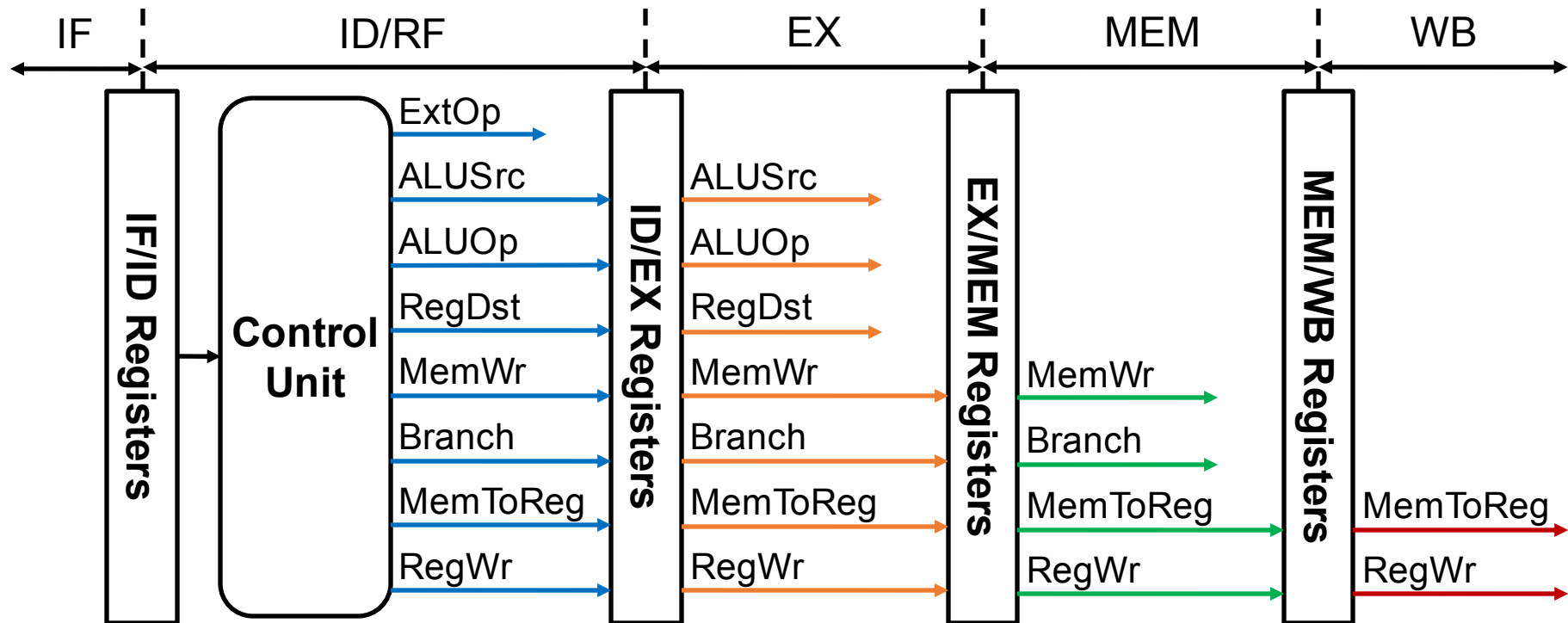
```
I1: LW R2, R9(10)
I2: ADD R4, R3, R2
I3: AND R6, R5, R4
```

ALUSrc=0
ALUOp=10
RegDst=0
MemWr=0
Branch=0
MemToReg=0
RegWr=1
I3-AND

MemWr=0
Branch=0
MemToReg=0
RegWr=1
I2-ADD

MemToReg=1
RegWr=1 84
I1-LW

另一个例子：结果正确吗？



例子程序

```
I1: LW R2, R9(10)
I2: ADD R4, R3, R2
I3: AND R6, R5, R4
```

寄存器值还未被
更新就被取出！

ALUSrc=0
ALUOp=10
RegDst=0
MemWr=0
Branch=0
MemToReg=0
RegWr=1
I3-AND

MemWr=0
Branch=0
MemToReg=0
RegWr=1
I2-ADD

MemToReg=1
RegWr=1 85
I1-LW

本节课目录

- 复习回顾与流水线基本概念 P5
- 流水线数据通路 P42
- 流水线的冒险 P89
 - 结构冒险
 - 数据冒险
 - 控制冒险
- 数据通路的实现（考虑冒险）

流水线冒险

- 各个指令、流水线的各个阶段之间可能存在关联
 - 前面的指令或运行阶段会影响后续的指令或运行阶段
- 下一条指令可能某些无法获得的资源
 - 访问权？新的数据？下一个执行步骤？

这些限制就是：**流水线的冒险（Pipeline Hazards）**

问题

- 冒险如何影响流水线化的运算？

流水线冒险

- **结构冒险 (Structural Hazard or Resource Conflict)**
 - 两条指令需要使用同样的硬件
- **数据关联冒险 (Data Hazard)**
 - 指令数据依赖于流水线中运行的其他指令结果
- **控制冒险 (Control Hazard)**
 - 取指令的PC依赖于流水线中指令的结果

流水线冒险

- **结构冒险 (Structural Hazard or Resource Conflict)**
 - 两条指令需要使用同样的硬件
- 数据关联冒险 (Data Hazard)
 - 指令依赖于流水线中运行的其他指令结果
- 控制冒险 (Control Hazard)
 - 取指令依赖于流水线中指令的结果

流水线：结构冒险

- 资源使用冲突

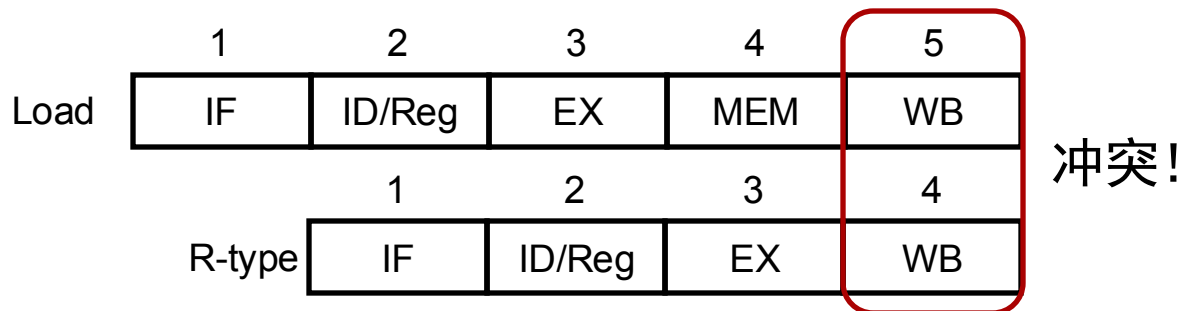
- 定义为：两条指令需要同时访问相同的硬件资源

- 结构冒险的例子

- 如果对load指令采用5级流水，第5级写回寄存器

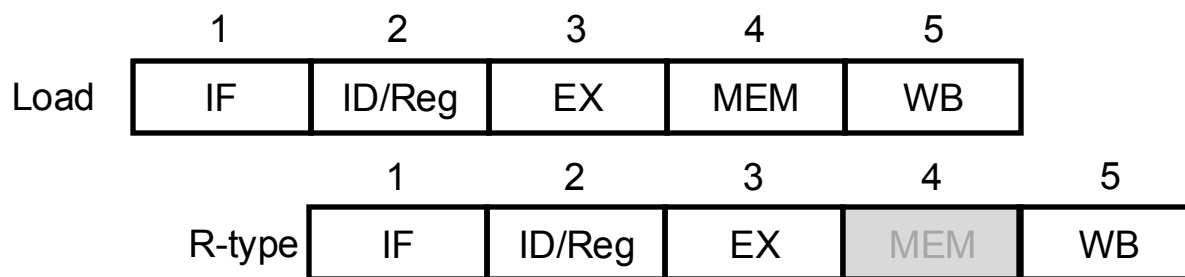
- 如果对R型指令采用4级流水，第4级写回寄存器

- 在Write Back阶段产生冲突



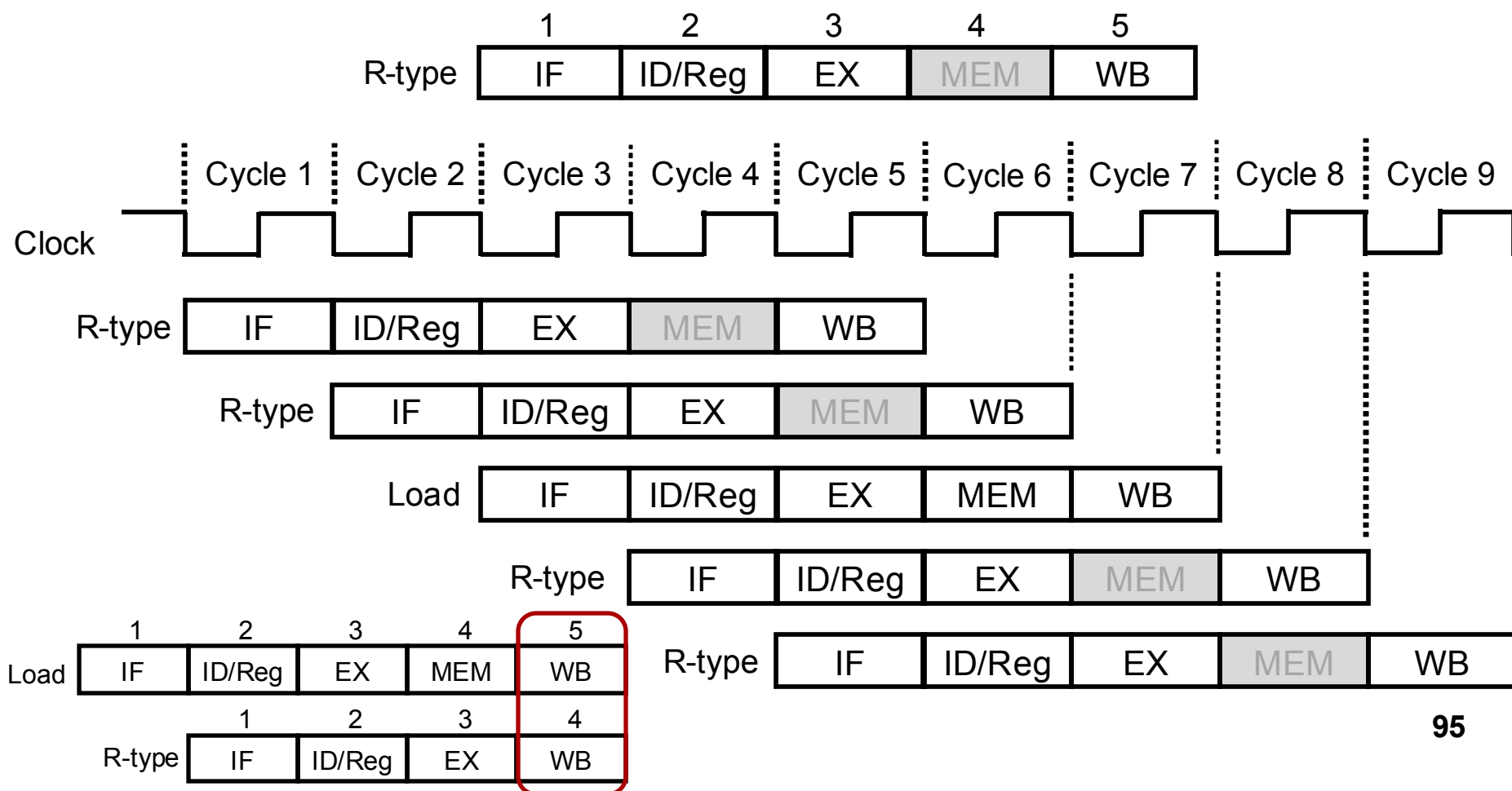
流水线：结构冒险

- 资源使用冲突
 - 定义为：两条指令需要同时访问相同的硬件资源
- 结构冒险的例子
 - 如果对load指令采用5级流水，第5级写回寄存器
 - 如果对R型指令采用4级流水，第4级写回寄存器
 - 在Write Back阶段产生冲突



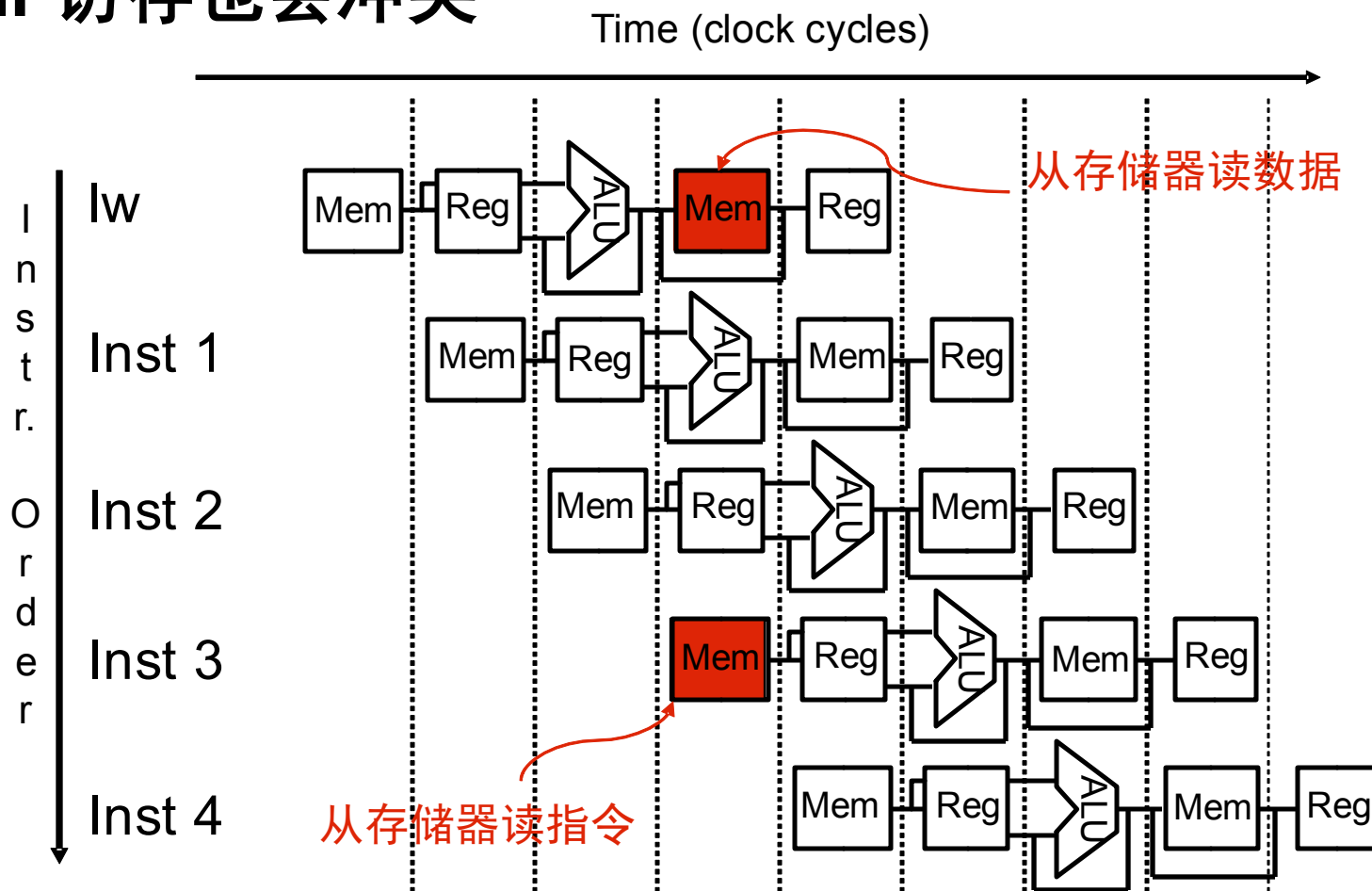
流水线：结构冒险

- 如果将R型指令的WB阶段延迟一个周期，采用5级流水，MEM阶段对于R型指令来说是空的



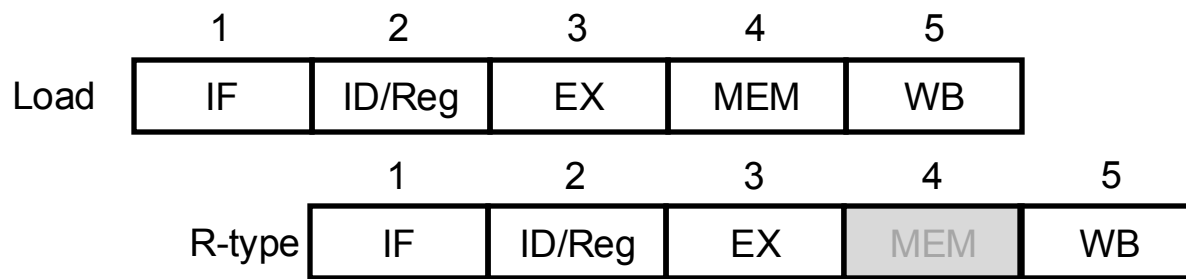
流水线：结构冒险

- 如果只有一个存储器和一个读取接口：LW访存与IF访存也会冲突



流水线：结构冒险

- 产生的原因
 - 流水执行的多条指令在同一个周期使用同一个硬件资源
- 解决方法：
 - 增加资源
 - 增加运算单元：例如PC+4和ALU运算分离
 - 采用指令存储器和数据存储器分离的哈佛体系结构
 - 调整指令对资源占用的周期



流水线：结构冒险

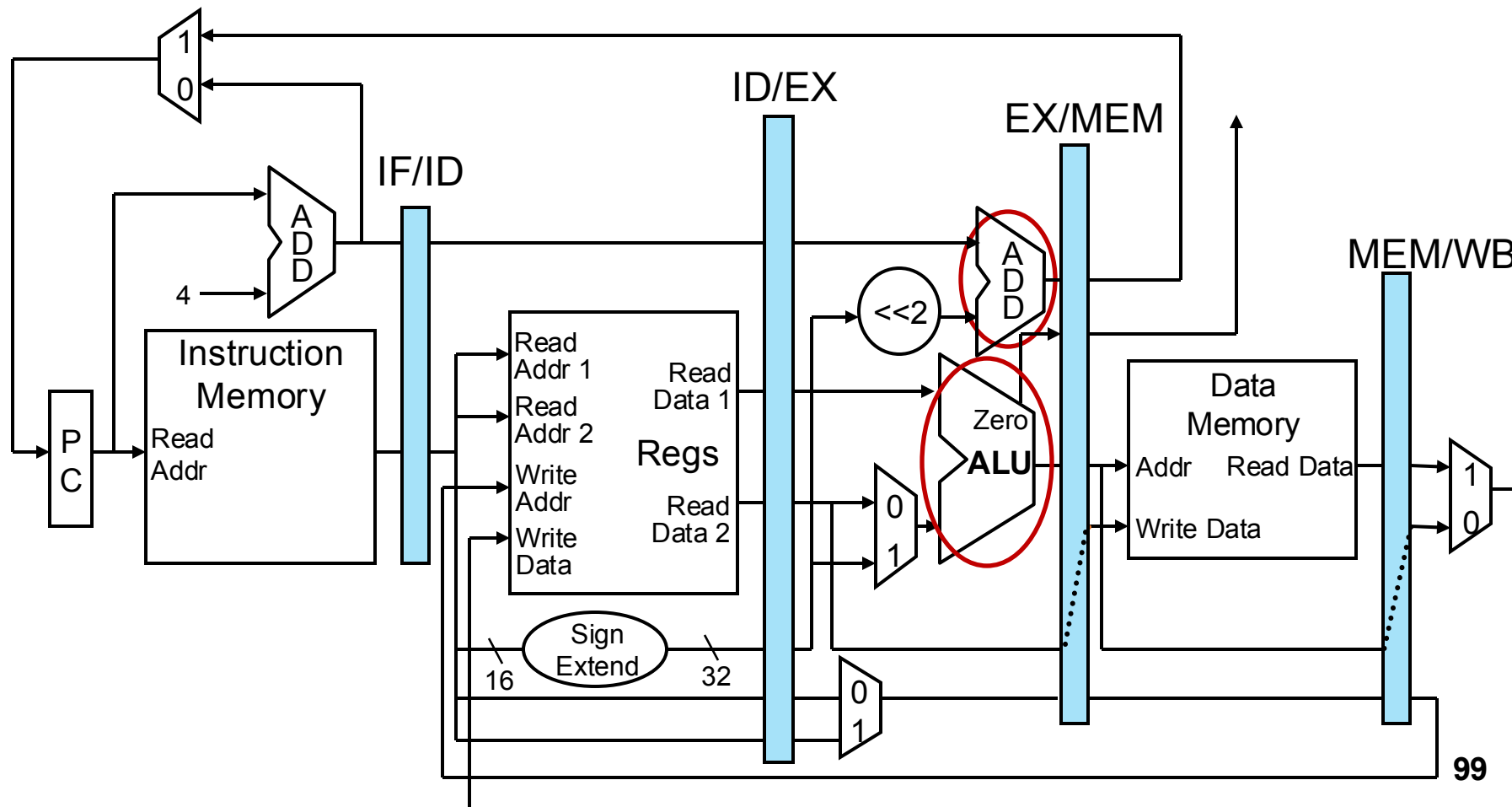
- 转化为流水线，结构冒险有哪些
 - 表格中不在同一行，却使用相同资源就可能引起结构冒险
 - **ALU使用冲突**
 - **寄存器堆写入冲突**

R类型	存储器访问	分支
IR <= MemInst[PC]; PC <= PC + 4		
op <= IR[31:26]; A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]]; ALUOut <= PC + signext(IR[15:0]) << 2		
ALUOut <= A op B	ALUOut <= A + signext(IR[15:0])	if (A = B) PC <= ALUOut
Reg[IR[15:11]] <= ALUOut	Lw:MDR <= MemData[ALUOut]; Sw:MemData[ALUOut] <= B	
	Reg[IR[20:16]] <= MDR	

流水线：结构冒险

- 增加硬件资源：

- ALUOut计算分解为ALU和PCAdd



流水线：结构冒险

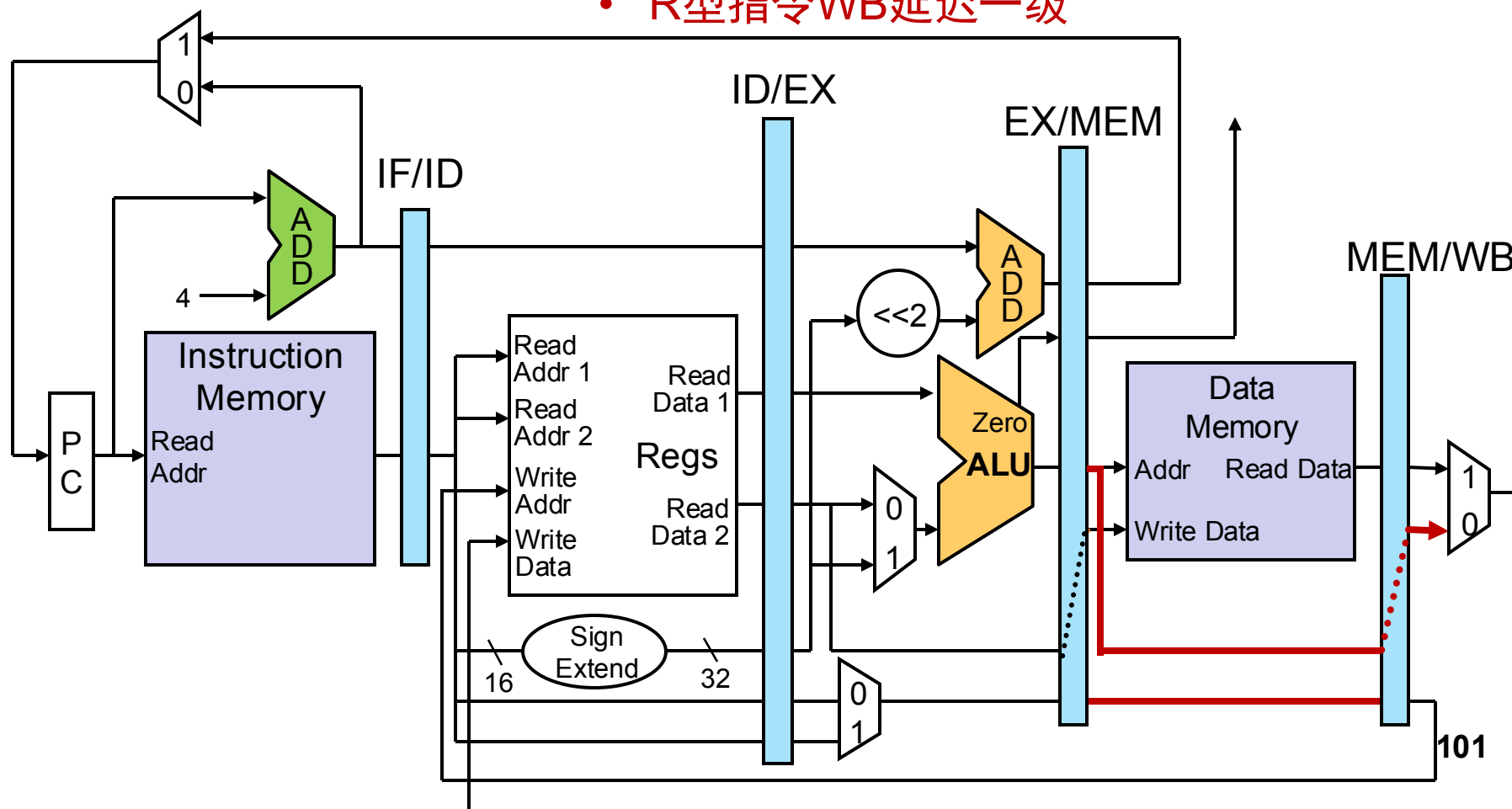
- 寄存器堆写入冲突

- R型与load冲突，R型指令WB延迟一级

流水线	R类型	读存储器	写存储器	分支
IF	IR <= MemInst[PC]; PC <= PC + 4			
ID	op <= IR[31:26]; A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]];			
EX	ALUOut <= A op B	ALUOut <= A + signext(IR[15:0])		ALUOut <= A - B PCAdd <= PC + signext(IR[15:0]) << 2
MEM	Reg[IR[15:11]] <= ALUOut	MDR <= MemData[ALUOut];	MemData[ALUOut] <= B	if (Zero) PC <= PCAdd
WB	 Reg[IR[15:11]] <= ALUOut	Reg[IR[20:16]] <= MDR		100

流水线：结构冒险小结

- 增加硬件资源：
 - ALUOut计算分解为ALU和PCAdd
 - 额外的加法器用于PC+4
 - 分离的指令/数据存储器
 - R型指令WB延迟一级



流水线：更多冒险

- 结构冒险通过增加资源（IM, DM分离）、计算指令增加空的MEM操作（使得各类指令对齐）等方式解决
- 寄存器堆读和写是否冲突？若读和写目标相同？是否使用了正确的数据？（数据冒险）

流水线	R类型	读存储器	写存储器	分支
IF	IR <= MemInst[PC]; PC <= PC + 4			
ID	op <= IR[31:26]; A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]];			
EX	ALUOut <= A op B	ALUOut <= A + signext(IR[15:0])		ALUOut <= A - B PCAdd <= PC + signext(IR[15:0]) << 2
MEM		MDR <= MemData[ALUOut];	MemData[ALUOut] <= B	if (Zero) PC <= PCAdd
WB	Reg[IR[15:11]] <= ALUOut	Reg[IR[20:16]] <= MDR		102

流水线：更多冒险

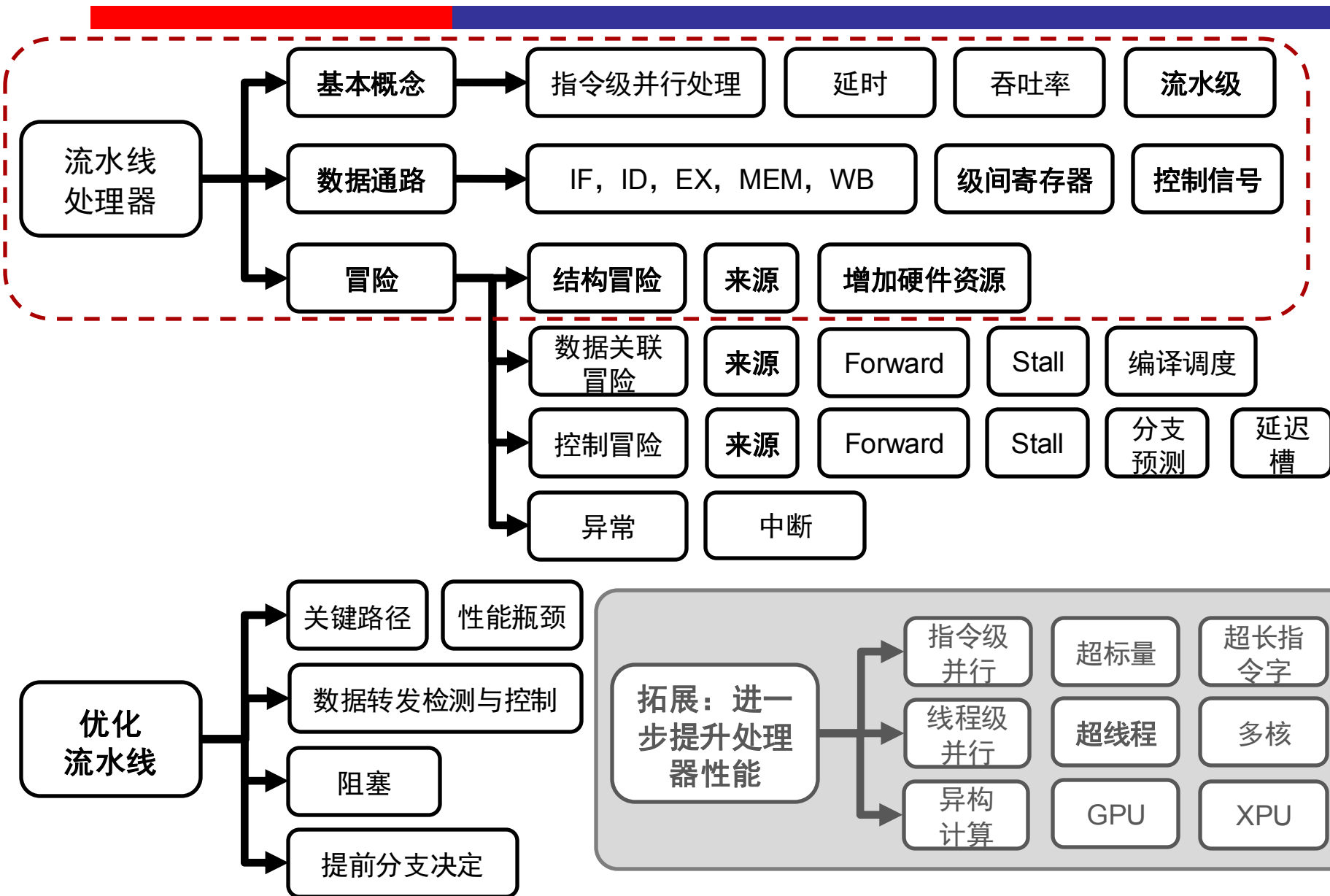
- 结构冒险通过增加资源（IM, DM分离）、计算指令增加空的MEM操作（使得各类指令对齐）等方式解决
- PC仍然存在冲突？是否执行了正确的指令？PC寄存器是否正确？（控制冒险）

流水线	R类型	读存储器	写存储器	分支
IF	IR <= MemInst[PC]; PC <= PC + 4			
ID	op <= IR[31:26]; A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]];			
EX	ALUOut <= A op B	ALUOut <= A + signext(IR[15:0])		ALUOut <= A - B PCAdd <= PC + signext(IR[15:0]) << 2
MEM		MDR <= MemData[ALUOut];	MemData[ALUOut] <= B	if (Zero) PC <= PCAdd
WB	Reg[IR[15:11]] <= ALUOut	Reg[IR[20:16]] <= MDR		103

小结

- **单周期 vs 多周期 vs 流水线**
 - 资源利用率、吞吐率
- **MIPS指令的流水线方法**
 - 五级流水线：IF→ID/RF→EX→MEM→WB
 - 流水级间寄存器：保持中间数据和控制信号
- **流水线中可能具有冒险**

知识点框图



大作业说明

- 汇编大作业（本周发布，见网络学堂）
 - DDL：第19周周一（2025年1月13日）23:59
 - 原则上DDL不会延长，如有特殊情况请和助教、老师商量
- 大作业查重说明
 - 包括数字逻辑大作业和汇编大作业
 - 鼓励讨论，但是要求所有代码与实验报告均独立完成
 - 严禁抄袭，包括“不恰当地”使用ChatGPT
 - 如果被发现抄袭往年“版本”或互相抄袭，会被要求向助教、老师单独解释自己写的代码，如果说不清楚我们将会上报教学办处理

第二次习题课说明

- 第二次习题课

- 时间：11月30日（周六）19:00 – 21:00

- 地点：系馆3层报告厅

- 内容：单周期多周期处理器回顾与部分习题讲解，汇编编程基础、MARS使用与大作业要求

- 课后进行答疑，录屏将上传至网络学堂

作业

- 要求

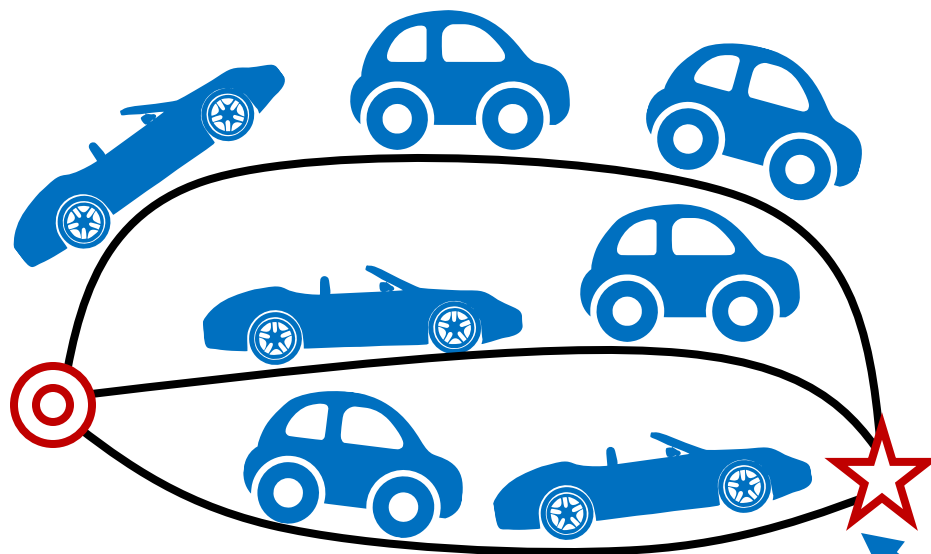
- 上传电子版至网络学堂
- 不要抄袭或迟交

- 作业

- 第一部分：教材第7章习题1，2，3，4
 - 以网络学堂作业附件为准
- 第二部分：参考课件中数据通路与控制信号的例子，补充sw指令的控制信号，并指出各控制信号在哪个流水阶段（IF、ID、EX、MEM、WB）被使用

影响性能的因素

- 高速公路带来的启发



并行执行多条指令

- 多条路多辆车
- 优点：更高的吞吐量
- 现在的瓶颈？

